

TRIBHUVAN UNIVERSITY

FACULTY OF HUMANITIES AND SOCIAL SCIENCES

BACHELOR OF COMPUTER APPLICATION (BCA)

INTERNET OF THINGS



BCA

CACS460 INTERNET OF THINGS (IOT)

TABLE OF CONTENTS

CHAPTER 1 INTRODUCTION OF IOT	1
1.1 Definition and Characteristics of IOT	1
1.1.1 IoT Definition	1
1.1.2 Characteristics of IOT	2
1.2 Physical and Logical Design of IoT	2
1.2.1 Physical Design of IoT	2
1.2.2 Logical design of IoT	7
1.3 IoT Enabled Technologies	11
1.3.1 Wireless sensor network.....	11
1.3.2 Cloud computing:	12
1.3.3 Big Data Analytics	13
1.3.4 Communications protocol:	14
1.3.5 Embedded systems	14
1.4 IoT and M2M	14
1.4.1 Machine to Machine (M2M)	14
1.4.2 Difference between IoT and M2M	15
1.4.3 SDN and NFV for IoT :	16
1.5 Domain specific IoTs:	18
1.5.1 Home Automation	18
1.5.2 Cities.....	20
1.5.3 Environment	22
1.5.4 Energy	24
1.5.5 Retail	26
1.5.6 Logistics	27
1.5.7 Agriculture.....	28
1.5.8 Industry.....	29
1.5.9 Health and Lifestyle	30
CHAPTER 2 SENSOR, ACTUATORS AND INTERFACING.....	31
2.1 Roles of Sensors and actuators.....	31
2.1.1 Sensors.....	31
2.1.2 Actuators	32
2.1.3 The Role of Sensors and Actuators in IoT	32
2.1.2 Types of Sensor.....	32

2.1.2.1 Active and Passive sensor	32
2.1.2.2 Analog and Digital Sensor.....	32
2.1.2.3 Contact and No-Contact Sensor	32
2.1.2.4 Absolute and relative	33
2.2 Working of Sensors:.....	33
2.2.1 Position Sensors.....	33
2.2.2 Occupancy and Motion Sensors	34
2.2.3 Velocity and Acceleration Sensors.....	35
2.2.4 Force and Pressure Sensors	36
2.2.5 Flow Sensors	36
2.2.6 Acoustic Sensors	37
2.2.7 Humidity Sensors	38
2.2.8 Light Sensors	38
2.2.9 Radiation Sensors	38
2.2.10 Temperature Sensors	39
2.2.11 Chemical Sensors	39
2.2.12 Biosensor Sensors.....	40
2.2.13 Camera Sensors	40
2.3 Development boards:.....	41
2.3.1 Arduino.....	41
2.3.2 Raspberry Pi	53
2.3.3 Interface and programming using Python	56
CHAPTER 3 IOT PLATFORM DESIGN METHODOLOGY	57
3.1 Case Study on IoT System for Weather Monitor	58
CHAPTER 4 DATA AND ANALYTICS FOR IOT.....	60
4.1 An Introduction to Data Analytics for IoT.....	60
4.1.1 Structured Versus Unstructured Data.....	60
4.1.2 Data in Motion Versus Data at Rest	62
4.1.3 IoT Data Analytics Overview.....	63
4.1.4 IOT Data Analytics Challenges.....	64
4.2 Machine Learning	65
4.2.1 Machine Learning Overview	65
4.2.2 Supervised Learning.....	66
4.2.3 Unsupervised Learning.....	67

4.2.4 Neural Networks.....	69
4.2.5 Machine Learning and Getting Intelligence from Big Data	71
4.2.6 Predictive Analytics.....	72
4.3 Big Data Analytics Tools and Technology	73
4.3.1 Massively Parallel Processing Databases	74
4.3.2 NoSQL Databases	75
4.3.3 Hadoop	75
4.3.4 Yet Another Resource Negotiator (YARN)	77
4.3.5 The Hadoop Ecosystem.....	77
4.4 Edge Streaming Analytics.....	81
4.4.1 Comparing Big Data and Edge Analytics.....	81
4.4.2 Edge Analytics Core Functions	82
4.5 Network Analytics.....	86
CHAPTER 5 IOT PHYSICAL SERVERS AND CLOUD OFFERING	88
5.1 Cloud storage models of IoT Systems.....	88
5.2 Communication APIs of IoT Systems.....	88
CHAPTER 6 SECURING IOT SYSTEMS.....	90
6.1 IoT Security Challenges	90
6.1.1 Erosion of Network Architecture	90
6.1.2 Pervasive Legacy Systems	91
6.1.3 Insecure Operational Protocols.....	91
6.1.4 Modbus.....	92
6.1.5 DNP3 (Distributed Network Protocol).....	92
6.1.6 ICCP (Inter-Control Center Communications Protocol)	93
6.1.7 OPC (OLE for Process Control).....	93
6.1.8 International Electro technical Commission (IEC) Protocols	94
6.1.9 Other Protocols.....	95
6.1.10 Device Insecurity.....	95
6.1.11 Dependence on External Vendors	97
6.1.12 Security Knowledge	97
6.2 IoT System's Security Practices.....	98
REFERENCE.....	100

CHAPTER 1

INTRODUCTION OF IOT

Introduction

Internet of Things (IoT) is a concept which enables communication between internetworking devices and applications, whereby physical objects or ‘things’ communicate through the Internet. The concept of IoT began with things classified as identity communication devices. Radio Frequency Identification Device (RFID) is an example of an identity communication device. Things are tagged to these devices for their identification in future and can be tracked, controlled and monitored using remote computers connected through the Internet.

The concept of IoT enables, for example, GPS-based tracking, controlling and monitoring of devices; machine-to-machine (M2M) communication; connected cars; communication between wearable and personal devices and Industry

1.1 Definition and Characteristics of IOT

1.1.1 IoT Definition

The Internet is a vast global network of connected servers, computers, tablets and mobiles that is governed by standard protocols for connected systems. It enables sending, receiving, or communication of information, connectivity with remote servers, cloud and analytics platforms.

Thing in English has number of uses and meanings. In a dictionary, thing is a word used to refer to a physical object, an action or idea, a situation or activity, in case when one does not wish to be precise. Example of reference to an object is—an umbrella is a useful thing in rainy days. Streetlight is also referred to as a thing. Example of reference to an action is— such a thing was not expected from him. Example of reference to a situation is—such things were in plenty in that regime.

Thus, combining both the terms, the definition of IoT can be explained as follows:

Internet of Things means a network of physical things (objects) sending, receiving, or communicating information using the Internet or other communication technologies and network just as the computers, tablets and mobiles do, and thus enabling the monitoring, coordinating or controlling process across the Internet or another data network.

Another source, defines the term IoT as follows:

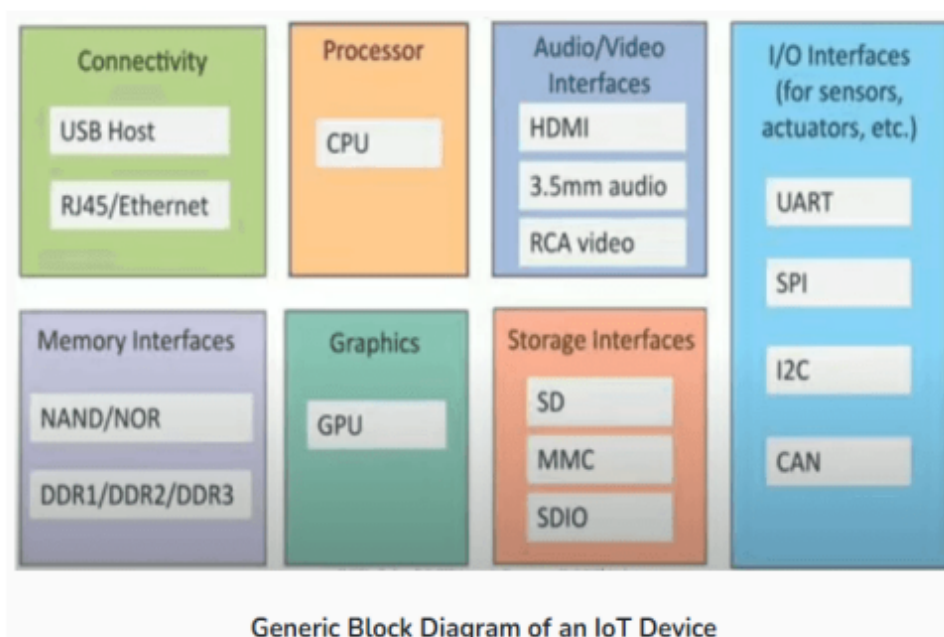
Internet of Things is the network of physical objects or 'things' embedded with electronics, software, sensors and connectivity to enable it to achieve greater value and service by exchanging data with the manufacturer, operator and/or other connected devices. Each thing is uniquely identifiable through its embedded computing system but is able to interoperate within the existing Internet infrastructure.

1.1.2 Characteristics of IOT

- **Dynamic and self-Adapting:** IoT devices and systems may have the capability to dynamically adapt with the changing contexts and take actions based on their operating condition Eg: Surveillance cameras can adapt their modes based on whether it is day or night.
- **Self – Configuring:** IoT devices may have self-Configuring capability allowing a large number of devices to work together to provide certain functionality.
- **Interoperable communication protocols:** IoT Devices may support a number of interoperable communication protocols and can communicate with other devices and also with the infrastructure.
- **Unique Identity:** Each IoT devices has a unique identity and a unique identifier. IP address, URI).IoT systems may have intelligent interfaces which adapt based on the context, allow communication with users and the environment contexts.
- **Integrated into information network:** IoT devices are usually integrated into the information network that allows them to communicate and exchange data with other devices and systems.

1.2 Physical and Logical Design of IoT

1.2.1 Physical Design of IoT



The diagram shows the four layers of a typical IoT system:

- **Perception Layer:** This layer is responsible for collecting data from the environment using sensors. The sensors in this layer can be active or passive, contact or non-contact, analog or digital.
- **Network Layer:** This layer is responsible for transmitting the data collected by the sensors to the cloud or other processing devices. The network layer can use wired or wireless communication technologies.
- **Processing Layer:** This layer is responsible for processing the data collected by the sensors. The processing layer can be located in the cloud or on the edge of the network.
- **Actuation Layer:** This layer is responsible for controlling actuators in the environment. The actuators in this layer can be used to control lights, motors, valves, and other devices.

The diagram also shows the following components of an IoT system:

- **Sensors:** Sensors are devices that collect data from the environment. Sensors can measure a variety of physical quantities, such as temperature, pressure, humidity, light, and motion.
- **Actuators:** Actuators are devices that control the environment. Actuators can be used to turn lights on and off, open and close valves, and move motors.
- **Gateways:** Gateways are devices that connect the sensors and actuators to the network. Gateways can be used to convert the data from the sensors to a format that can be transmitted over the network.
- **Cloud:** The cloud is a network of servers that store and process data. The cloud can be used to store the data collected by the sensors and to process the data to generate insights.

The physical design of an IoT system is an important part of the overall design of the system. The physical design must be carefully considered to ensure that the system meets the requirements of the application and the budget.

1.2.1.1 Things of IoT

The “Things” in IoT usually refers to IoT devices which have unique identities and can perform remote sensing, Actuating and monitoring capabilities. IoT devices can exchange data with other connected devices and applications (directly or indirectly), or collect data from other devices and process the data locally or send the data to Centralized servers or cloud based applications back ends for processing the data or from some task locally and other task within the IoT infrastructure,

based on temporal and space constraints (ie : Memory, processing calibrators, communication latencies and speed and deadlines).

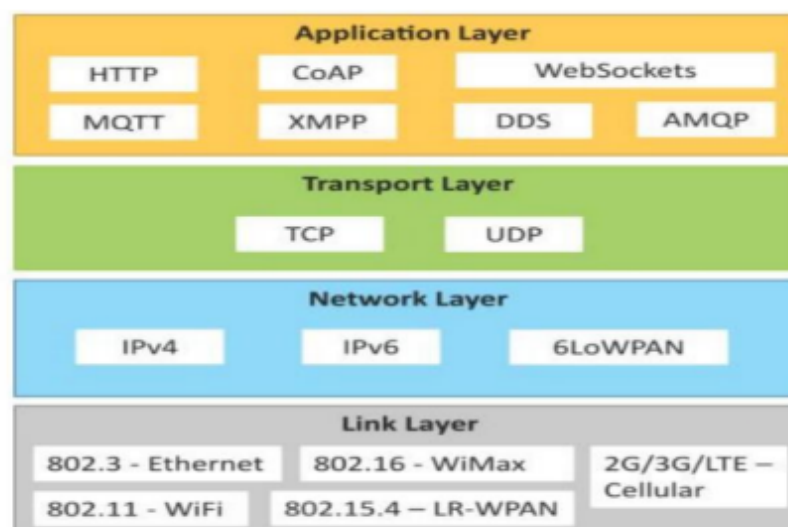
An IoT device may consist of several interfaces connections to other devices, both wired and wireless. These include

- IoT interfaces for sensors
- Interfaces for internet connectivity
- Memory and storage interfaces
- Audio video interfaces.

An IoT Device can collect various types of data from the onboard or attached sensors, such as temperature, humidity, and light intensity.

IoT devices can also be varied types, for instance, wearable sensors, smart watches, LED light automobiles and industrial machines. Almost all I would advise generate data in some form or the other which when processed by Data Analytics systems leads to Useful information to guide further actions locally or remotely.

1.2.1.2 IoT Protocol



Application layer:

Application layer protocol define how the application interfaces with the lower layer protocols to send the data over the network. Data are typically in files, is encoded by the application layer protocol and encapsulated in the transport layer protocol .Application layer protocol enable process-to-process connection using ports.

- **Http:** Hypertext transfer protocol is the application layer protocol that forms the foundations of world wide web http includes, ,commands such as GET, PUT, POST, DELETE, HEAD,

TRACE, OPTIONS etc. The protocol follows a request-response model where client sends request to server using the http, commands. Http is a stateless protocol and each http request is independent from request and http client can be a browser or an application running on the client example and application running on an IoT device, mobile, mobile applications or other software.

- **CoAP:** Constrained application protocol is an application layer protocol for machine to machine application M2M meant for constrained environment with constrained devices and constrained networks. Like http CoAP is a web transfer protocol and uses a request- response model, however it runs on the top of the UDP instead of TCP CoAP uses a client –server architecture where client communicate with server using connectionless datagrams. It is designed to easily interface with http like http, CoAP supports method such as GET, PUT, and DELETE.
- **Web socket:** Web socket protocol allows full duplex communication over a single socket connections for sending message between client and server. Web socket is based on TCP and Allows streams of messages to be sent back and forth between the client and server while keeping the TCP connection open. The client can be a browser, a mobile application and IoT device
- **MQTT:** Message Queue Telemetry Transport it is a lightweight message protocol based on public -subscribe model MQTT uses a client server Architecture by the clients such as an IoT device connect to the server also called the MQTT broker and publishers message to topic on the server. The broker forward the message to the clients subscribed to topic MQTT is well suited for constrained and environments.
- **XMPP:** Extensible Messaging and Presence Protocol it is a protocol for real-time communication and streaming XML data between network entities XMPP powers wide range of applications including messaging, presence, data syndication, gaming multiparty chat and voice / voice calls. XMPP Allows sending small chunks of XML data from one network entity to another in real time. XMPP supports both client to server and server –client communication path.
- **DDS:** Data distribution service is the data centric middleware standard for device-to-device machine to machine communication DDS uses a publish subscribe model where publisher example device that generate data create topics to which subscribers per can subscribe publisher is an object responsible for data distributions and the subscriber responsible for receiving published data. DDS provide quality of service (QoS) control and configurable reliability
- **AMQP:** Advanced Message Queuing protocols. it is an open application layer protocol for business messaging. AMQP support point to point and publish - subscribe model routing and queuing. AMQP broker receive message from publishers example devices or applications that generate data and about them over connections to consumers publishers publish the message to exchange which then distribute message copies to queues.

Transport layer:

The Transport layer protocols provides end-to-end message transfer capability independent of the underlying network. The message transfer capability can be set up on connections, either using handshake or without handshake acknowledgements. Provides functions such as error control, segmentation, flow control and congestion control.

- **TCP:** Transmission control protocol is the most widely used to transport layer protocol that is used by the web browsers along with HTTP, HTTPS application layer protocols email program (SMTP application layer protocol) and file transfer protocol. TCP is a connection Oriented and stateful protocol while IP protocol deals with sending packets, TCP ensures reliable transmissions of packets in order. TCP also provide error deduction capability so that duplicate packets can be discarded and low packets are retransmitted. The flow control capability ensures that the rate at which the sender since the data is now to too to high for the receiver to process.
- **UDP:** unlike TCP, which requires carrying out an initial setup procedure, UDP is a connection less protocol. UDP is useful for time sensitive application they have very small data units to exchange and do not want the overhead of connection setup. UDP is a transactions oriented and stateless protocol. UDP does not provide guaranteed delivery, ordering of messages and duplicate eliminations.

Network / internet layer:

The network layer are responsible for sending of IP datagrams from the source network to the destination network. This layer Performs the host addressing and packet routing. The datagrams contains a source and destination address which are used to route them from the source to the destination across multiple networks. Host Identification is done using the hierarchy IP addressing schemes such as ipv4 or IPv6.

- **IPV4:** Internet protocol versions for open parents close (IPV4) is there most deployed internet protocol that is used to identify the device is on a network using a hierarchy latest schemes. It uses 32 bit addresses scheme that allows total of 2^{32} address. As more and more devices got connected to the internet. The Ipv4 has succeeded by IPv6.
- **IPv6:** It is the newest versions of internet protocol and successor to IPv4. IPv6 uses 128 bit address schemes that are lost total of 2^{128} are 3.4×10^{38} address.
- **6LoWPAN:** IPv6 over low power wireless personal area networks brings IP protocol to the low power device which have limited processing capability it operate in the 2.4 GHz frequency range and provide the data transfer rate off to 50 kb/s.

Link Layer:

Link Layer protocols determine how the data is physically sent over the network's physical layer or medium (example copper wire, electrical cable, or radio wave). The Scope of The Link Layer is the Last Local Network connections to which host is attached. Host on the same link exchange data packets over the link layer using the link layer protocol. Link layer determines how the packets are coded and signaled by the hardware device over the medium to which the host is attached.

- **802.3 Ethernet:** 802.3 is a collection of wired Ethernet standards for the link layer. For example 802.3 10BASE5 Ethernet that uses coaxial cable as a shared medium, 802.3.i is standard for 10 BASET Ethernet over copper twisted pair connection, Standards provide data rates from 10 Mb/s to 40 gigabits per second and the higher. The shared medium in Ethernet can be a coaxial cable twisted pair wire or and Optical fiber. Shared medium carries the communication for all the devices on the network.
- **802.11- WI-FI:** IEEE 802.11 is a collection of wireless Local area network. (WLAN) communication standards, including extensive descriptions of the link layer. For example 802.11a operate in the 5 GHz band, 802.11b and 802.11g operate in the 2.4 GHz band. 802.11ac operates in the 5G hertz band.
- **802.16 WiMAX:** IEEE 802.16 is a collection of wireless broadband and Standards, including extensive descriptions for the link layer also called WiMAX wimax standard provides a data rates from 1.5 Mb/s to 1Gb/s the recent update provides data rates of hundred megabits per second for mobile station.
- **802.15.4 LR-WPAN:** IEEE 802.15.4 is a collection of standard for low rate wireless personal area network(LR-WPAN).These standard form the basis of specifications for high level communication Zigbee. LR-WPAN standards provide data rates from 40 k b/ s. These standards provide low cost and low speed Communications for power constrained devices.
- **2G / 3G / 4G mobile communications:** These are the different generations of mobile communication standards including second generation (2G including GSM and CDMA). 3rd Generation (3G including UMTS and CDMA2000) and 4th generation 4G including LTE.

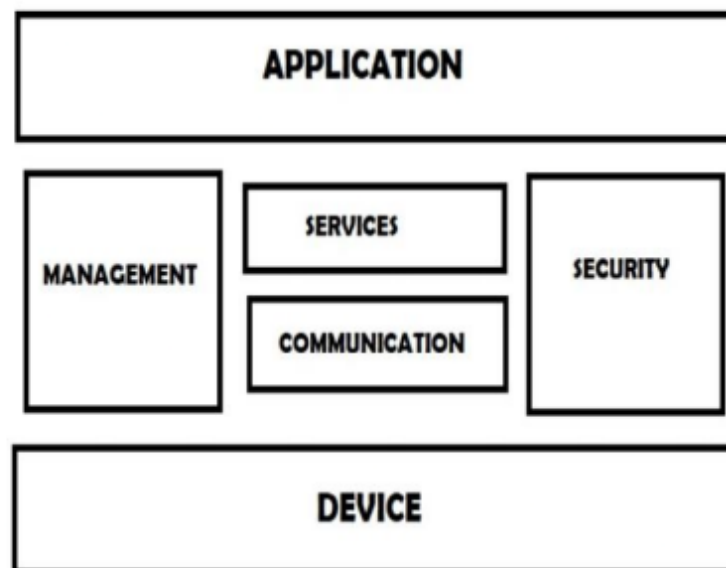
1.2.2 Logical design of IoT

A logical design for an IoT system is the actual design of how its components (computers, sensors, and actuators) should be arranged to complete a particular function. It doesn't go into the depth of describing how each component will be built with low-level programming specifics.

IoT logical design includes:

1.2.2.1 IoT functional block

An IoT system comprises of a number of functional blocks that provide the system the capabilities for identification, sensing, actuation, communication and Management.



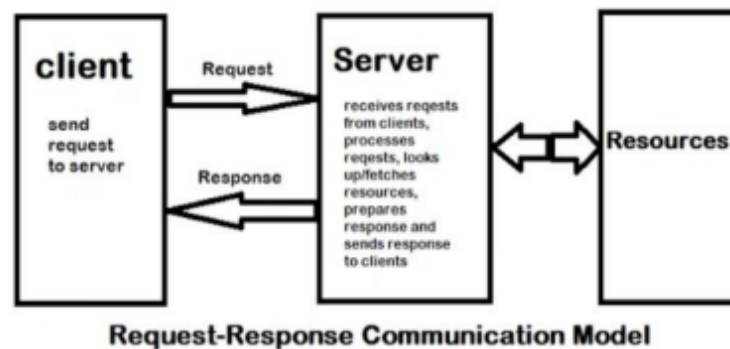
The function blocks are described as follows

- **Devices:** An IoT system comprises of the devices that provide sensing, actuation, monitoring and control function.
- **Communication:** communication block handle the communication systems
- **Services:** An IoT system uses various types of IoT services such as services for device monitoring, device control services, data publishing services and services for device Discovery.
- **Management:** Functional blocks provide various functions to govern the IoT system
- **Security:** Security functional block security IoT system and by providing functions such as application authorization message and content integrity and data security.
- **Application:** IoT application provides and interface that the user can used to control and monitor various aspects of the IoT system. Application also allow users to view the system status and view or analyze the processed to data.

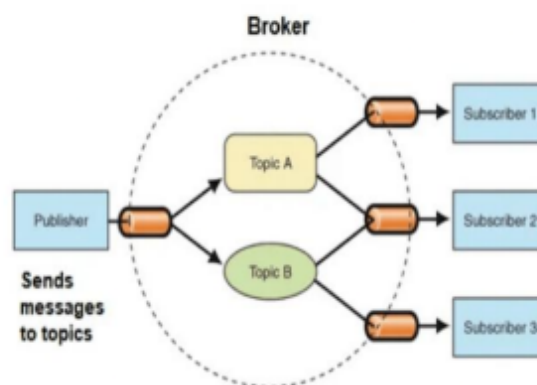
1.2.2.2 IoT communication model

- **Request response:** Request-response is a Communications model in which the client sends request to the server and the server responds to the requests. when the server receives a request it decides how to respond, if it shows the data retrieved resources definitions for the response , and then send the response to the client. Access to response model is a stateless

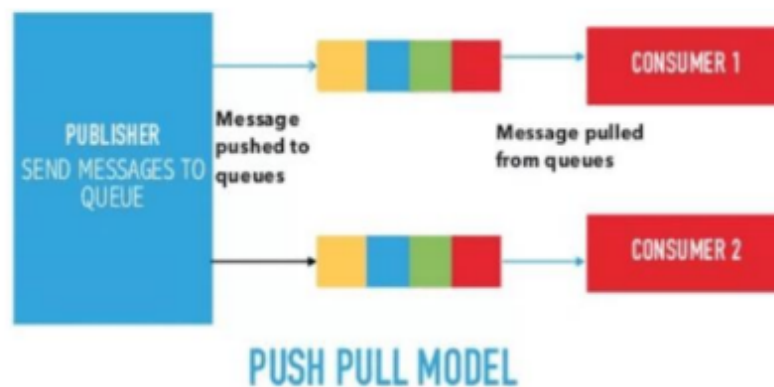
communication model and each request response per is independent of others the crime and server interactions in the request response model.



- **Publish - Subscribe:** Respect is a communication model that involve Publishers brokers and consumers. Publishers are the source of data. Publishers send the data to the topics which is managed by the broker. Publishers are not aware of the consumer. Consumers Subscribe to the topic which are managed by the broker. When the broker receives the data for a topic from the publisher, it send the data to all the subscribed consumers.



- **Push pull:** Push pull is communication model in which the data producers push the data to queues and the consumers pull the data from the queues. Producers do not need to be aware of the consumer. Queues help in decoupling the messaging between the Producers and Consumers. It also act as a buffer which helps in situations when there is a mismatch between the rate at which the produces push data and the rate at which the consumers full the data



- **Exclusive pair:** Exclusive pair is a bi directional, fully duplex communication model that uses a persistent connections between the client and the server. Once the condition is setup it remains open until the client sends a request to close the connection. client and server can send messages to each other after connection setup. Exclusive pair is a state full Communications model and the server is aware of all the open connections.



1.2.2.3 IoT communication APIs

REST- based communication API:

Representational state transfer is a set of architectural principles by which you can design web service and Web API that focus on a system resources and how resources states and addressed the transferred. REST API follow the request- response communication model.

The REST architectural constraints apply to the components, connectors, and data elements.

- **Client server:** The principle behind the client-server conference separations of concerns for example client should not be concerned with the storage of data which is their concern of the server. Similarly the server should not be concerned about the user interface which is a concern of the client. Separation allows client and server to be independently deployed and updated.
- **Stateless:** Each request from client to server must contain all the information necessary to understand the request , and cannot take advantage of any stored context on the server
- **Catchable:** Catch constrain requires that the data within the response to a request be implicitly or explicitly labeled as catchable or non-catchable. Then a client cache is given the right to reuse that response data for later, equivalent requests. Completely eliminate some attractions and improve efficiency and scalability.
- **Layered system:** System constraint come off constraints, constrains the behavior of components such that each component cannot see beyond the immediate layer with which

they are interacting. Example client cannot tell whether it is connected directly to the end server or to an intermediary along the way system scalability can be improved allowing intermediaries to respond to request instead of tender server.

- **Uniform interface:** Uniform interface constraints requires that the method of communication between client and server must be uniform. Resources are identified in the request and separate from the representation of the resource that are returned to the client. When climbing holds a representation of your resource it has all the information required to update or delete the resource
- **Code on demand:** Service can provide executable code script for clients to execute in their context.

Web Socket based communication API:

Web Socket API allow bi directional, full duplex communication between client and server. Unlike request-response API allow full duplex communication and do not require new connection to be set up for each message to be sent. Web socket communication begins with connection setup request send by the client to the server. The request is sent over http and the server interprets it as an upgrade request. If the server support protocol response to the website handshake response after the connection setup the client and the server can send data or messages to each other in full duplex model. Web Socket API reduce network traffic and latency as there is no overhead for connection setup and determination records to each message.

1.3 IoT Enabled Technologies

It is enabled by several Technologies including wireless sensor networks, cloud computing big Data Analytics, embedded system, security protocols and architectures, communication protocols, web service, mobile internet and semantic search engine.

1.3.1 Wireless sensor network

Wireless sensor network (wsn) comprise of distributed devices with the sensor which are used to monitor the environmental and physical conditions. A WSN consists of a number of end nodes and routers and a coordinator. End nodes have several sensors attached to them. End node can also act as a routers. Routers are responsible for routing the data packet from end nodes to the coordinator. The coordinator node collect the data from all the notes coordinator also act as a Gateway that connects the WSN to the internet.

IoT systems are described as follows

- Weather monitoring system using WSN in which the nodes collect temperature, humidity and other data which is aggregated and analyzed.
- Indoor air quality monitoring system using WSN to collect data on the indoor air quality and connections of various gases.
- Soil moisture monitoring system using WSN to monitor soil moisture at various location.
- Surveillance systems use WSN for collecting surveillance data(motion detection data)
- Smart grid use wireless sensor network for monitoring the grid at various point.
- Structural health monitoring systems use WSN to monitor the health of structure by writing vibration data from sensor nodes deployed at various points in the structure.

1.3.2 Cloud computing:

Cloud Computing is a transformative computing paradigm that involves delivering applications and services over the internet. Cloud Computing involves provisioning of computing networking and storage resources on demand and providing these resources as metered services to the users, in a “pay as you go” model. Cloud Computing resources can be provisioned on demand by the user without requiring interactions with the Cloud Service Provider. The process of provisioning resources used automatic Cloud Computing resources can be accessed then it worked using standard access mechanism that provide platform-independent access through the use of heterogeneous client platforms such as workstations laptops tablets and Smartphones the computing and storage resources provided by Cloud Service Provider our food to serve multiple user using multi Tenancy. Multi-tenant aspects on the multiple users to be served by the same physical hardware.

Cloud Computing services are offered to user in different forms

- **Infrastructure as a service (IAAS):** IaaS provides the user the ability provision computing and storage resources. These resources are provided to the users as virtual machine instances and virtual storage. Users can start, stop configure and manage the virtual machines instance on the virtual storage using can deploy operating systems and applications on their choice on the actual resources provisions in the cloud. Cloud Service Provider manages the underlying infrastructure.
- **Platform as a service (PaaS):** platform as a service provides the user the ability to develop and deploy application in the cloud using the deployment tool application programming interfaces API, software libraries and services provided by the Cloud Service Provider. The Cloud Service

Provider manages the underlying cloud infrastructure including servers, network, operating systems and storage.

- **Software as a service (SaaS):** Provide the user a complete software applications of the user interface to the application itself. The Cloud Service Provider manage the underlying cloud infrastructure including server, network storage and application software, and the user is unaware of the underlying architecture of the cloud. Applications are provided to the user through a thin client interface example Browser application. SaaS applications are accessed from various client smartphones running different operating system.

1.3.3 Big Data Analytics

Big data is defined as collections of data set whose volume, velocity in terms of its temporal variations)or variety, is so large that it is difficult to store, manage, process and analyze the data using traditional database and data processing tools. Big Data Analytics involving several steps starting from Data cleaning data munging data processing and visualization.

Some examples of big data generated by IoT systems are described as follows

- Sensor data generated by IoT system such as weather monitoring stations
- Machine sensor data collected from sensor embedded in Industrial and energy system for monitoring their files and protecting failure
- Health and fitness data generated by IoT devices such as wearable fitness band.
- Data generated by IoT system for Location tracking of vehicle.
- Data generated by retail inventory monitoring system.

Characteristics of data include:

- **Volume:** Through there is no fixed threshold for volume of data to be considered as big data, however the term big data is used for massive scale data that is difficult to store, manage and process using traditional data bases and data processing architecture. The volume of data generated by modern IT, industrial and Healthcare systems for example is a growing exponentially driven by the lowering cost of data storage and processing architectures and the need to extract valuable insights from the data to improve business processes, efficiency and services to consumer.
- **Velocity:** Velocity is another important characteristics of big data and the primary reasons for exponential growth of data velocity of the data of a store how fast the data is generated and how

frequently it varies. Modern IT Industrial and other systems are generating data at increasing the highest speeds.

- **Variety:** Variety refers to the forms of the data. Big data comes in for different forms such as structured or unstructured data including text data, audio, video and sensor data.

1.3.4 Communications protocol:

Communications protocols form the backbone of IoT system and enable network connectivity and coupling to applications. Communications protocols allow device to exchange data over the network. These protocols define the data exchange formats and data encoding schemes for devices and routing of packets from source to destination. Other function of the protocol include sequence control flow control and transmissions of lost packet.

1.3.5 Embedded systems

An Embedded system is computer system that has computer hardware and software embedded perform specific task. In contrast to general purpose computers or personal computers which can perform various types of tasks, embedded systems are designed to perform a specific set of tasks. Embedded system include Microprocessor and Microcontroller memory Ram ROM cache networking units (Ethernet WI-FI adaptor) input/output unit display keyboard , display and storage such as Flash Memory some embedded system have specialist processes such as digital signal processor DSP graphic processor and application.

1.4 IoT and M2M

1.4.1 Machine to Machine (M2M)

Machine to machine (M2M) refers to networking of Machines for the purpose of remote monitoring and control and data exchange. The end-to-end architecture for M2M systems comprising of M2M area networks, Communications Network and application domain.

An M2M area network comprises of machines which have embedded hardware module for sensing actuation and communication. Various Communication protocols can be used for M2M local area network such as Zigbee, Bluetooth, Modbus M-bus, wireless, power LINE Communication ,6LoWPAN.

These Communications protocols provide connectivity between M2M nodes within and M2M area network. The Communications Network provides connectivity to remote m2m area network.

Communication network can use wired or wireless network. The M2M area network use either proprietary or non IP based protocol.

The communication between the M2M nodes and the M2M Gateway is based on the communication protocol. M2M Gateway protocol translation to enable IP connectivity for M2M. M2M Gateway act as a proxy performing translation from / to native protocol to M2M area network.

M2M data is gathered into point solution such as enterprise applications, service management application for remote monitoring applications. M2M has various application domain such as smart metering, Home Automation, industrial Automation, smart grid.

1.4.2 Difference between IoT and M2M

The difference between m2m and IoT are described as follows:

- **Communication protocols:**

M2m and IoT can differ in how the communication between the machines are device happens. M2M uses other proprietary or not IP based communication protocol for communication with in the M2M area networks. Commonly uses M2m protocol include zigbee, Bluetooth, ModBus, wireless M-Bus, power line communication.

The focus of communication in M2M is usually on the protocols below the network layer. Focus of communication in IoT is usually a protocol in network layer such as http web sockets, MQTT, XMPP, DDS, AMQP.

- **Machines in M2M vs Things in IoT:**

The “things” IoT refers to Physical objects that have unique identifier and can sense and communicate with the external environment or their internal physical status. The unique identifiers the things in IoT are the IP addresses. Things have software component for accessing processing and storing sensor information on controlling actuator connector. IoT system can include IoT devices of various types such as fire alarms, door alarms, lighting control devices.

- **Hardware versus software emphasis:**

While the emphasis of M2M is more on hardware with embedded modules, the emphasis modules, the emphasis of IoT is more on software. IoT devices run specialist software sensor Data Collection, data analysis and interfacing with cloud through IP based communication.

- **Data collection and analysis:**

M2M data is collected in point solutions and often in on premises storage infrastructure. In contrast to M2M, the data in IoT is connected in the cloud. The analytical component analysis the data and stores the result in the cloud database. Data and analysis results are visualized with the

Components of IoT applications run distributed algorithms on a large number of virtual machine that require huge amounts of data exchange between virtual machines. Such computing environment require highly scalable and easy to manage network architectures with minimal manual configuration which is becoming raising a difficult with a conventional networks.

Key Elements of SDN and follow:

- ***Centralized network controller:*** with Decoupled control and the data plan and centralized network controller, the network administrator can rapidly configure the network. SDN applications can be deployed Programmable open API. This speed have innovation as the network status no longer need to wait for other device vendors to embed features in their proprietary hardware.
- ***Programmable open APIs:*** SDN architecture propose Programmable open API for interface between the SDN application and control layers with these open API is various network services can be implemented such as routing quality of services access.
- ***Standard communication interface (Open Flow)*** SDN architecture uses a standard communication interface between the control and infrastructure layers. Open flow, which is Defined by the open networking Foundation is the broadly accepted SDN protocol for the southbound interface. Open flow, the forwarding plan of the network devices can be directly accessed and manipulated.

Open floor uses the concept of close try different network traffic based on three different rules. Floor can be programmed statically and dynamically by the SDN control software. Components of and open flow switch comprising one or more tables and their group table which perform packet lookup and forwarding, and open flow channels to an external Controller System open flow protocol is implemented at both sides of the interface between the controller and the network devices.

1.4.3.2 Network function virtualization:

Network function virtualization is a technology that leverages virtualization to consolidate the heterogeneous network devices on to industry standard high-volume service switches and storage. NFV is complementary to SDN as NFV can provide at the infrastructure on which SDN can run. NFV and SDN mutually beneficial to each other but not dependent. Network functions can be virtualized without SDN which very beneficial to each other but not dependent

Elements of architecture as follows:

Virtualize to network function VNF:

VNF is a software implementations of a network function which is capable of running over the enough Infrastructures

NFV Infrastructure NFVI:

It into Computer Network and storage resources that are visualized.

NFV management and orchestrations: Orchestrations focuses on all visualization specific management task and the covers the orchestrations and lifecycle management of physical and / or software resources that support the infrastructure utilization and the lifecycle management of VNF.

NFV comprises of network functions implemented in software that run on Virtualized resources in the cloud. NFC enabled separations of network function which are implemented in software from the underlying hardware.

1.5 Domain specific IoTs:

1.5.1 Home Automation

Home automation is a domain of IoT that uses sensors, actuators, and software to control devices in the home. This can include everything from lighting and temperature control to security cameras and alarm systems

Here are some of the most common IoT devices used in home automation are Smart Lighting, Smart Appliances, Smoke / Gas Detector, Smart thermostats, Smart locks: Security cameras, Alarm systems rtc.

1.5.1.1 Smart lighting:

Smart lighting for homes helps in saving energy by adapting the lighting to the ambient condition and switching on/off or dimming the lights when needed. Key enabling technologies for smart lighting include solid-state lighting such as LED light and IP enabled the lights.

For solid state lighting solution both spectral and temporal characteristics can be configured to adapt illumination to various needs. Smart lighting solutions for Home achieve energy saving by sensing the human movements and their environments and controlling the lights accordingly.

Wireless enabled and internet connected lights can be controlled remotely from IoT applications such as a mobile or web application. Smart lights with sensor for occupancy, Temperature Lux level etc. can be configured to adapt the lighting based on the ambient conditions sensed, in order to provide a good ambience. In controllable LED lighting system is presented that is embedded with ambient intelligence gathered from a distributed smart wireless sensor network to optimize and control the lighting system to be more efficient and user oriented, A solid state lighting model is implemented on your wireless sensor network that provide services for sensing illumination changes and dynamically adjusting luminary brightness according to user preferences.

1.5.1.2 Smart Appliances

Modern homes have a number of appliances such as TVs, refrigerator, music system, washer / dryer etc. Managing and controlling these appliances can be cumbersome with the each appliance having its own controls or remote controls. Smart appliance make the management easier and also provide status information to the user remotely.

Examples smart watches /dryers that can be controlled remotely and notify when the washing / driving cycle is complete smart thermostat Allow controlling the temperature remotely and can learn the user preferences smart refrigerator can keep track of the item stored and send update to the user when an item is low on stock.

Smart TV Allows user to search and stream videos and movies from the internet on a local storage drive, search TV channel schedule and fetch news weather updates and other content from the internet.

Open remote is an open source automation platform for homes buildings. Open remote is platform agnostic and works with standard hardware. With Open Remote, user can control various appliances using mobile or web publications.

Open Remote comprises of three components - a controller that manages scheduling and runtime integrations between device, a designer that allows you to create both configuration for the controller and create user interface design and control panel that allow you to interact with the devices and control them.

1.5.1.3 Smoke / Gas Detector

Smoke detectors are installed in home and buildings to detect smoke that is typically and early sign of Fire. Smoke detectors use optical detection class ionization for sampling techniques to detect smoke.

Alerts raised by smoke detectors can be in the form of signals to fire alarm system. Gas detectors can detect the presence of harmful gases such as carbon monoxide liquid Petroleum gas (LPG).

A Smoke / gas detector raise alerts in human was this describing where the problem is send or an SMS or email to the user or the local fire safety department and provide visual feedback on its status the design of the system that detects gas leakage on smoke and it gives visual level indication.

1.5.2 Cities

1.5.2.1 Smart Parking

Finding a parking space during rush hours in crowded cities can be time consuming and frustrating. Further for the more drivers blindly searching for parking spaces create additional traffic conditions for Star smart parking make the search for parking space easier and convenient for drivers.

Smart parking for powered by IoT system that detect the number of empty parking slots and send the information over the internet to smart parking application back ends. These applications can be accessed by the drivers from smartphones, tablets and in car navigation system.

In smart parking sensors are used for each parking slot, to detect whether the slot is empty or occupied. This information is aggregated by your local controller and then send over the internet to the database.

Design and implementation of a prototype smart parking system based on wireless sensor Network Technology with just like a remote parking monitoring, automated guidance, and departing reservations mechanism.

1.5.2.2 Smart Lighting:

Smart lighting system for road parks and building can help in saving energy. According to a IEA report, lightening is responsible for 19 % of global electricity use and around 6% of global Greenhouse gas emission.

Smart lighting allows lighting to be dynamically controlled and also adapted to the ambient conditions. Smart lights connected to the internet can be controlled remotely to configure lighting schedules and lighting intensity.

Lighting configuration can be set for different situations such as a foggy day, a festival etc. Smart. Lights equipped with the sensors and can communicate with other lights and exchange information on the sensed ambient conditions to adapt lightening.

1.5.2.3 Smart Road

Smart roads equipped with sensors can provide information on driving conditions, travel time estimates and alerts in case of poor driving conditions, traffic congestions and accidents. Such information can help in making the roads safe and help in reducing traffic jams.

Information sense to from the roads can be communicated via internet to cloud based applications and social media and disseminated to the drivers who subscribed to such applications.

Distributed and autonomous systems of sensor networks notes for improving driving safety proposed system can provide the drivers and passengers with a consistent view of the road situations of a few hundred meters ahead of them or a few dozen miles away, so that they can react to potential dangers early enough.

1.5.2.4 Structural Health Monitoring:

Structural health monitoring system uses a network of sensors to monitor the vibrations levels in the structures such as bridges and buildings. The data collected from the sensors is analyzed to assess the health of the structures.

By analyzing the data it is possible to detect cracks and mechanical breakdown, locate the damage to a structure and also calculate the remaining life of the structure. Using such systems advance warning can be given in the case of imminent failure of the structure.

An environmental effect removal based structural health monitoring schemes is an IoT environment is proposed. Since structural health monitoring schemes use large number of wireless sensor nodes which are powered by traditional batteries researchers are exploring energy harvesting Technologies to harvesting ambient energy, such as mechanical vibrations, sunlight and wind.

1.5.2.5 Surveillance

Surveillance of infrastructure, public transport and even in cities is required to ensure safety and security. City wide surveillance infrastructure comprising of large number of distributed and internet connected video surveillance cameras can be created. The video feeds from surveillance cameras can be aggregated in cloud based storage solutions. Cloud-based video analytics applications can be deployed to search for patterns for specific events from the video feed.

1.5.2.6 Emergency Response

IoT Systems can be used for monitoring the critical infrastructure in cities such as building, gas and water pipelines, public transport and power substation systems. IoT systems for fire directions, gas and water leakage directions can help in generating alerts and minimizing their effects on the critical infrastructure.

IoT systems for critical infrastructure monitoring enable aggregations and sharing of information is collected from large number of sensors. Cloud based architecture multi model information such as sensor data, audio, video feeds can be analyzed in near real-time to detect adverse event.

Response to alerts generated by such systems can be in the form of alerts sent to the public, re-routing of traffic, evacuation of the affected areas. Traffic management system for emergency services is describe the system adapt by dynamically adjusting traffic lights, changing related driving policies, recommending behavior change to drivers and applying essential security controls

1.5.3 Environment

1.5.3.1 Weather Monitoring:

IoT- based weather monitoring system can collect data from a number of sensor attached such as temperature, humidity, pressure etc. and send the data to cloud based application and storage back-ends. The data collected in the cloud can then be analyzed and visualized by cloud based application.

Weather alerts can be sent to the subscribed users from such applications AirPi weather and air quality monitoring kit capable of recording and uploading information about temperature, humidity, air pressure light levels, UV levels, carbon monoxide Nitrogen dioxide and smoke level to the internet.

1.5.3.2 Air Pollution Monitoring

IoT based air pollution monitoring system can monitor emission of harmful gases (CO₂, CO, NO, NO₂) by factories and automobiles using gases and dermatological sensors the collected data can be analyzed to make informed decisions on pollutions control approaches.

In real time air quality monitoring system is presented that comprises of several distributed monitoring stations that communicate via wireless with a backend server using machine to machine communication. In addition sums of GPRS modem and GPS module.

1.5.3.3 Noise Pollution Monitoring

Due to growing Urban Development, noise levels in cities have increased and even become alarmingly high in some cities. Noise pollution can cause health hazards for humans due to sleep destruction and stress. Noise pollution monitoring can help in generating noise maps for cities.

Urban noise maps can help the policy makers in urban planning and making policies to control noise level near Residency areas, schools and parks. IoT based noise pollution smart metering system use a number of noise monitoring station that are deployed at different places in a city.

The data on noise levels from the stations is collected on server or in the cloud. The data is then aggregated to generate noise map. In noise mapping study for a city is presented which revealed that the cities suffers from serious noise pollution .In the design of smart phone application is described that allows a user to continuously measure noise level and send it to a central server where all generated information is aggregated and mapped to a meaningful noise visualizations map.

1.5.3.4 Forest Fire Detection

Forest fires can cause damage to natural resources, property and human life. There can be different causes of forest fires including lightening, human negligence, volcanic eruptions and sparks from rock fall. Early deduction of forest fires can help in minimizing the damage.

IoT based forest fire detection systems can use a number of monitoring nodes deployed at a different locations in a forest. Each monitoring node collects measurements on ambient conditions including temperature, humidity, and light levels.

A system for early detection of forest fire is described in that provides early warning of a potential forest fire and estimate the scale and intensity of the fire if it materializes in a forest fire detection system based on wireless sensor networks is presented.

The system uses multi-criteria deduction which is implemented by the artificial neural network. The ANN fuses sensing data corresponding to multiple attributes of your forest fire such as temperature, humidity, infrared and divisible light to detect forest fire.

1.5.3.5 River Flood Detection:

River flood can cause extensive damage to the natural and human resources and human life. River flood occurs due to continuous train for which cause the river level to rise and flow rates to increase rapidly. Early warnings of floods can be given by monitoring the water level and flow rate.

IoT based river flood monitoring system uses a number of sensor nodes that monitor the water level using ultrasonic sensors and flow rate using the flow velocity sensors. Data from a number of such sensor nodes is aggregated in a server or in the cloud.

Monitoring applications raise alert when rapid increase in water level and flow rate is detected in a river flood monitoring system is described that Measures River and weather conditions through wireless sensor nodes equipped with different sensors. The systems include water level monitoring module, and data processing module that provide flood information is in the form of raw data, predicted data, and video feed.

1.5.4 Energy

1.5.4.1 Smart grid

Smart grid is a data communication network integrated with electrical grid that collects and analyses data captured in real-time about power transmission, distribution and consumption. Smart grid Technology provides protective information and recommendations to utilities, their suppliers, and their customers on how best to manage power.

Smart Grids collect data regarding electricity generation (centralized or distributed conceptions (instantaneous or predictive) storage (or conversion of the energy into other forms), distributions and equipment health data.

Smart fleet use high speed, fully integrated two way communication real time information and power exchange. Smart meters can capture almost real time consumption, remotely control the conceptions of electricity and remotely switch off supply when required.

Power thefts can be prevented using smart metering by analyzing the data on power generation, transmission and consumption smart grid can improve efficiency throughout the electric system.

Storage collection and analysis of smart grids data in the cloud can help in dynamic smart grids data can improve energy usage levels via energy feedback to user coupled with real-time pricing information.

Real time demand response and management strategies can be used for lowering peak demand and the overall load via appliance control and energy storage mechanism.

Condition monitoring data collected from power generation and transmission system can help in detecting fault and predicting outage.

1.5.4.2 Renewable Energy System:

Due to the variability in the output from renewable energy sources such as solar and wind integrating them into the grid can cause grid stability and reliability problems. Variable output produces local voltage swing that can impact power quality. Existing grids were designed to handle power flow from centralized to generation source to the loads through Transmission and distribution lines.

When distributed renewable energy sources are integrated into the grid, they create power bidirectional power flow for which the grid were not originally designed to handle power flow from temperature generation sources to the loss through transmissions and distribution lines.

When Distributed renewable energy sources are integrated into the grid is a create power bi-directional power flows for which the grid were not originally designed. IoT based system integrated within Transformer at the point of interconnections measures the electrical variables and how much power is fed into the grid.

To ensure the grid stability one solution is to simply cut of the over protections. For wind energy systems, closed-loop controls can be used to regulate the voltage at points of interconnection

1.5.4.3 Prognostics:

Energy systems (Smart grids, power plants, wind turbine forms) have a large number of critical components that must function correctly so that the system can perform their operations correctly.

For example a wind turbine has a number of critical components example bearing, turning gears, for instance that must be monitored carefully as wear and tear in such critical components or sudden change in operating conditions of the missions can result in failures. In system such as Power Grids, real time information using specialist electrical sensor is called phasor measurement unit (PMU) substations.

The information received from PMU must be monitored in real time for estimating the state of a system and for predicting failures. Energy system have thousands of sensors that gather real time maintenance data continuously for condition monitoring and failure prediction purposes. IoT based prognostic real-time health management systems can predict performance of machine or energy Systems by analyzing the extent of deviations of the system from its normal operating profiles.

Analyzing massive amounts of maintenance data collected from sensors in energy systems and equipment can provide protections for the impending failures (potentially in real time) so that their

reliability and availability can be improved. Prognostic health Management systems have been developed for different energy systems open PDC set of applications for processing of streaming time series data collected from phasor measurement units PMU in real time.

1.5.5 Retail

1.5.5.1 Inventory Management:

Inventory management for retail has become increasingly important in recent years with the growing competition. While over stocking of products can result in additional storage expenses and risk understocking can lead to loss of revenue. IoT system using radio frequency identification RFID tags can help in inventory management and maintaining the right inventory levels.

RFID tags attached to the products allow them to be tracked in real time so that the inventory levels can determined accurately and products which are low on stock can be replenished. Tracking can be done using RFID readers attached to the retail store shelves or in the warehouse. An RFID database inventory management system for time sensitive materials is described.

1.5.5.2 Smart Payments

Smart Payments solutions contact list payments powered by technology such as near field communication and Bluetooth. Near field communication is a set of standards for smartphones and other devices to communicate with each other by bringing them into proximity or by touching them.

Customers can store credit card information in the NFC enabled Smartphones and make payment by bringing the smart phones and make payments by bringing the smart phones near the point of sale terminals . NFC may be used in combinations with Bluetooth, where NFC initiates initial pairing of devices to establish their Bluetooth connections while the actual data transfer takes place over Bluetooth.

1.5.5.3 Smart Vending Machines

Smart vending machines connected to the internet allow remote monitoring of inventory levels, elastic pricing of products, promotions, and contact less payments using NFC smart phone applications that communicate with smart vending machine allow user preferences to be remembered and the learnt with the time when a user moves from one vending machine to the other and pass the smartphone with the vending machines using specific interface is presented.

Users can save their preferences and favorite products sensor in a smart vending machine monitor its operations and send the data to the cloud which can be used for predictive maintenance. Smart

vending machines can communicate with other vending machines in their vicinity and share their inventory levels so that the customers can be routed to the nearest machines in case product goes out of stock in a machine

1.5.6 Logistics

1.5.6.1 Route Generation and Scheduling

Modern transportation systems are driven by data collected from multiple sources which is processed to provide new services to the stockholders. By collecting large amount of data from various sources and processing the data into Useful information data driven.

Transportation system can provide new services such as advanced route guidance dynamic vehicle routing anticipating customer demand for pickup and delivery problem, for instance route generations and scheduling systems candidate end-to-end using combinations of road patterns and transportation smooth and feasible schedule based on the availability of vehicles.

1.5.6.2 Fleet Tracking

Vehicle fleet tracking system using GPS technology to track the locations of vehicle in real time. Cloud based fleet tracking systems can be scaled up on-demand to handle large number of vehicles.

Alerts can be generated in case of deviation in planned routes. The vehicle locations and routes data can be aggregated and analyzed for detecting bottlenecks in the supply chain such as a traffic conditions or route of elements and generations of alternative route and supply chain Optimization in a fleet tracking system for commercial vehicle is described the system can analyze messages sent from the vehicles to identify unexpected incidence and descriptions is between the actual and applied data.

1.5.6.3 Shipment Monitoring:

Shipment monitoring solutions for Transportation systems allow monitoring the conditions inside container. For example, containers carrying Fresh Food produce can be monitored to prevent spoilage of food.

IoT based shipment monitoring system you sensor such as temperature pressure and humidity for instance to monitor the conditions inside the container and send the data to the cloud where it can be analyzed to detect food spoilage.

The analysis and interpretation of the data in the environmental conditions in the container and food truck positioning can enable more effective routing decisions and their time therefore it is possible to take remedial measures such as the food that has a limited time budget before it get rotten can be rerouted to a closer destinations, alerts can be raised to the driver and the distributor about the transit conditions, such as container temperature exceeding the allowed limit, humidity levels going out of the allowed limit.

For instance, and corrective actions can be taken before the food gets damaged. A Cloud-based frame work for real time fresh Food Supply tracking and monitoring was proposed. For fragile products vibrations levels during shipment can be tracked using accelerometer and gyroscope sensors attached to IoT device.

A system for monitoring container in integrity and operating conditions described. The system monitors the vibrations patterns on their container and its contents to reveal information related to its operating environment and integrity during transport handling and storage.

1.5.6.4 Remote Vehicle Diagnostics

Remote vehicle diagnostic systems can detect faults in the vehicles warn of impending fault. These Diagnostic system use on-board IoT devices for collecting data on vehicle operation such a speed, engine RPM, coolant temperature, fault code number ,and status of the various vehicle subsystem such data can be captured by integrating on-board diagnostic systems with IoT devices using protocols such as CAN bus.

Modern commercial vehicles support on-board diagnostics OBD standards such as OBD-II. OBD system provides real time data status of vehicle subsystems and diagnostic trouble code which Allow rapidly identifying the fault in the vehicle. IoT based vehicle diagnostic system can send the vehicle data to centralized serves or the cloud where it can be analyzed to generate alerts and suggest remedial actions.

1.5.7 Agriculture

1.5.7.1 Smart Irrigation

Smart irrigation systems can improve crop yield while saving water. Smart irrigation system using IoT devices with soil moisture sensors to determine the amount of moisture in the soil and realize the flow of water through the irrigation pipe only when the moisture level go below a predefined threshold. Smart irrigation systems also collect Moisture level measurements on a server or in the cloud where they collected data can be analyzed to plant watering schedule.

1.5.7.2 Green House Control

Green house structures with glass or plastic roofs that provide conducive environment for growth of plants. The Climatological conditions inside a Greenhouse can be monitored and controlled to provide the best conditions for growth of plants. The temperature, humidity, soil moisture, light and carbon dioxide levels are monitored using sensors and their climatological conditions are controlled automatically using actuation devices.

IoT system plays an important role in greenhouse controlled and help in improving productivity. The data collected from various sensors is stored on centralized servers or in the cloud where analysis is performed to optimize the control strategies and also correlate the productivity with different control strategies.

The system uses wireless sensor network to monitor and control the agriculture parameters like temperature and humidity in real time for better management and maintenance of Agricultural production

1.5.8 Industry

1.5.8.1 Machine Diagnosis and Prognosis:

Machine prognosis refers to predicting the performance of a machine by analyzing the data and the current operating conditions and how much deviations exist from the normal operating conditions. Machine diagnosis refer to determining the causes of a machine fault.

IoT plays a major role in both the prognosis s and Diagnostics of industrial machines. Industrial machines have a large number of components that must function correctly for the machine to perform its operations. Sensors in machine can monitor the operating conditions such as temperatures and vibrations levels. The sensor data measurements are done on time scale of few milliseconds to few seconds, which leads to generations of the massive amount of the data.

Case-based reasoning (CBR) is a commonly used method that find solutions to new problem based on past experience. This past experience is organized and represented as case in a case base. CBR is an effective techniques for problem solving in the field in which it is hard to establish a quantitative mathematical model, such as machine Diagnostics and prognosis.

Since foe each machine, data from a very large number of sensors is collected using search high-dimensional data for creation of a case library reduce the case retrieval efficiency. Data reduction and feature extraction methods are used to find the representatives set of ability as the of features.

1.5.8.2 Indoor Air Quality Monitoring:

Monitoring indoor air quality in factories is important for health and safety of the workers. harmful and toxic gas such as carbon monoxide, nitrogen monoxide and Nitrogen dioxide etc. t can cause serious health problems .IoT based gas monitoring system can help in monitoring the indoor air quality using various gas sensors. The indoor air quality can vary for different locations. Wireless sensor networks based IoT devices can identify the other hazardous zones, so that a corrective measures can be taken to ensure proper ventilation

1.5.9 Health and Lifestyle

1.5.9.1 Health and fitness monitoring

Wearable IoT devices that are low noninvasive and continuous monitoring of physiological parameters can help in continuous health and fitness monitoring. These wearable devices may can be in various forms such as built under wrist bands.

The wearable devices from a wireless sensor networks called body area networks in which the measurements from a number of wearable device are continuously send to a master nodes such as a smartphone which then send the data to the server or a cloud end for analysis and achieving. Healthcare providers can analyze the collected Healthcare data to determine any health conditions or anomalies.

1.5.9.2 Wearable Electronics

Wearable Electronics such as smart watches smart glasses wristband and fashion electronics (with electronic integrated in clothing and accessories ,example Google glass for Moto 360 smart watches provide various functions and future to assist us in our daily activities and making as lead healthy Lifestyle .

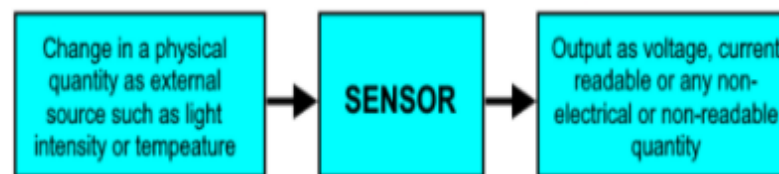
.Smart watches the user can search the internet a play audio video file make calls with or without paired mobile phones, play games and use various kinds of mobile applications smart classes allow users to take photos and record videos ,get map directions check flight status and search the internet by using voice commands

CHAPTER 2

SENSOR, ACTUATORS AND INTERFACING

2.1 Roles of Sensors and actuators

Sensors and actuators are two of the most important components in the Internet of Things (IoT). Sensors collect data from the physical world, while actuators take actions based on that data. Together, they allow IoT devices to interact with their environment and respond to changes in real time.



2.1.1 Sensors

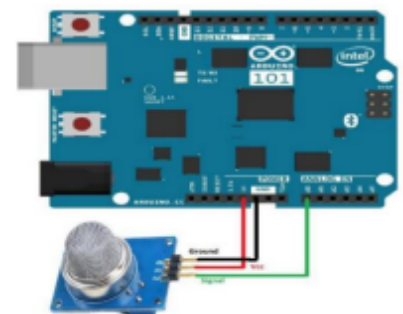
Sensors are devices that convert physical phenomena into electrical signals. These signals can then be processed and analyzed by an IoT device or system. There are many different types of sensors, each designed to measure a specific physical quantity. Some common types of sensors include: Temperature sensors, Pressure sensors, Humidity sensors, Light sensors, Motion sensors, and Gas sensors

MQ Sensor

- Sensor Gases detected
- MQ 3 Alcohol
- MQ 7 CO
- MQ 4 Methane, Natural Gas
- MQ135 Air Quality Control
- MQ6 LPG, Propane
- MQ 8 Hydrogen
- MQ 2 General combustible gas
- MQ 5 LPG, Natural gas, Coal gas

Connection

- Connect GND pin of Gas sensor to GND on the board
- Connect Out pin of Gas Moisture sensor to A0 on the board
- Connect VCC pin of Gas Moisture sensor to 5V on the board
- Connect power supply to the Arduino and USB to USB Client Port on the board



2.1.2 Actuators

Actuators are devices that convert electrical signals into physical actions. These actions can be anything from turning a valve to moving a robotic arm. There are many different types of actuators, each designed to perform a specific action. Some common types of actuators include: Electric motors, Solenoids, Pneumatic actuators, Hydraulic actuators

2.1.3 The Role of Sensors and Actuators in IoT

Sensors and actuators work together to allow IoT devices to interact with their environment and respond to changes in real time. For example, a sensor in a smart thermostat can detect changes in the temperature of a room. If the temperature gets too high, the thermostat can send a signal to an actuator that will turn on the air conditioner.

Sensors and actuators are used in a wide variety of IoT applications. Some common examples include: Smart home automation, Industrial automation, Healthcare, Transportation, and Agriculture

2.1.2 Types of Sensor

2.1.2.1 Active and Passive sensor

Active sensors emit energy to measure the environment, while passive sensors only receive energy from the environment. For example, a radar sensor is an active sensor, while a photoresistor is a passive sensor.

2.1.2.2 Analog and Digital Sensor

Analog sensors measure a physical quantity and produce a continuous output signal, while digital sensors measure a physical quantity and produce a discrete output signal. For example, a temperature sensor can be either analog or digital.

2.1.2.3 Contact and No-Contact Sensor

Contact sensors require physical contact with the object being measured, while no-contact sensors do not require physical contact. For example, a proximity sensor is a contact sensor, while an ultrasonic sensor is a no-contact sensor

2.1.2.4 Absolute and relative

Absolute sensors measure the absolute value of a physical quantity, while relative sensors measure the change in a physical quantity from a reference value. For example, a thermometer is an absolute sensor, while a level sensor is a relative sensor.

Here is a table summarizing the different types of sensors:

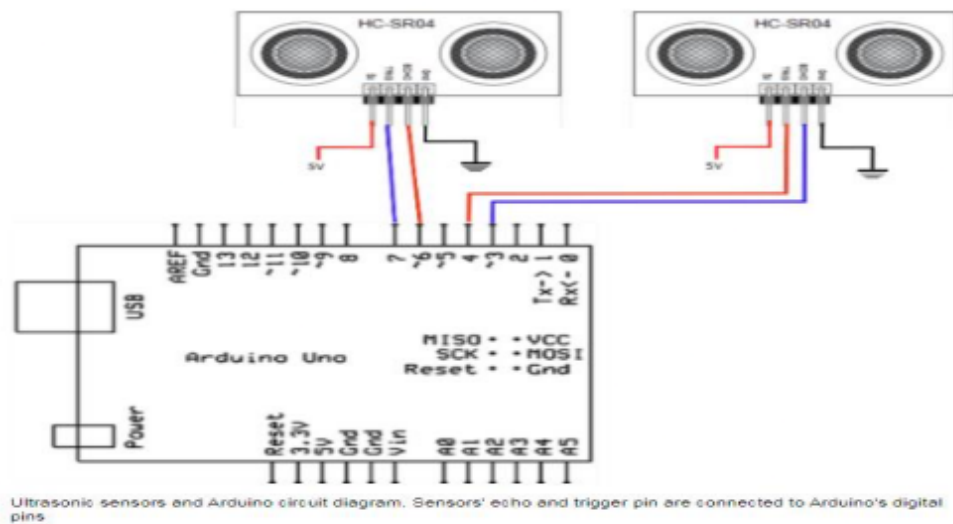
Type	Definition	Example
Active	Emits energy to measure the environment	Radar sensor
Passive	Only receives energy from the environment	Photoresistor
Analog	Measures a physical quantity and produces a continuous output signal	Temperature sensor
Digital	Measures a physical quantity and produces a discrete output signal	Proximity sensor
Contact	Requires physical contact with the object being measured	Proximity sensor
No-contact	Does not require physical contact with the object being measured	Ultrasonic sensor
Absolute	Measures the absolute value of a physical quantity	Thermometer
Relative	Measures the change in a physical quantity from a reference value	Level sensor

2.2 Working of Sensors:

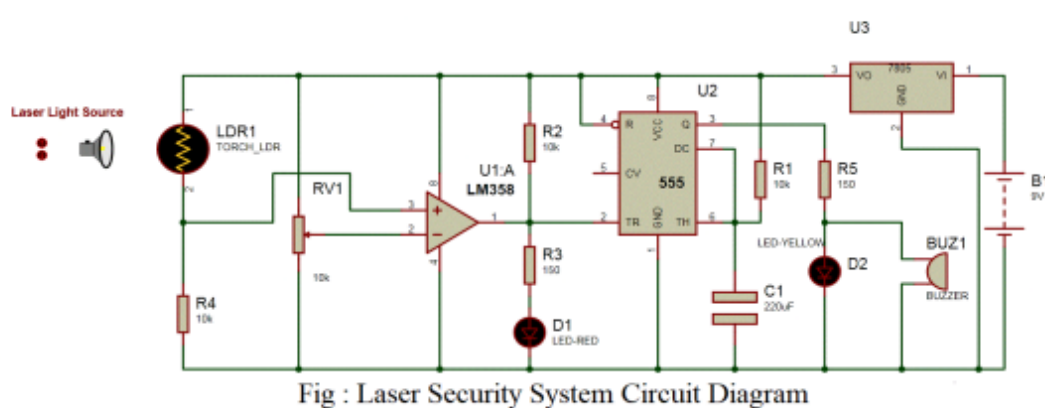
2.2.1 Position Sensors

Position sensors use a variety of technologies to determine the location of an object or person. For example

- **Ultrasonic sensor:** An ultrasonic sensor emits a sound wave and measures the time it takes for the wave to return. The distance to the object is then calculated based on the speed of sound.



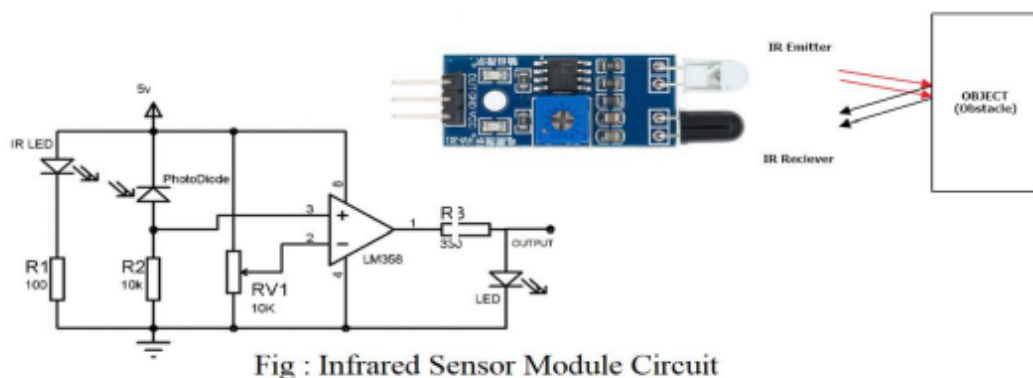
- Laser sensor:** A laser sensor emits a laser beam and measures the angle of reflection. The distance to the object is then calculated based on the angle of reflection and the known distance between the sensor and the laser emitter.



2.2.2 Occupancy and Motion Sensors

Occupancy and motion sensors detect the presence or movement of people or objects. For example

- **Infrared sensor:** An infrared sensor detects the heat emitted by a person or object. The sensor can then be used to determine whether an object is present or moving



- **Motion sensor:** A motion sensor detects changes in the environment, such as the movement of air. The sensor can then be used to determine whether an object is moving.

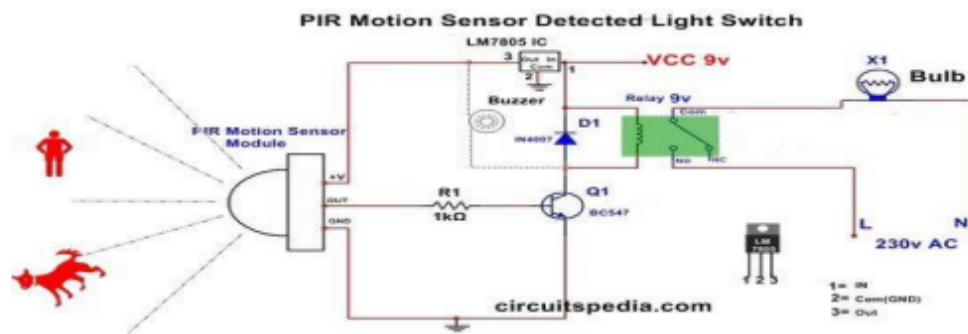


Fig : Motion sensor diagram

2.2.3 Velocity and Acceleration Sensors

Velocity and acceleration sensors measure the speed and direction of movement. For example,

- **Accelerometer:** An accelerometer measures the acceleration of an object in three dimensions. The acceleration can then be used to calculate the velocity and speed of the object.

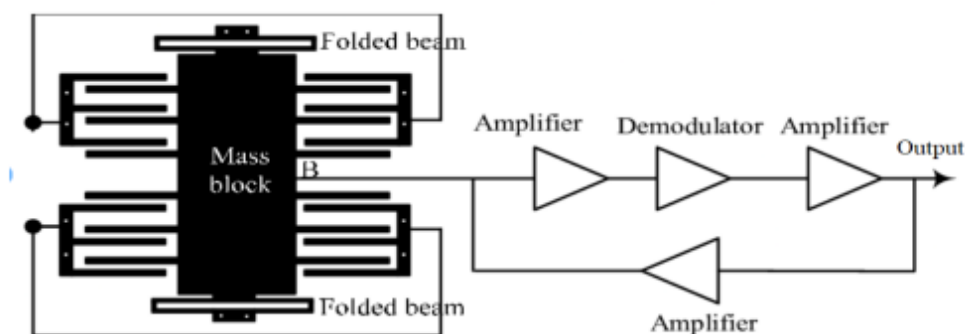


Fig : Accelerometer diagram

- **Gyroscope:** A gyroscope measures the rotation of an object. The rotation can then be used to calculate the orientation of the object.

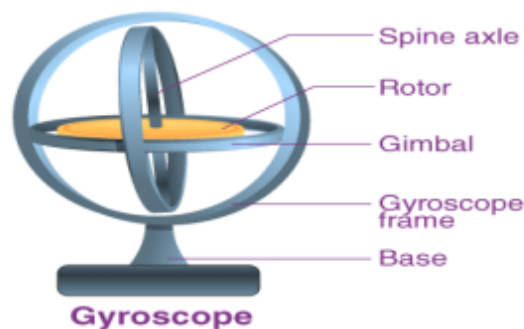


Fig : Gyroscope Diagram

2.2.4 Force and Pressure Sensors

Force and pressure sensors measure the amount of force or pressure applied to an object. For example,

- **Strain gauge:** A strain gauge is a thin metal strip that is bonded to a surface. When the surface is deformed, the strain gauge changes its resistance. The change in resistance can then be used to calculate the amount of force or pressure applied to the surface.

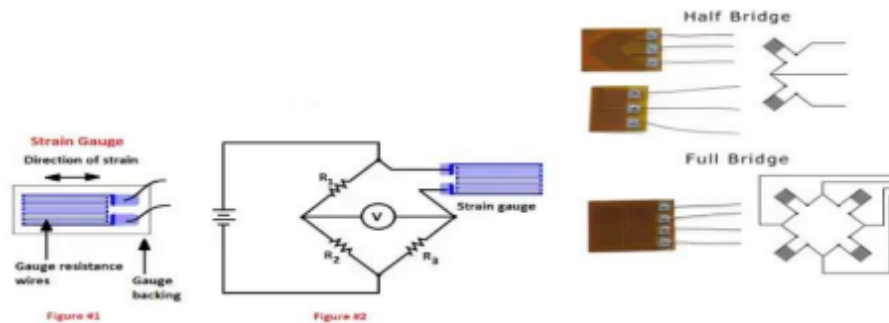


Fig : Strain Gauge Diagram

- **Pressure sensor:** A pressure sensor measures the amount of pressure applied to a surface. The pressure can then be used to calculate the force applied to the surface

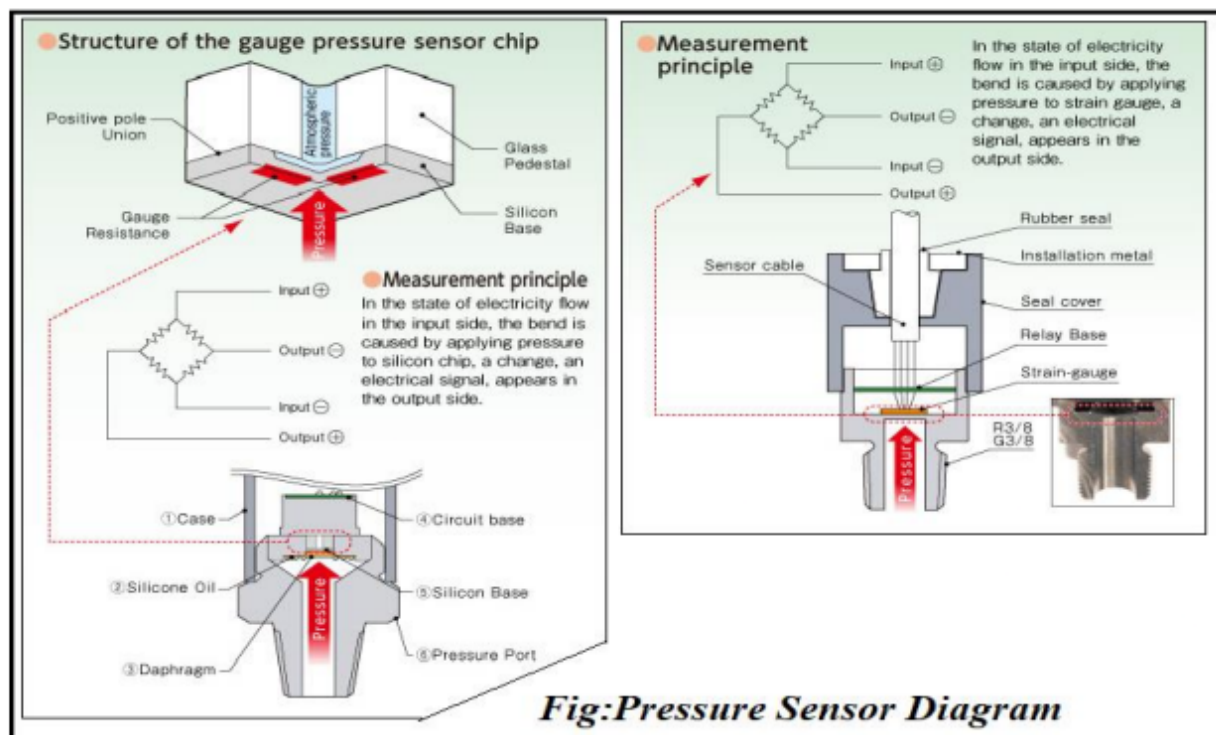


Fig:Pressure Sensor Diagram

2.2.5 Flow Sensors

Flow sensors measure the rate of flow of a fluid. For example,

- **Ultrasonic flow sensor:** An ultrasonic flow sensor emits a sound wave and measures the time it takes for the wave to travel through the fluid. The flow rate is then calculated based on the speed of sound and the known distance between the sensor and the emitter.

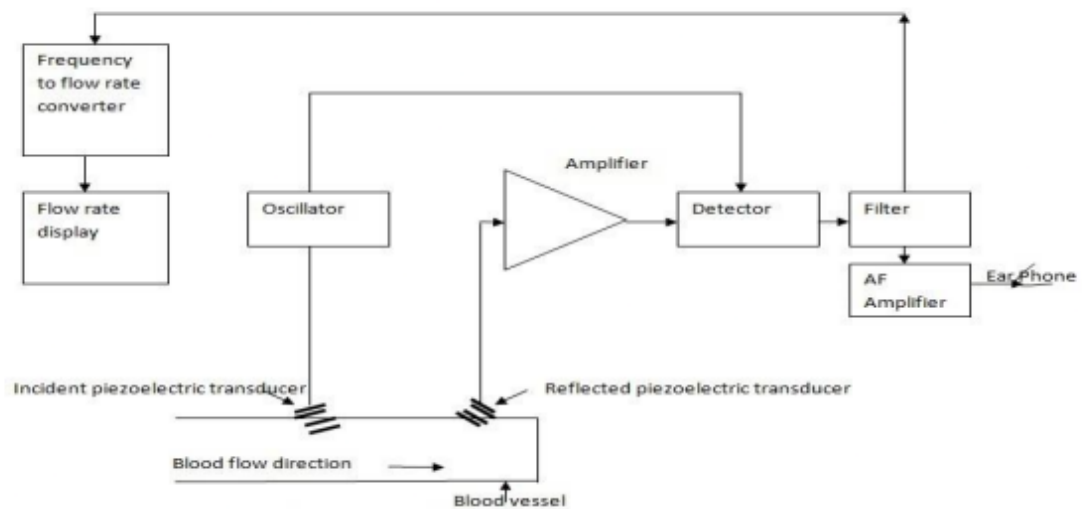


Fig: ultrasonic flow meter

- **Vortex flow sensor:** A vortex flow sensor measures the vortices created by the flow of fluid. The flow rate is then calculated based on the number of vortices created per unit time.

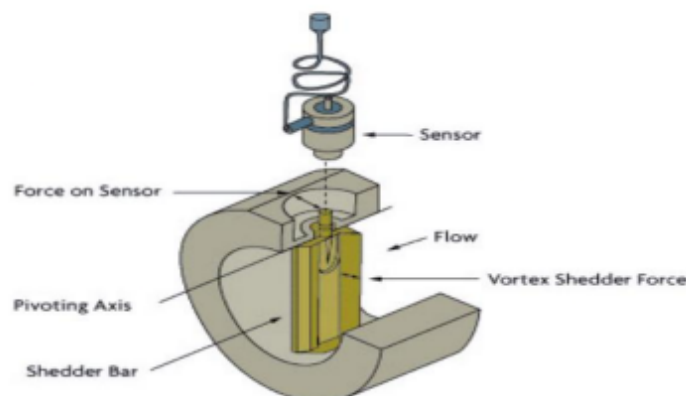


Fig: Vortex flow sensor

2.2.6 Acoustic Sensors

Acoustic sensors detect sound waves. For example,

- **Microphone:** A microphone converts sound waves into electrical signals. The electrical signals can then be processed to extract information about the sound, such as its frequency, amplitude, and direction of travel.

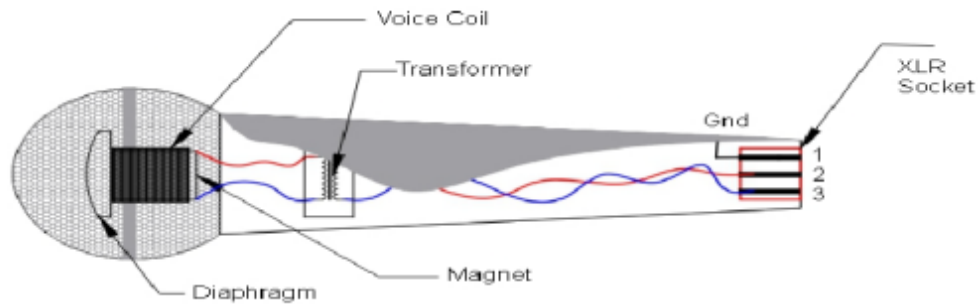


Fig : Microphone Sensor Diagram

2.2.7 Humidity Sensors

Humidity sensors measure the amount of water vapor in the air. For example,

- **Capacitance sensor:** A capacitance sensor measures the change in capacitance of a material when it is exposed to water vapor. The change in capacitance can then be used to calculate the humidity of the air.

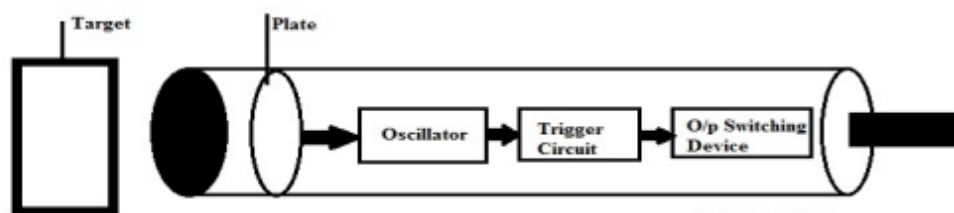


Fig : Capacitance Sensor Diagram

2.2.8 Light Sensors

Light sensors measure the amount of light in an environment. For example,

- **Photodiode:** A photodiode converts light into an electrical current. The electrical current can then be used to measure the brightness of the light.

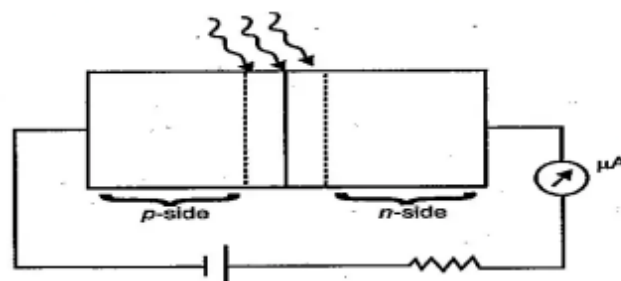


Fig : Photodiode Diagram

2.2.9 Radiation Sensors

Radiation sensors measure the amount of radiation in an environment. For example,

- **Geiger counter:** A Geiger counter detects the presence of ionizing radiation. The Geiger counter produces a clicking sound when it detects radiation.

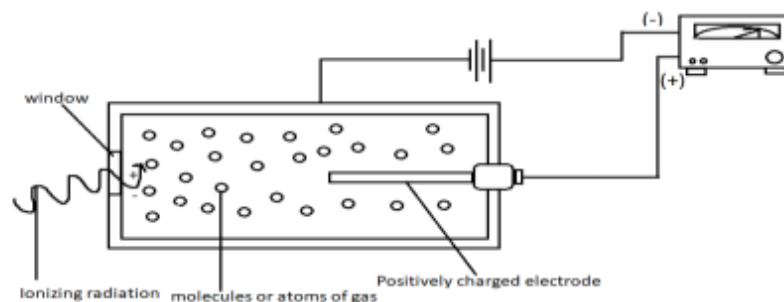


Fig : Geiger counter diagram

2.2.10 Temperature Sensors

Temperature sensors measure the temperature of an object or environment. For example,

- **Thermistor:** A thermistor changes its resistance in response to changes in temperature. The change in resistance can then be used to calculate the temperature.

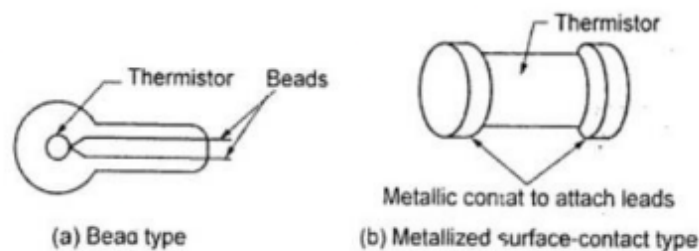


Fig : Thermistor Diagram

2.2.11 Chemical Sensors

Chemical sensors detect the presence of chemicals in an environment. For example,

- **Electrochemical sensor:** An electrochemical sensor generates an electrical signal in response to the presence of a specific chemical. The electrical signal can then be used to detect the presence of the chemical.

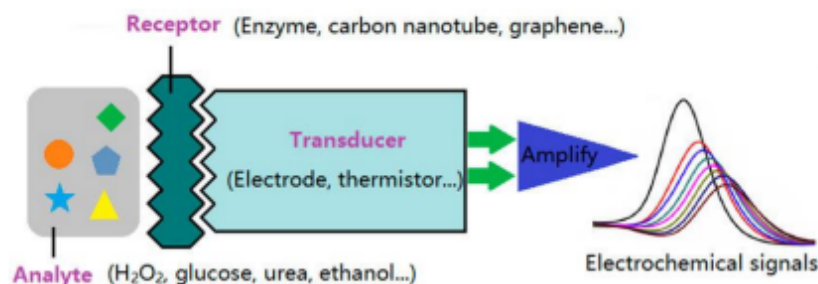


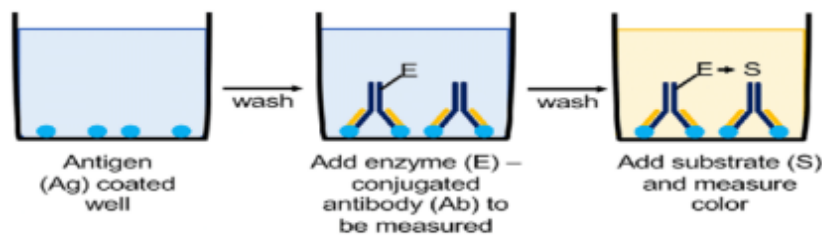
Fig: Electrochemical Sensor Diagram

2.2.12 Biosensor Sensors

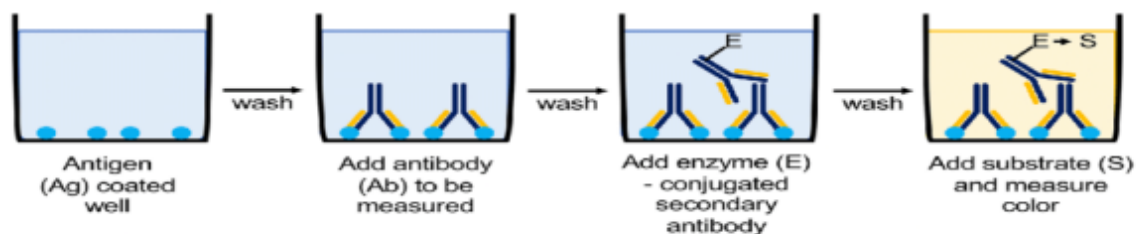
Biosensors detect the presence of biological molecules, such as DNA or proteins. For example,

- **Enzyme-linked immunosorbent assay (ELISA):** An ELISA can detect the presence of an antibody by binding to it and generating a measurable signal.

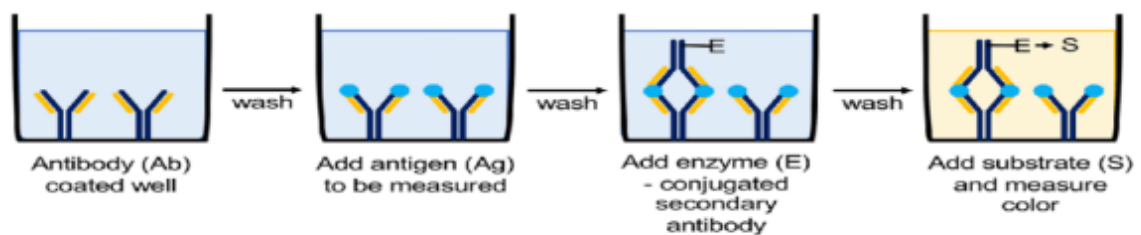
(a) Direct ELISA



(b) Indirect ELISA



(c) Sandwich ELISA



(d) Competitive ELISA

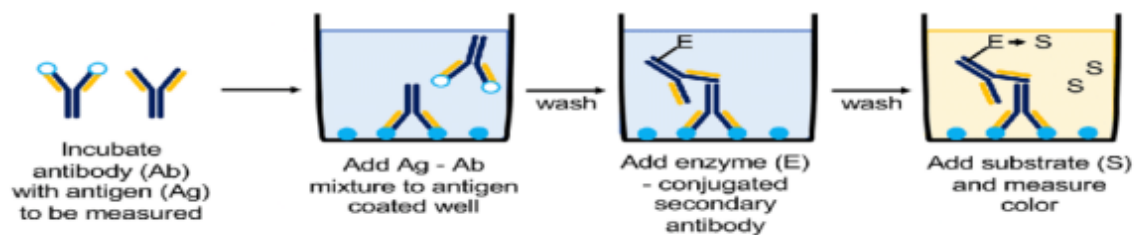


Fig : Enzyme-linked immunosorbent assay (ELISA) Diagram

2.2.13 Camera Sensors

Cameras capture images of an environment. Cameras use a variety of technologies to capture images, including optical lenses, image sensors, and image processors.

- **Optical lenses:** Optical lenses focus light onto an image sensor. The image sensor then converts the light into an electrical signal.

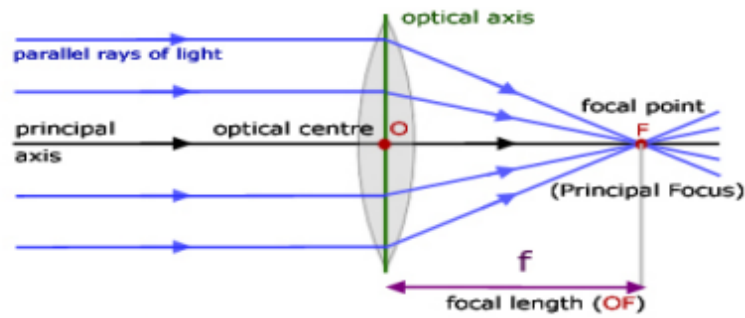


Fig : Optical lenses Diagram

- **Image sensors:** Image sensors convert light into an electrical signal. The electrical signal can then be processed to create an image

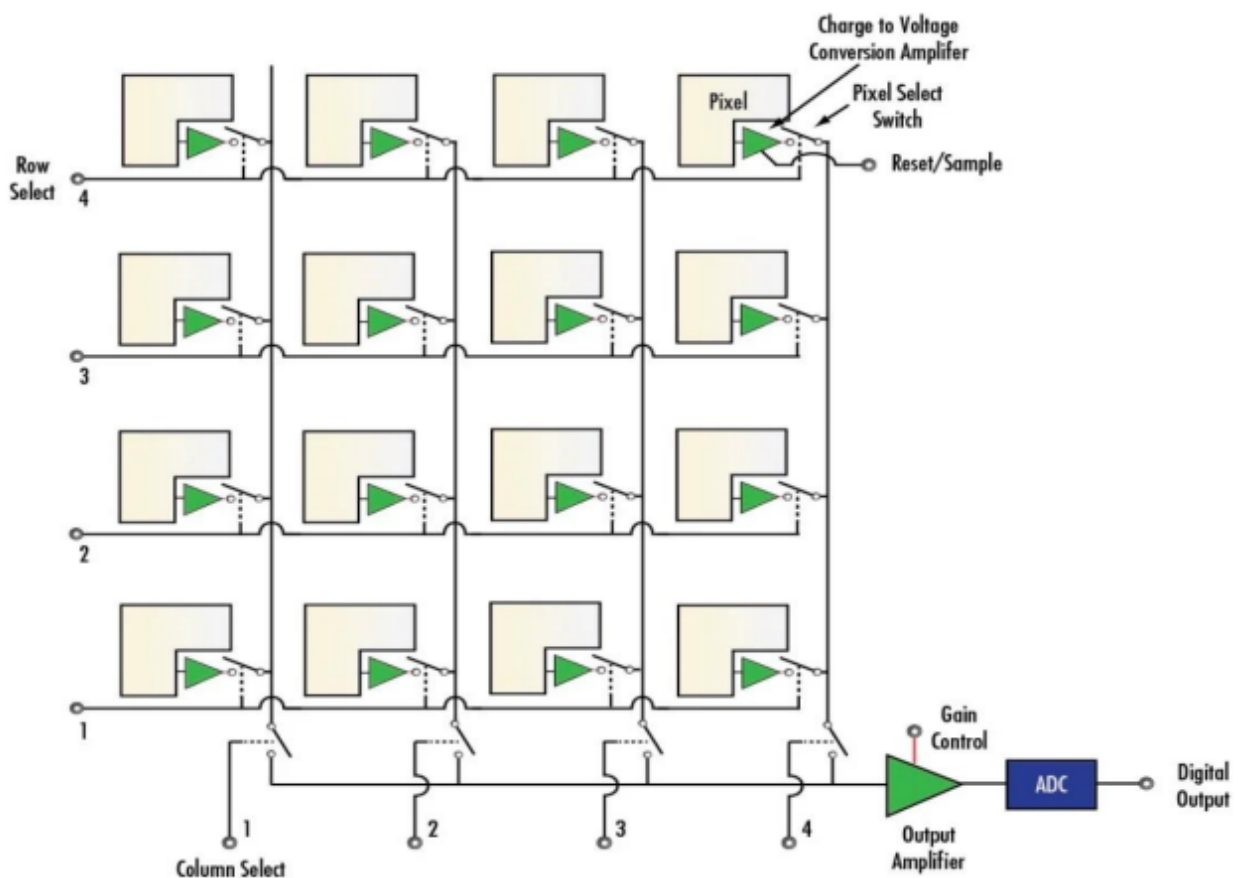


Fig : Image sensors Diagram

- **Image processors:** Image processors process the electrical signals from the image sensor

2.3 Development boards:

2.3.1 Arduino

Arduino is a prototype platform (open-source) based on an easy-to-use hardware and software. It consists of a circuit board, which can be programmed (referred to as a microcontroller) and a ready-

made software called Arduino IDE (Integrated Development Environment), which is used to write and upload the computer code to the physical board.

The key features are –

- Arduino boards are able to read analog or digital input signals from different sensors and turn it into an output such as activating a motor, turning LED on/off, connect to the cloud and many other actions.
- You can control your board functions by sending a set of instructions to the microcontroller on the board via Arduino IDE (referred to as uploading software).
- Unlike most previous programmable circuit boards, Arduino does not need an extra piece of hardware (called a programmer) in order to load a new code onto the board. You can simply use a USB cable.
- Additionally, the Arduino IDE uses a simplified version of C++, making it easier to learn to program.
- Finally, Arduino provides a standard form factor that breaks the functions of the microcontroller into a more accessible package.

Board Types

Various kinds of Arduino boards are available depending on different microcontrollers used. However, all Arduino boards have one thing in common: they are programmed through the Arduino IDE.

The differences are based on the number of inputs and outputs (the number of sensors, LEDs, and buttons you can use on a single board), speed, operating voltage, form factor etc. Some boards are designed to be embedded and have no programming interface (hardware), which you would need to buy separately. Some can run directly from a 3.7V battery, others need at least 5V.

Here is a list of different Arduino boards available.

Arduino Board Name	Voltage of System	CLK Speed	Digital I/O	Analog Inputs	PWM	UART	Interface of Programming
Arduino Uno – R3	5V	16MHz	14	14	6	1	USB through ATmega16U2

Arduino Uno R3 SMD	5V	16MHz	14	14	6	1	USB through ATMega16U2
Red Board	5V	16MHz	14	14	6	1	USB through FTDI
Arduino Pro 3.3V/8MHz	3.3V	8MHz	14	14	6	1	FTDI-Compatible Header
Arduino Pro 5V/16MHz	5V	16MHz	14	14	6	1	FTDI-Compatible Header
Arduino Mini 05	5V	16MHz	14	14	8	1	FTDI-Compatible Header
Arduino Pro Mini 3.3V/8MHz	3.3V	8MHz	14	14	6	1	FTDI-Compatible Header
Arduino Pro Mini 5V/16MHz	5V	16MHz	14	8	6	1	FTDI-Compatible Header
Arduino Ethernet	5V	16MHz	14	6	6	1	FTDI-Compatible Header
Arduino Fio	3.3V	8MHz	14	8	6	1	FTDI-Compatible Header or Wirelessly via XBee†
LilyPad Arduino 328 Main Board	3.3V	8MHz	14	6	6	1	FTDI-Compatible Header
LilyPad Arduino Simple Board	3.3V	8MHz	9	4	5	0	FTDI-Compatible Header

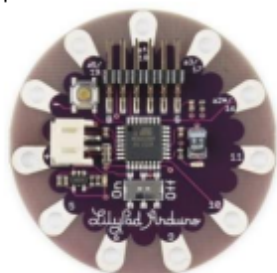
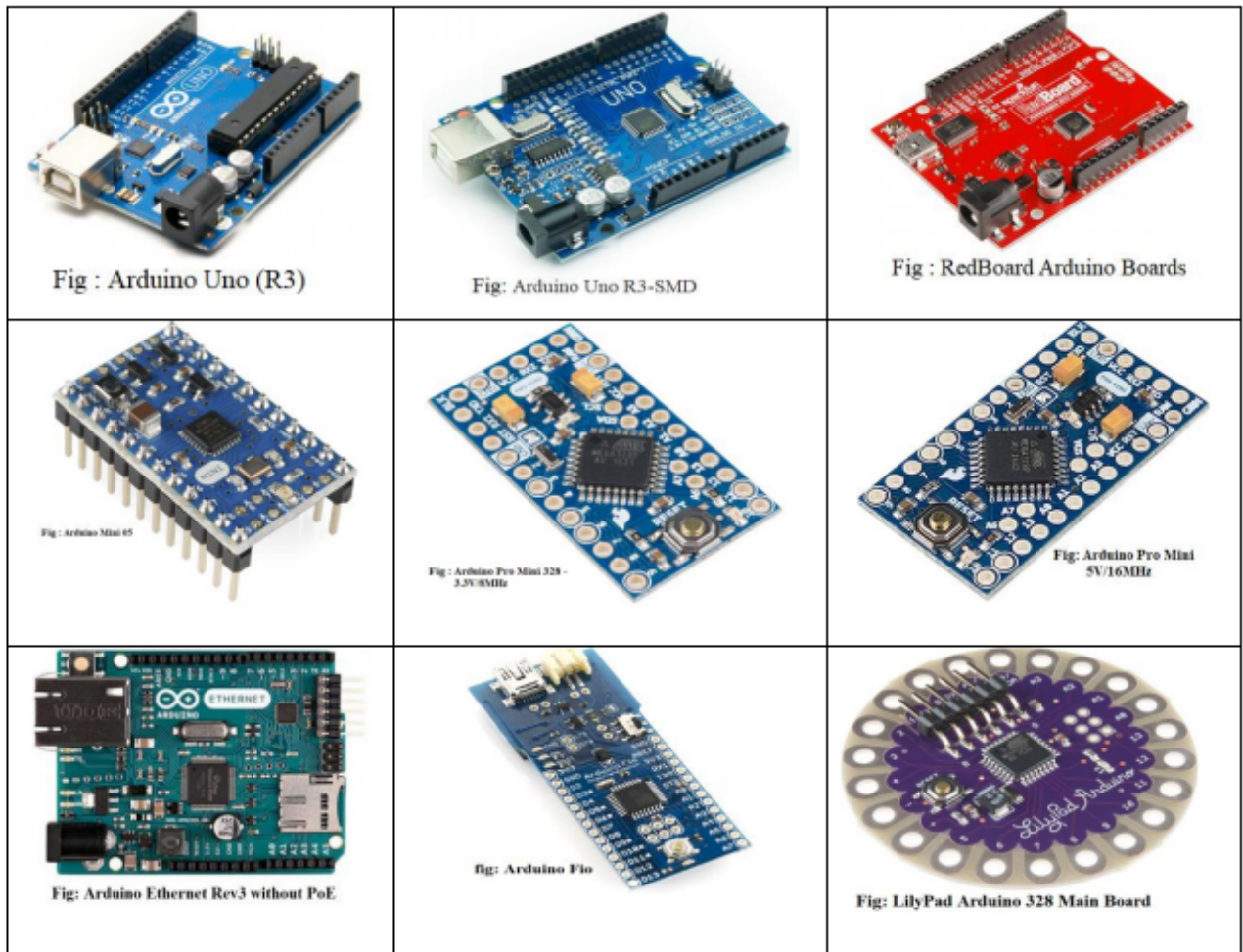


Fig: LilyPad Arduino Simple Board

Arduino boards based on ATMEGA32u4 microcontroller

Board Name	Operating Volt	Clock Speed	Digital i/o	Analog Inputs	PWM	UART	Programmin Interface
Arduino Leonardo	5V	16MHz	20	12	7	1	Native USB
Pro micro 5V/16MHz	5V	16MHz	14	6	6	1	Native USB
Pro micro 3.3V/8MHz	5V	16MHz	14	6	6	1	Native USB
LilyPad Arduino USB	3.3V	8MHz	14	6	6	1	Native USB

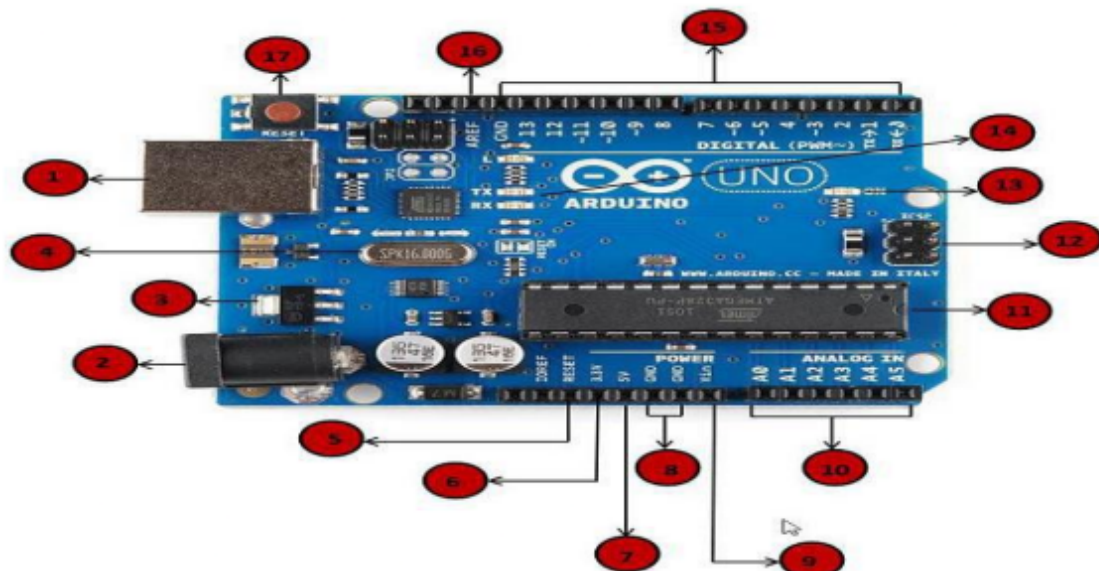
Arduino boards based on ATMEGA2560 microcontroller

Board Name	Operating Volt	Clock Speed	Digital i/o	Analog Inputs	PWM	UART	Programming Interface
Arduino Mega 2560 R3	5V	16MHz	54	16	14	4	USB via ATmega16U2B
Mega Pro 3.3V	3.3V	8MHz	54	16	14	4	FTDI-Compatible Header
Mega Pro 5V	5V	16MHz	54	16	14	4	FTDI-Compatible Header
Mega Pro Mini 3.3V	3.3V	8MHz	54	16	14	4	FTDI-Compatible Header

Arduino boards based on AT91SAM3X8E microcontroller

Arduino Board Name	Voltage of System	CLK Speed	Digital I/O	Analog Inputs	PWM	UART	Interface of Programming
Arduino Mega 2560 R3	3.3V	84MHz	54	12	12	4	USB Native

We will learn about the different components on the Arduino board. We will study the Arduino UNO board because it is the most popular board in the Arduino board family. In addition, it is the best board to get started with electronics and coding. Some boards look a bit different from the one given below, but most Arduinos have majority of these components in common.



1	Power USB Arduino board can be powered by using the USB cable from your computer. All you need to do is connect the USB cable to the USB connection (1).
2	Power (Barrel Jack) Arduino boards can be powered directly from the AC mains power supply by connecting it to the Barrel Jack (2).
3	Voltage Regulator The function of the voltage regulator is to control the voltage given to the Arduino board and stabilize the DC voltages used by the processor and other elements.
4	Crystal Oscillator The crystal oscillator helps Arduino in dealing with time issues. How does Arduino calculate time? The answer is, by using the crystal oscillator. The number printed on top of the Arduino crystal is 16.000H9H. It tells us that the frequency is 16,000,000 Hertz or 16 MHz.
5,17	Arduino Reset You can reset your Arduino board, i.e., start your program from the beginning. You can reset the UNO board in two ways. First, by using the reset button (17) on the board. Second, you can connect an external reset button to the Arduino pin labelled RESET (5).
6,7 8,9	Pins (3.3, 5, GND, Vin) 3.3V (6) – Supply 3.3 output volt 5V (7) – Supply 5 output volt - Most of the components used with Arduino board works fine with 3.3 volt and 5 volt.

	▪ GND (8)(Ground) – There are several GND pins on the Arduino, any of which can be used to ground your circuit. ▪ Vin (9) – This pin also can be used to power the Arduino board from an external power source, like AC mains power supply.
10	Analog pins The Arduino UNO board has six analog input pins A0 through A5. These pins can read the signal from an analog sensor like the humidity sensor or temperature sensor and convert it into a digital value that can be read by the microprocessor.
11	Main microcontroller Each Arduino board has its own microcontroller (11). You can assume it as the brain of your board. The main IC (integrated circuit) on the Arduino is slightly different from board to board. The microcontrollers are usually of the ATMEL Company. You must know what IC your board has before loading up a new program from the Arduino IDE. This information is available on the top of the IC. For more details about the IC construction and functions, you can refer to the data sheet.
12	ICSP pin Mostly, ICSP (12) is an AVR, a tiny programming header for the Arduino consisting of MOSI, MISO, SCK, RESET, VCC, and GND. It is often referred to as an SPI (Serial Peripheral Interface), which could be considered as an "expansion" of the output. Actually, you are slaving the output device to the master of the SPI bus.
13	Power LED indicator This LED should light up when you plug your Arduino into a power source to indicate that your board is powered up correctly. If this light does not turn on, then there is something wrong with the connection.
14	TX and RX LEDs On your board, you will find two labels: TX (transmit) and RX (receive). They appear in two places on the Arduino UNO board. First, at the digital pins 0 and 1, to indicate the pins responsible for serial communication. Second, the TX and RX led (13). The TX led flashes with different speed while sending the serial data. The speed of flashing depends on the baud rate used by the board. RX flashes during the receiving process.

15	Digital I/O The Arduino UNO board has 14 digital I/O pins (15) (of which 6 provide PWM (Pulse Width Modulation) output. These pins can be configured to work as input digital pins to read logic values (0 or 1) or as digital output pins to drive different modules like LEDs, relays, etc. The pins labeled “~” can be used to generate PWM.
16	AREF AREF stands for Analog Reference. It is sometimes, used to set an external reference voltage (between 0 and 5 Volts) as the upper limit for the analog input pins.

After learning about the main parts of the Arduino UNO board, we are ready to learn how to set up the Arduino IDE. Once we learn this, we will be ready to upload our program on the Arduino board.

2.3.1.1 Arduino Installation

We will learn in easy steps, how to set up the Arduino IDE on our computer and prepare the board to receive the program via USB cable.

Download the Arduino IDE from the Arduino website: <https://www.arduino.cc/en/software/>

Step 1 – First we must have our Arduino board (we can choose our favorite board) and a USB cable. In case we use Arduino UNO, Arduino Duemilanove, Nano, Arduino Mega 2560, or Diecimila, we will need a standard USB cable (A plug to B plug), the kind we would connect to a USB printer as shown in the following image.

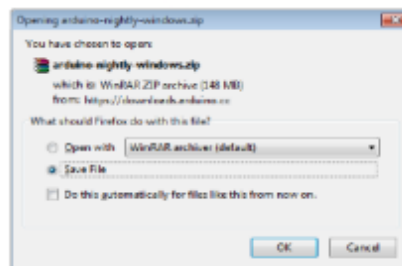


In case we use Arduino Nano, we will need an A to Mini-B cable instead as shown in the following image.



Step 2 – Download Arduino IDE Software.

You can get different versions of Arduino IDE from the Download page on the Arduino Official website. You must select your software, which is compatible with your operating system (Windows, IOS, or Linux). After your file download is complete, unzip the file.



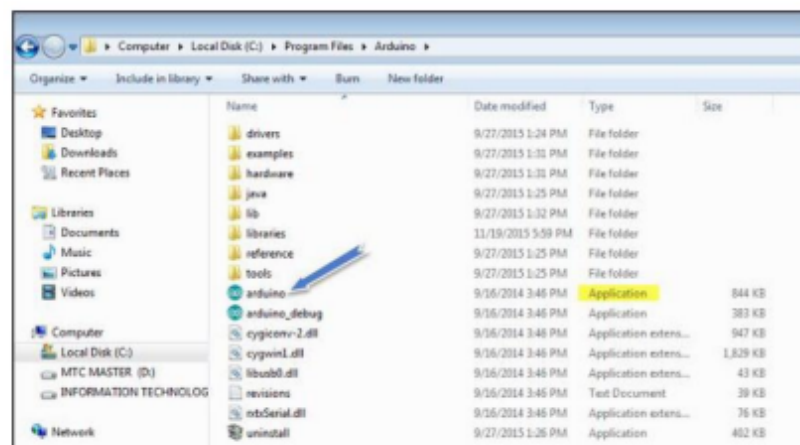
Step 3 – Power up your board.

The Arduino Uno, Mega, Duemilanove and Arduino Nano automatically draw power from either, the USB connection to the computer or an external power supply. If you are using an Arduino Diecimila, you have to make sure that the board is configured to draw power from the USB connection. The power source is selected with a jumper, a small piece of plastic that fits onto two of the three pins between the USB and power jacks. Check that it is on the two pins closest to the USB port.

Connect the Arduino board to your computer using the USB cable. The green power LED (labeled PWR) should glow.

Step 4 – Launch Arduino IDE.

After your Arduino IDE software is downloaded, you need to unzip the folder. Inside the folder, you can find the application icon with an infinity label (application.exe). Double-click the icon to start the IDE.

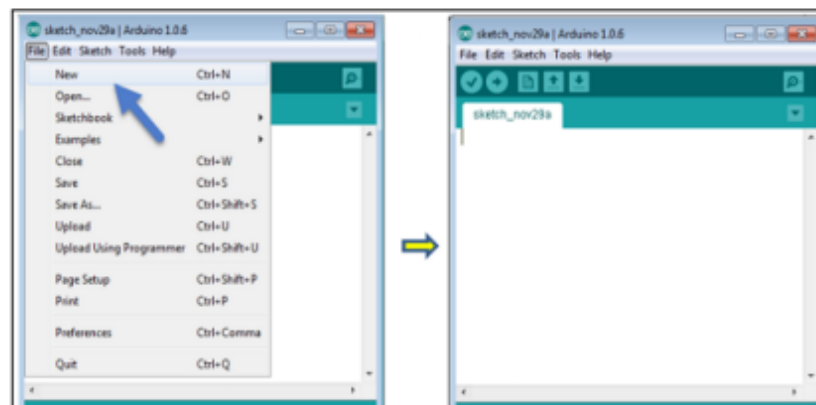


Step 5 – Open your first project.

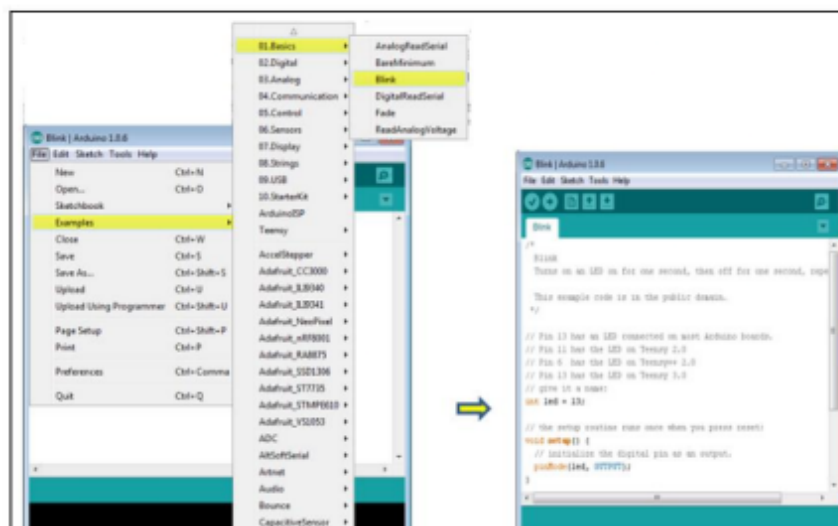
Once the software starts, you have two options –

- Create a new project.
- Open an existing project example.

To create a new project, select File → New.



To open an existing project example, select File → Example → Basics → Blink.

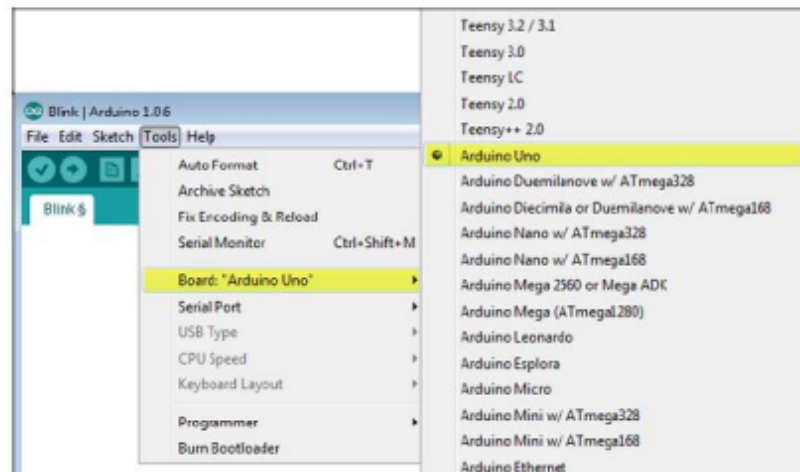


Here, we are selecting just one of the examples with the name Blink. It turns the LED on and off with some time delay. You can select any other example from the list.

Step 6 – Select your Arduino board.

To avoid any error while uploading your program to the board, you must select the correct Arduino board name, which matches with the board connected to your computer.

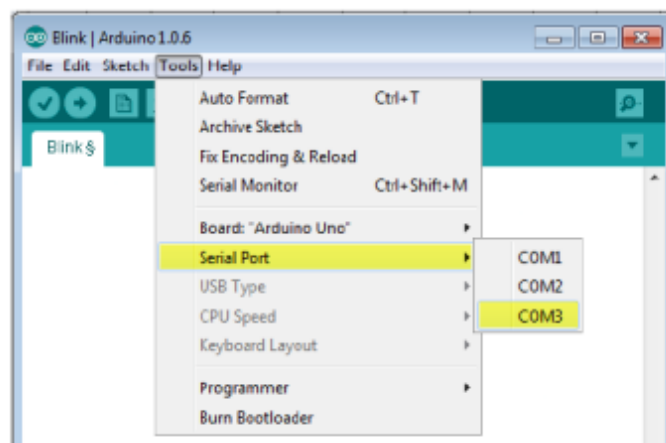
Go to Tools → Board and select your board.



Here, we have selected Arduino Uno board according to our tutorial, but you must select the name matching the board that you are using.

Step 7 – Select your serial port.

Select the serial device of the Arduino board. Go to Tools → Serial Port menu. This is likely to be COM3 or higher (COM1 and COM2 are usually reserved for hardware serial ports). To find out, you can disconnect your Arduino board and re-open the menu, the entry that disappears should be of the Arduino board. Reconnect the board and select that serial port.



Step 8 – Upload the program to your board.

Before explaining how we can upload our program to the board, we must demonstrate the function of each symbol appearing in the Arduino IDE toolbar.



A – Used to check if there is any compilation error.

B – Used to upload a program to the Arduino board.

C – Shortcut used to create a new sketch.

D – Used to directly open one of the example sketch.

E – Used to save your sketch.

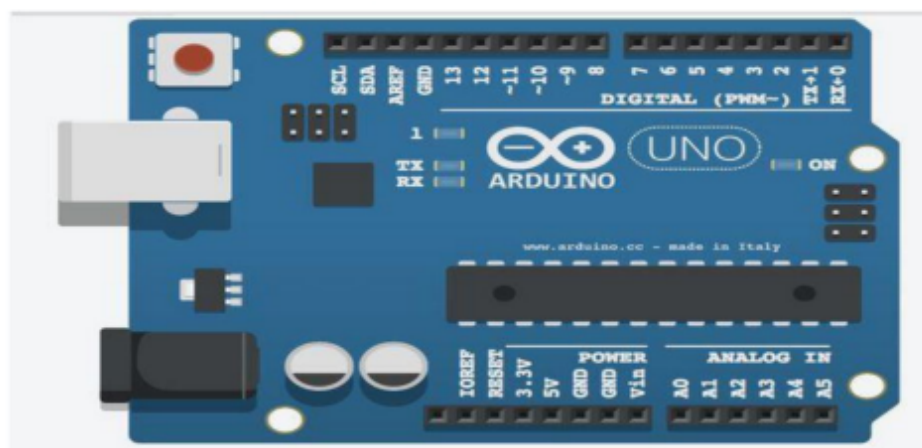
F – Serial monitor used to receive serial data from the board and send the serial data to the board.

Now, simply click the "Upload" button in the environment. Wait a few seconds; you will see the RX and TX LEDs on the board, flashing. If the upload is successful, the message "Done uploading" will appear in the status bar.

IoT Platforms Overview: Arduino, Raspberry Pi

The IoT concepts imply a creation of network of various devices interacting with each other and with their environment. Interoperability and connectivity wouldn't be possible without hardware platforms that help developers solve issues such as building autonomous interactive objects or completing common infrastructure related tasks. Let's go through the most popular IoT platforms and see how they work and benefit IoT software developers.

Arduino



The Arduino platform was created back in 2005 by the Arduino Company and allows for open source prototyping and flexible software development and back-end deployment while providing significant ease of use to developers, even those with very little experience building IoT solutions.

Arduino is sensible to literally every environment by receiving source data from different external sensors and is capable to interact with other control elements over various devices, engines and drives. Arduino has a built-in micro controller that operates on the Arduino software.

Projects based on this platform can be both standalone and collaborative, i.e. realized with use of external tools and plugins. The integrated development environment (IDE) is composed of the open source code and works equally good with Mac, Linux and Windows OS. Based on a processing programming language, the Arduino platform seems to be created for new users and for experiments. The processing language is dedicated to visualizing and building interactive apps using animation and Java Virtual Machine (JVM) platform.

Let's note that this programming language was developed for the purpose of learning basic computer programming in a visual context. It is an absolutely free project available to every interested person. Normally, all the apps are programmed in C/C++, and are wrapped with avr-gcc (WinAVR in OS Windows).

Arduino offers analogue-to-digital input with a possibility of connecting light, temperature or sound sensor modules. Such sensors as SPI or I2C may also be used to cover up to 99% of these apps' market.

Arduino is a microcontroller (generally it is the 8-bit ATmega microcontroller), but not a mini-computer, which makes Arduino somehow limited in its features for advanced users. Arduino provides an excellent interactivity with external devices and offers a wide range of user manuals, project samples as well as a large community of users to learn from / share knowledge with.

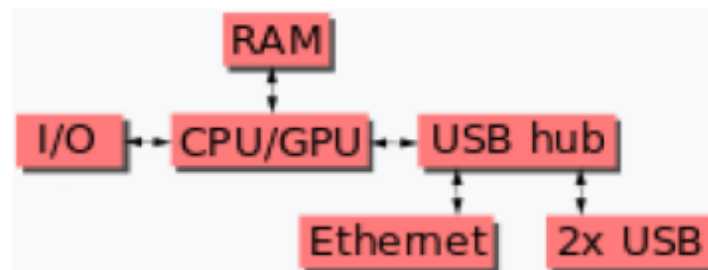
2.3.2 Raspberry Pi

Raspberry Pi (/paɪ/) is a series of small single-board computers developed in the United Kingdom by the Raspberry Pi Foundation in association with Broadcom. Early on, the Raspberry Pi project leaned towards the promotion of teaching basic computer science in schools and in developing countries. Later, the original model became far more popular than anticipated, selling outside its target market for uses such as robotics. It is now widely used in many areas, such as for weather monitoring, because of its low cost, modularity, and open design.

After the release of the second board type, the Raspberry Pi Foundation set up a new entity, named Raspberry Pi Trading, and installed Eben Upton as CEO, with the responsibility of developing technology. The Foundation was rededicated as an educational charity for promoting the teaching

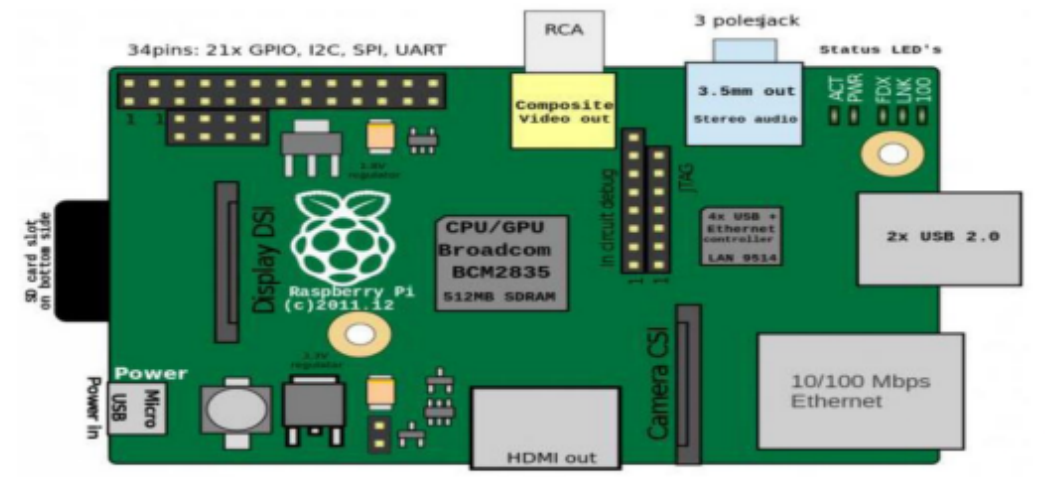
of basic computer science in schools and developing countries. The Raspberry Pi is one of the best-selling British computers.

The Raspberry Pi hardware has evolved through several versions that feature variations in the type of the central processing unit, amount of memory capacity, networking support, and peripheral-device support.



This block diagram describes Model B and B+; Model A, A+, and the Pi Zero are similar, but lack the Ethernet and USB hub components. The Ethernet adapter is internally connected to an additional USB port. In Model A, A+, and the Pi Zero, the USB port is connected directly to the system on a chip (SoC). On the Pi 1 Model B+ and later models the USB/Ethernet chip contains a five-port USB hub, of which four ports are available, while the Pi 1 Model B only provides two. On the Pi Zero, the USB port is also connected directly to the SoC, but it uses a micro USB (OTG) port. Unlike all other Pi models, the 40 pin GPIO connector is omitted on the Pi Zero, with solderable through-holes only in the pin locations. The Pi Zero WH remedies this.

Processor speed ranges from 700 MHz to 1.4 GHz for the Pi 3 Model B+ or 1.5 GHz for the Pi 4; on-board memory ranges from 256 MiB to 1 GiB random-access memory (RAM), with up to 8 GiB available on the Pi 4. Secure Digital (SD) cards in MicroSDHC form factor (SDHC on early models) are used to store the operating system and program memory. The boards have one to five USB ports. For video output, HDMI and composite video are supported, with a standard 3.5 mm tip-ring-sleeve jack for audio output. Lower-level output is provided by a number of GPIO pins, which support common protocols like I²C. The B-models have an 8P8C Ethernet port and the Pi 3, Pi 4 and Pi Zero W have on-board Wi-Fi 802.11n and Bluetooth.



Raspberry Pi is a mono-board computing platform that's as tiny as a credit card. Initially it was developed for computer science education with later on progress to wider functions.

Since the inception of Raspberry, the company sold out more than 8 million items. Raspberry Pi 3 is the latest version and it is the first 64-bit computing board that also comes with built-in Wi-Fi and Bluetooth functions. According to Raspberry Pi Foundation CEO Eben Upton, "it's been a year in the making". The Pi3 version is replaced with a quad-core 64-bit 1.2 GHz ARM Cortex A53 chip, 1GB of RAM, VideoCore IV graphics, Bluetooth 4.1 and 802.11n Wi-Fi. The developers claim the new architecture delivers an average 50% performance improvement over the Pi 2.

Another peculiarity of Raspberry Pi is the GPIO (General Purpose Input-Output), which is a low-level interface of self-operated control by input-output ports. Raspberry has it as a 40-pin connector.

Raspberry Pi uses Linux as its default operating system (OS). It's also fully Android compatible. Using the system on Windows OS is enabled through any virtualization system like Xen Desktop. If you want to develop an application for Raspberry Pi on your computer, it is necessary to download a specific toolset comprised of ARM-compiler and some libraries complied down to ARM-target platform like glibc.

2.3.2.1 Raspberry Pi Installation

Step 1: Download the Raspberry Pi OS image from the Raspberry Pi website:
<https://www.raspberrypi.org/software/operating-systems/>.

Step 2: Write the Raspberry Pi OS image to an SD card

Step 3: Insert the SD card into your Raspberry Pi

Step 4: Connect your Raspberry Pi to your computer using a USB cable

Step 5: Power on your Raspberry Pi

Step 6: Follow the on-screen instructions to set up your Raspberry Pi

2.3.3 Interface and programming using Python

Once you have installed Arduino and Raspberry Pi, you can use Python to interface and program them. Here are some examples:

- **Arduino:** You can use Python to interface with an Arduino board using the pyserial: <https://pypi.org/project/pyserial/> library.
- **Raspberry Pi:** You can use Python to interface with a Raspberry Pi using the RPi.GPIO : <https://pypi.org/project/RPi.GPIO/> library.

Here is an example of code that can be used to interface with an Arduino board using Python and the pyserial library

```
Python
import serial

# Open the serial port
ser = serial.Serial('/dev/ttyACM0', 9600)

# Send a message to the Arduino board
ser.write('Hello, Arduino!')

# Receive a message from the Arduino board
message = ser.readline()

# Print the message
print(message)

# Close the serial port
ser.close()
```

This code will open the serial port at 9600 baud and send the message "Hello, Arduino!" to the Arduino board. The Arduino board will then send a message back to the computer, which will be printed to the console. Finally, the serial port will be closed.

There are many other ways to interface and program Arduino and Raspberry Pi using Python. The specific method that we choose will depend on the needs of our project.

- **Interfacing:** Interfacing involves connecting sensors and actuators to the development board through various interfaces such as GPIO (General Purpose Input/Output) pins or specific communication protocols like I2C, SPI, UART, etc.
- **Programming using Python:** Python is a widely used programming language for IoT development due to its simplicity and ease of use. You can write Python scripts to read data from sensors, process it, and control actuators accordingly.

CHAPTER 3

IOT PLATFORM DESIGN METHODOLOGY

IoT Platform Design Methodology involves the systematic process of developing IoT solutions, including hardware, software, and communication infrastructure.

Here are the key steps in the design methodology:

1. **Identifying Requirements:** Understand the specific requirements of the IoT system, including the objectives, target environment, data to be collected, and desired outcomes.
2. **Selecting Sensors and Actuators:** Choose appropriate sensors and actuators based on the weather monitoring needs. For a weather monitoring system, sensors for temperature, humidity, wind speed, rainfall, etc., would be essential.
3. **Communication Protocol:** Decide on the communication protocol that enables seamless data transfer between the sensors and the central platform. Common protocols include Wi-Fi, LoRaWAN, Zigbee, or cellular networks.
4. **IoT Platform Selection:** Choose the IoT platform that aligns with the project's requirements. The platform should support data collection, storage, analysis, and visualization. Popular choices include AWS IoT, Microsoft Azure IoT, and Google Cloud IoT Core.
5. **Data Security and Privacy:** Implement robust security measures to protect the data from unauthorized access and ensure user privacy.
6. **Data Processing and Analytics:** Set up data processing pipelines to cleanse, aggregate, and analyze the collected data. Apply relevant algorithms for weather prediction and trend analysis.
7. **Integration and Interoperability:** Ensure seamless integration of all components in the IoT system, including sensors, actuators, communication modules, and the IoT platform.
8. **User Interface and Visualization:** Develop an intuitive user interface to visualize weather data and present insights to end-users effectively.
9. **Testing and Validation:** Conduct thorough testing to validate the system's functionality, accuracy, and reliability.
10. **Deployment and Maintenance:** Deploy the IoT weather monitoring system in the target environment and establish a maintenance plan to address issues and updates.

3.1 Case Study on IoT System for Weather Monitor

Case Study: IoT System for Weather Monitoring

Problem:

Weather monitoring is essential for many industries, such as agriculture, transportation, and energy. However, traditional weather monitoring systems are often expensive and difficult to maintain.

Solution:

An IoT system for weather monitoring can be a more cost-effective and scalable solution. This type of system uses sensors to collect weather data, which is then transmitted to a cloud-based platform for processing and analysis.

Benefits:

There are many benefits to using an IoT system for weather monitoring, including:

- **Cost-effectiveness:** IoT systems are typically less expensive to deploy and maintain than traditional weather monitoring systems.
- **Scalability:** IoT systems can be easily scaled to monitor large areas or multiple locations.
- **Real-time data:** IoT systems can collect and transmit weather data in real time, which can be used to make informed decisions.
- **Automated alerts:** IoT systems can be configured to send automated alerts when certain weather conditions are met. This can help to prevent damage or loss of life.

Implementation:

An IoT system for weather monitoring can be implemented in a few simple steps:

1. Select the sensors that will be used to collect weather data.
2. Deploy the sensors in the desired locations.
3. Connect the sensors to a cloud-based platform.
4. Configure the platform to collect and process weather data.
5. Set up alerts for specific weather conditions.

Conclusion:

An IoT system for weather monitoring can be a valuable tool for many industries. This type of system can help to improve decision-making, prevent damage, and save lives.

Here are some additional details about the implementation of an IoT system for weather monitoring:

- The sensors that are used to collect weather data will depend on the specific needs of the application. For example, a system that is used to monitor temperature and humidity will require different sensors than a system that is used to monitor wind speed and direction.
- The sensors can be deployed in a variety of ways, depending on the location and the environment. For example, sensors that are deployed outdoors will need to be weatherproof.
- The sensors are connected to a cloud-based platform using a variety of methods, such as Wi-Fi, cellular, or LoRaWAN.
- The platform that is used to collect and process weather data will need to be able to store large amounts of data and perform complex calculations.
- The alerts that are set up will depend on the specific needs of the application. For example, an alert may be sent when the temperature reaches a certain threshold or when the wind speed exceeds a certain limit.

Note: Use the link for more Details

<https://www.ijert.org/internet-of-things-iot-based-weather-monitoring-system>

<https://iarjset.com/wp-content/uploads/2021/03/IARJSET.2021.8305.pdf>

CHAPTER 4

DATA AND ANALYTICS FOR IOT

- As more and more devices are added to IoT networks, the data generated by these systems becomes overwhelming
- Traditional data management systems are simply unprepared for the demands of what has come to be known as “big data.”
- The real value of IoT is not just in connecting things but rather in *the data produced by those things, the new services you can enable via those connected things, and the business insights that the data can reveal.*
- However, to be useful, the data needs to be handled in a way that is organized and controlled.
- Thus, a new approach to data analytics is needed for the Internet of Things

4.1 An Introduction to Data Analytics for IoT

- In the world of IoT, the creation of massive amounts of data from sensors is common and one of the biggest challenges— not only from a transport perspective but also from a data management standpoint
- Modern jet engines are fitted with thousands of sensors that generate a whopping 10GB of data per second
- *Analyzing this amount of data in the most efficient manner possible falls under the umbrella of data analytics*
- Not all data is the same; it can be categorized and thus analyzed in different ways.
- Depending on how data is categorized, various data analytics tools and processing methods can be applied.
- Two important categorizations from an IoT perspective are whether the data is structured or unstructured and whether it is in motion or at rest.

4.1.1 Structured Versus Unstructured Data

- Structured data and unstructured data are important classifications as they typically require *different toolsets* from a data analytics perspective
- **Structured data means** *that the data follows a model or schema that defines how the data is represented or organized, meaning it fits well with a traditional relational database management system (RDBMS).*

- In many cases you will find structured data in a simple tabular form—for example, a spreadsheet where data occupies a specific cell and can be explicitly defined and referenced
- Structured data can be found in most computing systems and includes everything from banking transaction and invoices to computer log files and router configurations.
- IoT sensor data often uses structured values, such as temperature, pressure, humidity, and so on, which are all sent in a known format.
- Structured data is easily formatted, stored, queried, and processed
- Because of the highly organizational format of structured data, a wide array of data analytics tools are readily available for processing this type of data.
- From custom scripts to commercial software like Microsoft Excel and Tableau
- **Unstructured** data lacks a logical schema for understanding and decoding the data through traditional programming means.
- **Examples** of this data type include **text, speech, images, and video.**
- As a general rule, *any data that does not fit neatly into a predefined data model is classified as unstructured data*
- According to some estimates, around 80% of a business's data is unstructured.
- Because of this fact, data analytics methods that can be applied to unstructured data, such as cognitive computing and machine learning, are deservedly garnering a lot of attention.
- With machine learning applications, such as natural language processing (NLP), you can decode speech.
- With image/facial recognition applications, you can extract critical information from still images and video
- Smart objects in IoT networks generate both structured and unstructured data.
- Structured data is more easily managed and processed due to its well-defined organization.
- On the other hand, unstructured data can be harder to deal with and typically requires very different analytics tools for processing the data

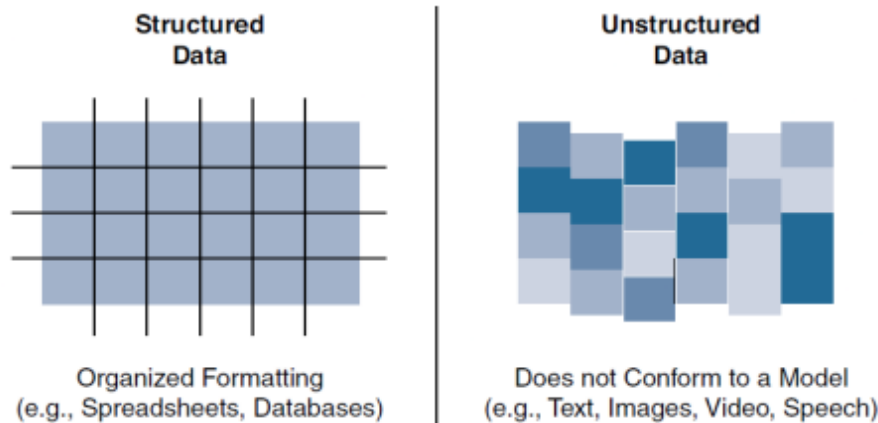


Figure *Comparison Between Structured and Unstructured Data*

4.1.2 Data in Motion Versus Data at Rest

- Data in IoT networks is either in transit (“data in motion”) or being held or stored (“data at rest”).
- Examples of data in motion include traditional client/server exchanges, such as web browsing and file transfers, and email.
- Data saved to a hard drive, storage array, or USB drive is data at rest.
- From an IoT perspective, the data from smart objects is considered data in motion as it passes through the network en route to its final destination.
- This is often processed at the edge, using fog computing.
- When data is processed at the edge, it may be filtered and deleted or forwarded on for further processing and possible storage at a fog node or in the data center.
- Data does not come to rest at the edge.
- When data arrives at the data center, it is possible to process it in real-time, just like at the edge, while it is still in motion.
- Tools with this sort of capability, are Spark, Storm, and Flink
- Data at rest in IoT networks can be typically found in IoT brokers or in some sort of storage array at the data center
- Hadoop not only helps with data processing but also data storage

4.1.3 IoT Data Analytics Overview

- The true importance of IoT data from smart objects is realized only when the analysis of the data leads to actionable business intelligence and insights.
- Data analysis is typically broken down by the types of results that are produced

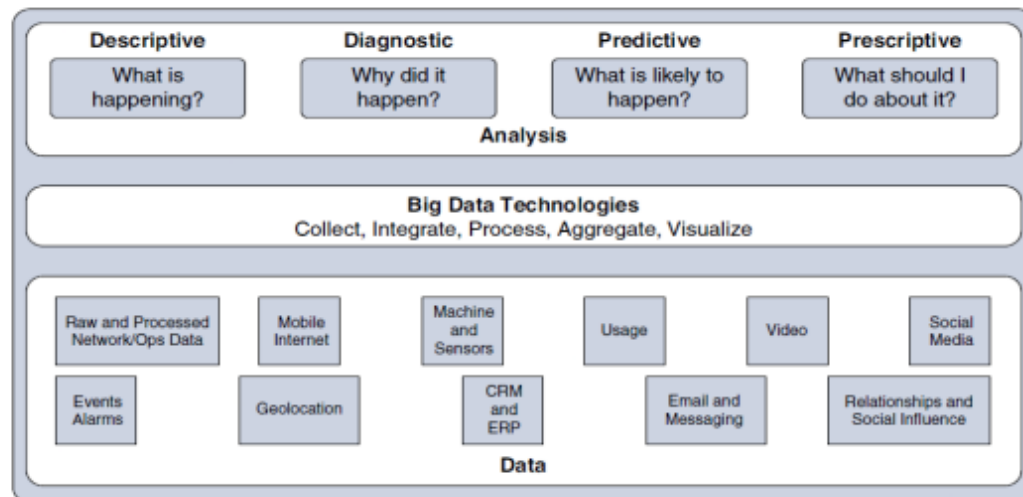


Figure *Types of Data Analysis Results*

There are four types of data analysis results

4.1.3.1 Descriptive:

- Descriptive data analysis tells you what is happening, either now or in the past.
- For example, a thermometer in a truck engine reports temperature values every second.
- From a descriptive analysis perspective, you can pull this data at any moment to gain insight into the current operating condition of the truck engine.
- If the temperature value is too high, then there may be a cooling problem or the engine may be experiencing too much load.

4.1.3.2 Diagnostic:

- When you are interested in the “why,” diagnostic data analysis can provide the answer.
- Continuing with the example of the temperature sensor in the truck engine, you might wonder why the truck engine failed.
- Diagnostic analysis might show that the temperature of the engine was too high, and the engine overheated.
- Applying diagnostic analysis across the data generated by a wide range of smart objects can provide a clear picture of why a problem or an event occurred

4.1.3.3 Predictive

- Predictive analysis aims to foretell problems or issues before they occur.
- For example, with historical values of temperatures for the truck engine, predictive analysis could provide an estimate on the remaining life of certain components in the engine.
- These components could then be proactively replaced before failure occurs.
- Or perhaps if temperature values of the truck engine start to rise slowly over time, this could indicate the need for an oil change or some other sort of engine cooling maintenance.

4.1.3.4 Prescriptive

- Prescriptive analysis goes a step beyond predictive and recommends solutions for upcoming problems.
- A prescriptive analysis of the temperature data from a truck engine might calculate various alternatives to cost-effectively maintain our truck
- These calculations could range from the cost necessary for more frequent oil changes and cooling maintenance to installing new cooling equipment on the engine or upgrading to a lease on a model with a more powerful engine.
- Prescriptive analysis looks at a variety of factors and makes the appropriate recommendation
- Both predictive and prescriptive analyses are more resource intensive and increase complexity, but the value they provide is much greater than the value from descriptive and diagnostic analysis

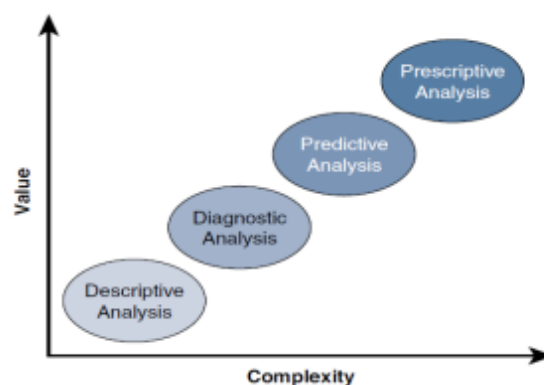


Figure : Application of Value and Complexity Factors to the Types of Data Analysis

4.1.4 IOT Data Analytics Challenges

As IoT has grown and evolved, it has become clear that traditional data analytics solutions were not always adequate. For example, traditional data analytics typically employs a standard RDBMS and corresponding tools, but the world of IoT is much more demanding. While relational databases

are still used for certain data types and applications, they often struggle with the nature of IoT data. IoT data places two specific challenges on a relational database:

- **Scaling Problems** (Performance issues, costly to resolve, req more h/w, architectures changes): Due to the large number of smart objects in most IoT networks that continually send data, relational databases can grow incredibly large very quickly. This can result in performance issues that can be costly to resolve, often requiring more hardware and architecture changes.
- **Volatility of Data** (Change in Schema):With relational databases, it is critical that the schema be designed correctly from the beginning. Changing it later can slow or stop the database from operating. Due to the lack of flexibility, revisions to the schema must be kept at a minimum. IoT data, however, is volatile in the sense that the data model is likely to change and evolve over time. A dynamic schema is often required so that data model changes can be made daily or even hourly.

To deal with challenges like scaling and data volatility, a different type of database, known as NoSQL, is being used. Structured Query Language (SQL) is the computer language used to communicate with an RDBMS. As the name implies, a NoSQL database is a database that does not use SQL. It is not set up in the traditional tabular form of a relational database. NoSQL databases do not enforce a strict schema, and they support a complex, evolving data model. These databases are also inherently much more scalable.

4.2 Machine Learning

- ML is central to IoT.
- Data collected by smart objects needs to be analyzed, and intelligent actions need to be taken based on these analyses.
- Performing this kind of operation manually is almost impossible (or very, very slow and inefficient).
- *Machines are needed to process information fast and react instantly when thresholds are met*
Eg: *advances in self-driving vehicle--abnormal pattern recognition in a crowd and automated intelligent and machine-assisted decision system*

4.2.1 Machine Learning Overview

- Machine learning is, in fact, part of a larger set of technologies commonly grouped under the term artificial intelligence (AI).
- AI includes any technology that allows a computing system to mimic human intelligence using any technique, from very advanced logic to basic “if-then-else” decision loops.

- Any computer that uses rules to make decisions belong to this group
- A simple example is an app that can help you find your parked car.
- A GPS reading of your position at regular intervals calculates your speed.
- A basic threshold system determines whether you are driving (for example, “if speed > 20 mph or 30 kmh, then start calculating speed”).
- When you park and disconnect from the car Bluetooth system, the app simply records the location when the disconnection happens.
- This is where your car is parked.
- In more complex cases, static rules cannot be simply inserted into the program because they require parameters that can change or that are imperfectly understood
- A typical example is a dictation program that runs on a computer.
- The program is configured to recognize the audio pattern of each word in a dictionary, but it does not know your voice’s specifics—your accent, tone, speed, and so on
- You need to record a set of predetermined sentences to help the tool match well-known words to the sounds you make when you say the words.
- This process is called machine learning.
- *ML is concerned with any process where the computer needs to receive a set of data that is processed to help perform a task with more efficiency.*
- ML is a vast field but can be simply divided in two main categories: *supervised and unsupervised learning*

4.2.2 Supervised Learning

- In supervised learning, *the machine is trained with input for which there is a known correct answer.*
- For example, *suppose that you are training a system to recognize when there is a human in a mine tunnel.*
- A sensor equipped with a basic camera can capture shapes and return them to a computing system that is responsible for determining whether the shape is a human or something else (such as a vehicle, a pile of ore, a rock, a piece of wood, and so on.).
- *With supervised learning techniques, hundreds or thousands of images are fed into the machine, and each image is labelled (human or nonhuman in this case).*
- This is called the training set.
- An algorithm is used to *determine common parameters and common differences between the images.*
- The comparison is usually done at the scale of the entire image, or pixel by pixel.

- Images are resized to have the same characteristics (resolution, color depth, position of the central figure, and so on), and each point is analyzed.
- Each new image is compared to the set of known “good images,” and a deviation is calculated to determine how different, the new image is from the average human image and, therefore, the probability that what is shown is a human figure. This process is called classification.
- After training, the machine should be able to recognize human shapes. Before real field deployments, the machine is usually tested with unlabeled pictures— this is called the validation or the test set, depending on the ML system used—to verify that the recognition level is at acceptable thresholds. If the machine does not reach the level of success expected, more training is needed
- In other cases, the learning process is not about classifying in two or more categories but about finding a correct value.
- For example, the speed of the flow of oil in a pipe is a function of the size of the pipe, the viscosity of the oil, pressure, and a few other factors.
- *When you train the machine with measured values, the machine can predict the speed of the flow for a new, and unmeasured, viscosity.*
- This process is called regression; regression predicts numeric values, whereas classification predicts categories

4.2.3 Unsupervised Learning

- In some cases, supervised learning is not the best method for a machine to help with a human decision.
- Suppose that you are processing IoT data from a factory manufacturing small engines.
- You know that about 0.1% of the produced engines on average need adjustments to prevent later defects, and your task is to identify them before they get mounted into machines and shipped away from the factory.
- With hundreds of parts, it may be very difficult to detect the potential defects, and it is almost impossible to train a machine to recognize issues that may not be visible
- However, you can test each engine and record multiple parameters, such as sound, pressure, temperature of key parts, and so on.
- Once data is recorded, you can graph these elements in relation to one another (for example, temperature as a function of pressure, sound versus rotating speed overtime).
- You can then input this data into a computer and use mathematical functions to find groups.

- For example, you may decide to group the engines by the sound they make at a given temperature.
- A standard function to operate this grouping, K-means clustering, finds the mean values for a group of engines (for example, mean value for temperature, mean frequency for sound).
- Grouping the engines this way can quickly reveal several types of engines that all belong to the same category (for example, small engine of chainsaw type, medium engine of lawnmower type).
- All engines of the same type produce sounds and temperatures in the same range as the other members of the same group.
- There will occasionally be an engine in the group that displays unusual characteristics (slightly out of expected temperature or sound range).
- This is the engine that you send for manual evaluation.
- The computing process associated with this determination is called unsupervised learning.
- This type of learning is unsupervised because there is not a “good” or “bad” answer known in advance.
- It is the variation from a group behavior that allows the computer to learn that something is different

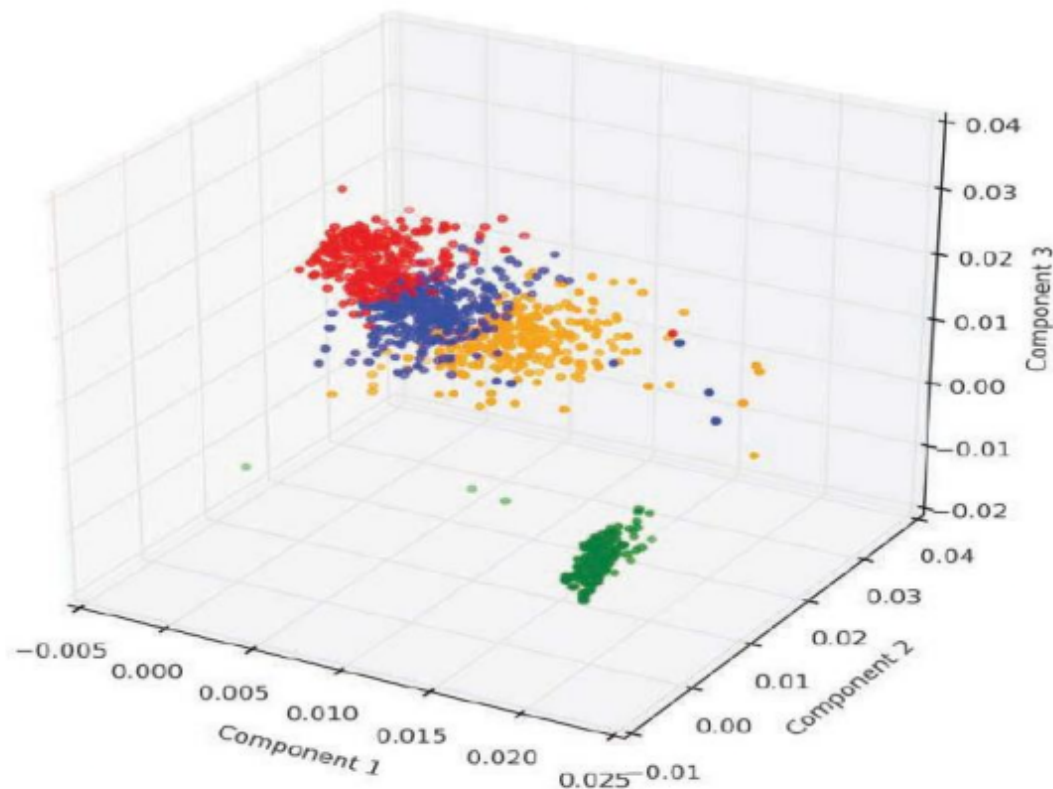


Figure : Clustering and Deviation Detection Example

4.2.4 Neural Networks

- Processing multiple dimensions requires a lot of computing power.
- It is also difficult to determine what parameters to input and what combined variations should raise red flags.
- Similarly, supervised learning is efficient only with a large training set; larger training sets usually lead to higher accuracy in the prediction
- Training the machines was often deemed too expensive and complicated.
- Distinguishing between a human and a car is easy.
- The computer can recognize that humans have distinct shapes (such as legs or arms) and that vehicles do not.
- But, distinguishing a human from another mammal is much more difficult
- The same goes for telling the difference between a pickup truck and a van.
- You can tell when you see one, but training a machine to differentiate them requires more than basic shape recognition
- Neural networks are ML methods that mimic the way the human brain works.
- When you look at a human figure, multiple zones of your brain are activated to recognize colors, movements, facial expressions, and so on.
- Your brain combines these elements to conclude that the shape you are seeing is human
- Neural networks mimic the same logic.
- The information goes through different algorithms (called units), each of which is in charge of processing an aspect of the information
- The resulting value of one unit computation can be used directly or fed into another unit for further processing to occur
- For example, *a neural network processing human image recognition may have two units in a first layer that determines whether the image has straight lines and sharp angles—because vehicles commonly have straight lines and sharp angles, and human figures do not.*
- If the image passes the first layer successfully (because there are no or only a small percentage of sharp angles and straight lines), a second layer may look for different features (presence of face, arms, and so on), and then a *third layer might compare the image to images of various animals and conclude that the shape is a human (or not)*

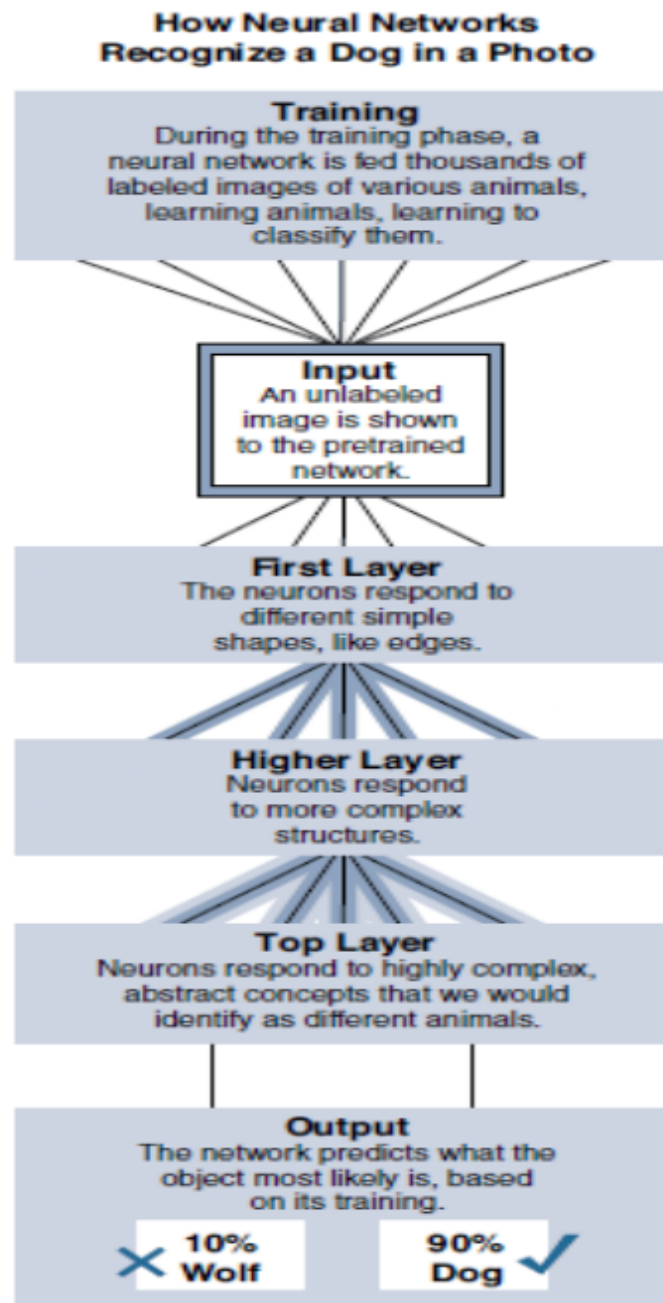


Figure: Neural Network Example

- Neural networks rely on the idea that information is divided into key components, and each component is assigned a weight.
- The weights compared together decide the classification of this information (no straight lines + face + smile = human).
- When the result of a layer is fed into another layer, the process is called deep learning (“deep” because the learning process has more than a single layer).
- One advantage of deep learning is that having more layers allows for richer intermediate processing and representation of the data.
- At each layer, the data can be formatted to be better utilized by the next layer. This process increases the efficiency of the overall result

4.2.5 Machine Learning and Getting Intelligence from Big Data

When the principles of machine learning are clear, the application to IoT becomes obvious. The difficulty resides in determining the right algorithm and the right learning model for each use case. ML operations into two broad subgroups:

- **Local learning:** Data is collected and processed locally, either in the sensor itself (the edge node) or in the gateway (the fog node)
- **Remote learning:** Data is collected and sent to a central computing unit (typically the data center in a specific location or in the cloud), where it is processed.

Regardless of the location where (and, therefore, the scale at which) data is processed, common applications of ML for IoT revolve around four major domains: *They are Monitoring, Behavior control, Operations optimization and Self-healing, self-optimizing.*

Monitoring

- Smart objects monitor the environment where they operate. Example such as air temperature, humidity, or presence of carbon dioxide in a mine etc

Behavior control

- Monitoring commonly works in conjunction with behavior control.
- When a given set of parameters reach a target threshold — defined in advance (that is, supervised) or learned dynamically through deviation from mean values (that is, unsupervised)—monitoring functions generate an alarm.
- This alarm can be relayed to a human, but a more efficient and more advanced system would trigger a corrective action, such as increasing the flow of fresh air in the mine tunnel, turning the robot arm, or reducing the oil pressure in the pipe.

Operations optimization

- Behavior control typically aims at taking corrective actions based on thresholds.
- However, analyzing data can also lead to changes that improve the overall process
- For example, a water purification plant in a smart city can implement a system to monitor the efficiency of the purification process based on which chemical (from company A or company B) is used, at what temperature, and associated to what stirring mechanism (stirring speed and depth).

Self-healing, self-optimizing

- The ML engine can be programmed to dynamically monitor and combine new parameters (randomly or semi-randomly) and automatically deduce and implement new optimizations when the results demonstrate a possible gain.
- The system becomes self-learning and self-optimizing
- *It also detects new K-means deviations that result in redetection of new potential defects, allowing the system to self-heal*

4.2.6 Predictive Analytics

- Machine learning and big data processing for IoT fit very well into the digitization
- The advanced stages of this model see the network self- diagnose and self-optimize.
- In the IoT world, this behavior is what the previous section describes
- When data from multiple systems is combined and analyzed together, predictions can be made about the state of the system. For example,
- Case of sensors deployed on locomotives. Multiple smart objects measure the pull between carriages, the weight on each wheel, and multiple other parameters to offer a form of cruise control optimization for the driver.
- At the same time, cameras observe the state of the tracks ahead, audio sensors analyze the sound of each wheel on the tracks, and multiple engine parameters are measured and analyzed.
- All this data can be returned to a data processing center in the cloud that can re-create a virtual twin of each locomotive.
- Modeling the state of each locomotive and combining this knowledge with anticipated travel and with the states (and detected failures) of all other locomotives of the same type circulating on the tracks of the entire city, province, state, or country allows the analytics platform to make very accurate predictions on what issue is likely to affect each train and each locomotive.
- Such predictive analysis allows preemptive maintenance and increases the safety and efficiency of operations.
- Similarly, sensors combined with big data can anticipate defects or issues in vehicles operating in mines, in manufacturing machines, or any system that can be monitored, along with other similar systems.

4.3 Big Data Analytics Tools and Technology

It is a common mistake for individuals new to the world of data management to use the terms big data and Hadoop interchangeably. Though it's true that Hadoop is at the core of many of today's big data implementations, it's not the only piece of the puzzle. Big data analytics can consist of many different software pieces that together collect, store, manipulate, and analyze all different data types. It helps to better understand the landscape by defining what big data is and what it is not. Generally, the industry looks to the "three Vs" to categorize big data:

Velocity

- Refers to how quickly data is being collected and analyzed.
- Hadoop Distributed File System is designed to ingest and process data very quickly.
- Smart objects can *generate machine and sensor data at a very fast rate and require database or file systems capable of equally fast ingest functions.*

Variety

- Refers to different types of data.
- Often you see data categorized as structured, semi-structured, or unstructured.
- Different database technologies may only be capable of accepting one of these types.
- Hadoop is able to collect and store all three types

Volume

- Refers to the scale of the data.
- Typically, *this is measured from gigabytes on the very low end to petabytes or even exabyte's of data on the other extreme*

The characteristics of big data

- The characteristics of big data can be defined by the sources and types of data.
- First is *machine data*, which is generated by IoT devices and is typically unstructured data.
- Second is *transactional data*, which is from sources that produce data from transactions on these systems, and, have high volume and structured.
- Third is social data sources, which are typically high volume and structured.
- Fourth is enterprise data, which is data that is lower in volume and very structured
- Hence big data consists of data from all these separate sources.

4.3.1 Massively Parallel Processing Databases

- Enterprises have used relational databases for storing structured, row and column style data types for decades.
- Relational databases are often grouped into a broad data storage category called data warehouses.
- Though they are the centerpiece of most data architectures, they are often used for longer-term archiving and data queries that can often take minutes or hours
- *Massively parallel processing (MPP) databases were built on the concept of the relational data warehouses but are designed to be much faster, to be efficient, and to support reduced query times*
- To accomplish this, MPP databases take advantage of multiple nodes (computers) designed in a scale-out architecture such that both data and processing are distributed across multiple systems
- MPPs are sometimes referred to as analytic databases because they are designed to allow for fast query processing and often have built-in analytic functions
- As the name implies, these database types process massive data sets in parallel across many processors and nodes
- An MPP architecture typically contains a single master node that is responsible for the coordination of all the data storage and processing across the cluster.
- It operates in a “shared-nothing” fashion, with each node containing local processing, memory, and storage and operating independently.

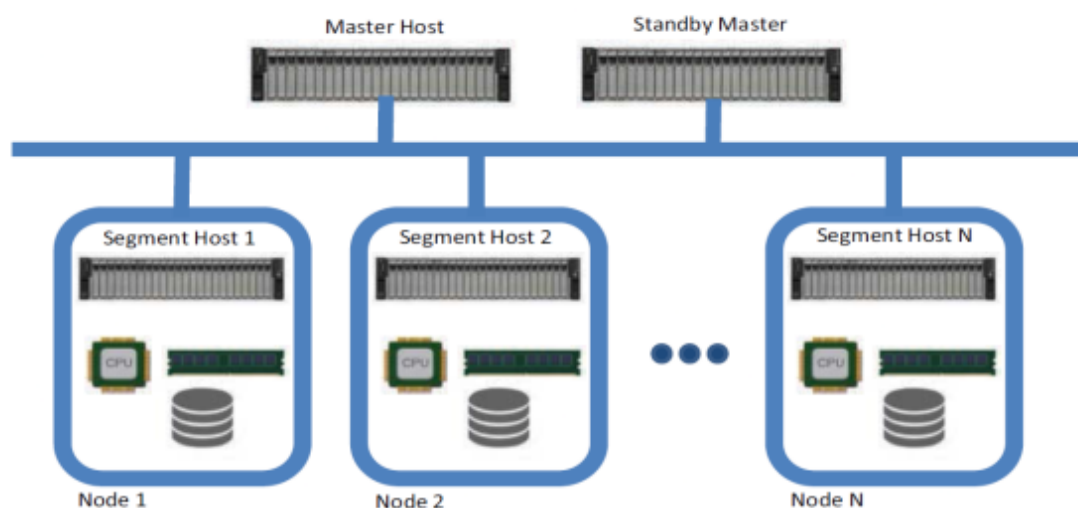


Figure : MPP Shared-Nothing Architecture

- Data storage is optimized across the nodes in a structured SQL-like format that allows data analysts to work with the data using common SQL tools and applications

4.3.2 NoSQL Databases

- NoSQL (“not only SQL”) is a class of databases that support semi-structured and unstructured data, in addition to the structured data handled by data warehouses and MPPs
- NoSQL is not a specific database technology; rather, it is an umbrella term that encompasses several different types of databases, including the following

Document stores:

- This type of database stores semi-structured data, such as XML or JSON.
- Document stores generally have query engines and indexing features that allow for many optimized queries

Key-value stores

- This type of database stores associative arrays where a key is paired with an associated value. These databases are easy to build and easy to scale

Wide-column stores

- This type of database stores similar to a key-value store, but the formatting of the values can vary from row to row, even in the same table

Graph stores

- This type of database is organized based on the relationships between elements.
- Graph stores are commonly used for social media or natural language processing, where the connections between data are very relevant.

4.3.3 Hadoop

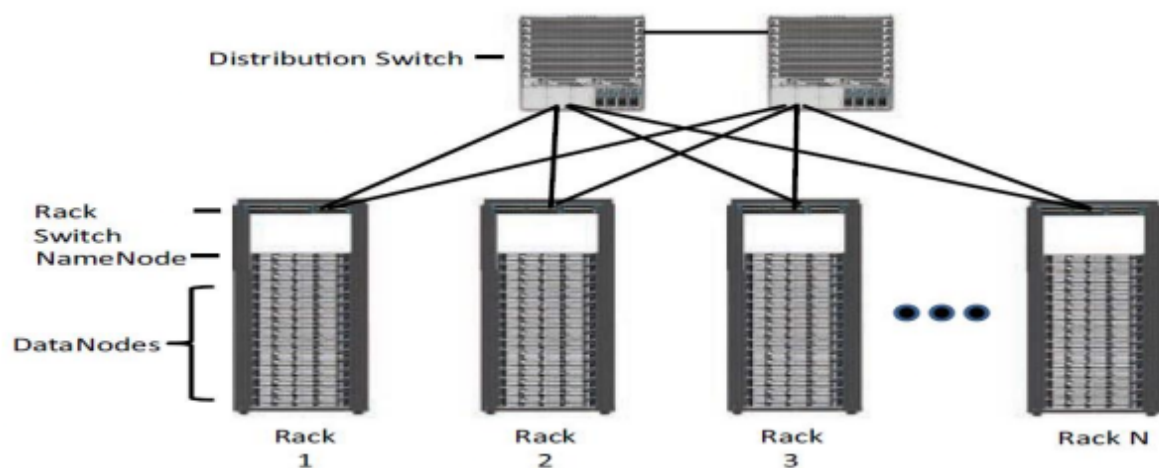
Hadoop is the most recent entrant into the data management market, but it is arguably the most popular choice as a data repository and processing engine. Hadoop was originally developed as a result of projects at Google and Yahoo!, and the original intent for Hadoop was to index millions of websites and quickly return search results for open source search engines. Initially, the project had two key elements:

- **Hadoop Distributed File System (HDFS):** A system for storing data across multiple nodes
- **MapReduce:** A distributed processing engine that splits a large task into smaller ones that can be run in parallel

Both of these elements are still present in current Hadoop distributions and provide.

MapReduce:

- A distributed processing engine that splits a large task into smaller ones that can be run in parallel.
- Hadoop relies on a scale-out architecture that leverages local processing, memory, and storage to distribute tasks and provide a scalable storage system for data.
- Both MapReduce and HDFS take advantage of this distributed architecture to store and process massive amounts of data and are thus able to leverage resources from all nodes in the cluster.
- For HDFS, this capability is handled by specialized nodes in the cluster, including NameNodes and DataNodes

**Fig; Distributed Hadoop Cluster****NameNodes**

- These are a critical piece in data adds, moves, deletes, and reads on HDFS.
- They coordinate where the data is stored, and maintain a map of where each block of data is stored and where it is replicated.
- All interaction with HDFS is coordinated through the primary (active) NameNode, with a secondary (standby) NameNode notified of the changes in the event of a failure of the primary.
- The NameNode takes write requests from clients and distributes those files across the available nodes in configurable block sizes, usually 64 MB or 128 MB blocks.
- The NameNode is also responsible for instructing the DataNodes where replication should occur.

DataNodes

- These are the servers where the data is stored at the direction of the NameNode.
- It is common to have many DataNodes in a Hadoop cluster to store the data.
- Data blocks are distributed across several nodes and often are replicated three, four, or more times across nodes for redundancy.
- Once data is written to one of the DataNodes, the DataNode selects two (or more) additional nodes, based on replication policies, to ensure data redundancy across the cluster

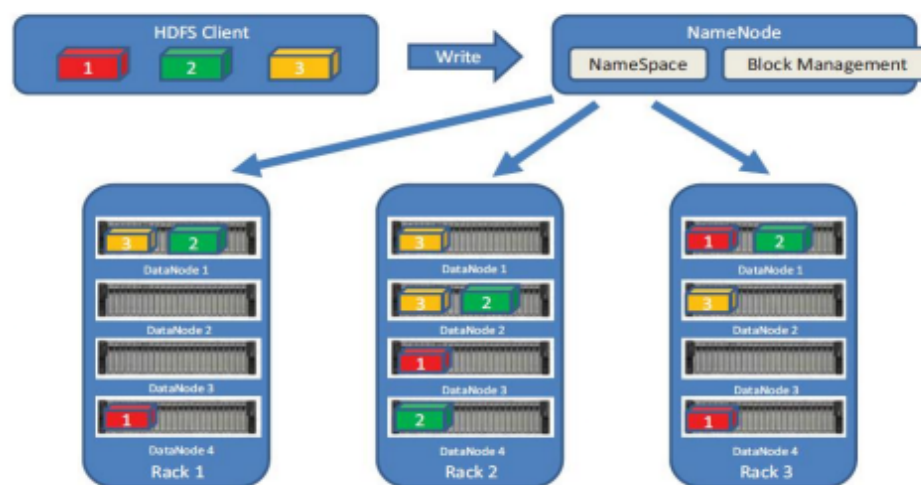


Figure Writing a File to HDFS

4.3.4 Yet Another Resource Negotiator (YARN)

- Introduced with version 2.0 of Hadoop, YARN (Yet Another Resource Negotiator) was designed to enhance the functionality of MapReduce.
- With the initial release, MapReduce was responsible for batch data processing and job tracking and resource management across the cluster.
- YARN was developed to take over the resource negotiation and job/task tracking, allowing MapReduce to be responsible only for data processing.

4.3.5 The Hadoop Ecosystem

- Since the initial release of Hadoop in 2011, many projects have been developed to add incremental functionality to Hadoop and have collectively become known as the Hadoop ecosystem.

○ Apache Kafka

Part of processing real-time events, such as those commonly generated by smart objects, is having them ingested into a processing engine. The process of collecting data from a sensor or log file and preparing it to be processed and analyzed is typically handled by messaging systems. Messaging systems are designed to accept data, or messages, from where the data is generated and deliver the data to stream-processing engines such as Spark Streaming or Storm. Apache Kafka is a distributed publisher-subscriber messaging system that is built to be scalable and fast. It is composed of topics, or message brokers, where producers write data and consumers read data from these topics. Figure: Apache Kafka Data Flow shows the data flow from the smart objects (producers), through a topic in Kafka, to the real-time processing engine. Due to the distributed nature of Kafka, it can run in a clustered configuration that can handle many producers and consumers simultaneously and exchanges information between nodes, allowing topics to be distributed over multiple nodes. The goal of Kafka is to provide a simple way to connect to data sources and allow consumers to connect to that data in the way they would like. The following sections describe several of these packages and discusses how they are used to collect or process data.

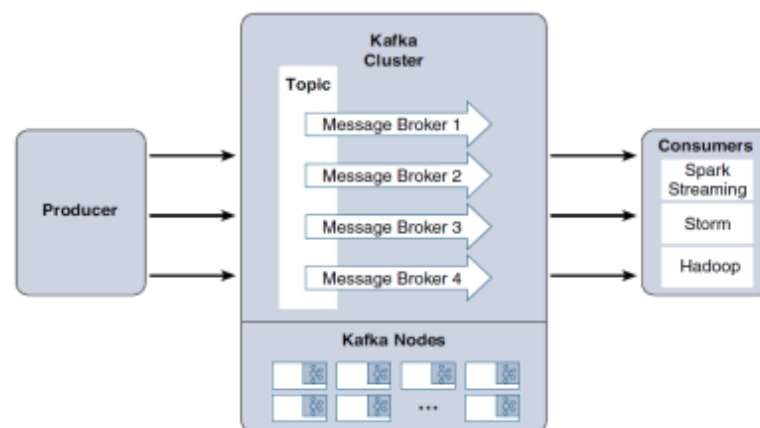


Figure Apache Kafka Data Flow

○ Apache Spark

Apache Spark is an in-memory distributed data analytics platform designed to accelerate processes in the Hadoop ecosystem. The “in-memory” characteristic of Spark is what enables it to run jobs very quickly. At each stage of a MapReduce operation, the data is read and written back to the disk, which means latency is introduced through each disk operation. However, with Spark, the processing of this data is moved into high-speed memory, which has significantly lower latency. This speeds the batch processing jobs and also allows for near-real-time processing of events.

Real-time processing is done by a component of the Apache Spark project called Spark Streaming. Spark Streaming is an extension of Spark Core that is responsible for taking live streamed data from a messaging system, like Kafka, and dividing it into smaller microbatches. These microbatches are called discretized streams, or DStreams. The Spark processing engine is able to operate on these smaller pieces of data, allowing rapid insights into the data and subsequent actions. Due to this “instant feedback” capability, Spark is becoming an important component in many IoT deployments. Systems that control safety and security of personnel, time-sensitive processes in the manufacturing space, and infrastructure control in traffic management all benefit from these real-time streaming capabilities.

○ **Apache Storm and Apache Flink**

As you work with the Hadoop ecosystem, you will inevitably notice that different projects are very similar and often have significant overlap with other projects. This is the case with data streaming capabilities. For example, Apache Spark is often used for both distributed streaming analytics and batch processing. Apache Storm and Apache Flink are other Hadoop ecosystem projects designed for distributed stream processing and are commonly deployed for IoT use cases. Storm can pull data from Kafka and process it in a near-real-time fashion, and so can Apache Flink. This space is rapidly evolving, and projects will continue to gain and lose popularity as they evolve.

○ **Lambda Architecture**

Ultimately the key elements of a data infrastructure to support many IoT use cases involves the collection, processing, and storage of data using multiple technologies. Querying both data in motion (streaming) and data at rest (batch processing) requires a combination of the Hadoop ecosystem projects discussed. One architecture that is currently being leveraged for this functionality is the Lambda Architecture. Lambda is a data management system that consists of two layers for ingesting data (Batch and Stream) and one layer for providing the combined data (Serving). These layers allow for the packages discussed previously, like Spark and MapReduce, to operate on the data independently, focusing on the key attributes for which they are designed and optimized. Data is taken from a message broker, commonly Kafka, and processed by each layer in parallel, and the resulting data is delivered to a data store where additional processing or queries can be run. Figure Lambda Architecture shows this parallel data flow through the Lambda Architecture.

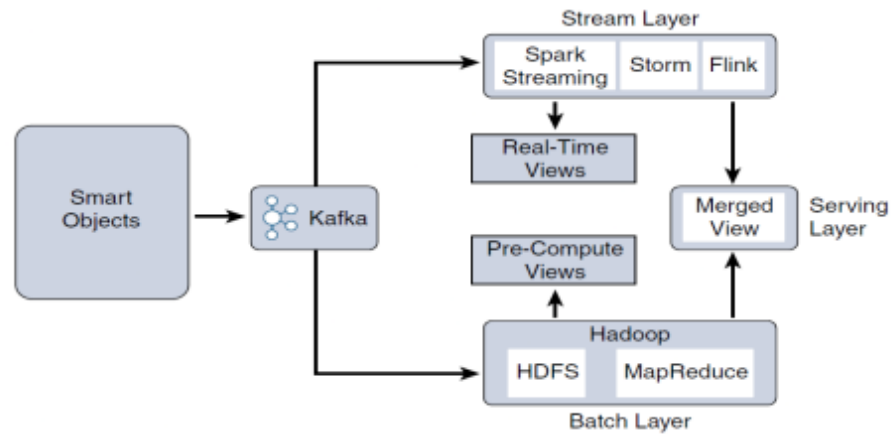


Figure - *Lambda Architecture*

The Lambda Architecture is not limited to the packages in the Hadoop ecosystem, but due to its breadth and flexibility, many of the packages in the ecosystem fill the requirements of each layer nicely:

- **Stream layer:** This layer is responsible for near-real-time processing of events. Technologies such as Spark Streaming, Storm, or Flink are used to quickly ingest, process, and analyze data on this layer. Alerting and automated actions can be triggered on events that require rapid response or could result in catastrophic outcomes if not handled immediately.
- **Batch layer:** The Batch layer consists of a batch-processing engine and data store. If an organization is using other parts of the Hadoop ecosystem for the other layers, MapReduce and HDFS can easily fit the bill. Other database technologies, such as MPPs, NoSQL, or data warehouses, can also provide what is needed by this layer.
- **Serving layer:** The Serving layer is a data store and mediator that decides which of the ingest layers to query based on the expected result or view into the data. If an aggregate or historical view is requested, it may invoke the Batch layer. If real-time analytics is needed, it may invoke the Stream layer. The Serving layer is often used by the data consumers to access both layers simultaneously.

The Lambda Architecture can provide a robust system for collecting and processing massive amounts of data and the flexibility of being able to analyze that data at different rates. One limitation of this type of architecture is its place in the network. Due to the processing and storage requirements of many of these pieces, the vast majority of these deployments are either in data centers or in the cloud. This could limit the effectiveness of the analytics to respond rapidly enough if the processing systems are milliseconds or seconds away from the device generating the data. When this is the case, a distributed edge-processing architecture may be needed to augment the central data center infrastructure.

4.4 Edge Streaming Analytics

A major area of evolution for IT in the past few years has been the transition to cloud services. Nearly every large technology company is now selling software and services from the cloud, and this includes data analytics systems, whether they are offered as a service from a public cloud operator or are built in massive private data center clouds. However, analyzing a massive volume of time-sensitive IoT data in a centralized cloud is often not ideal.

In the world of IoT, vast quantities of data are generated on the fly and often need to be analyzed and responded to immediately. Not only is the volume of data generated at the edge immense—meaning the bandwidth requirements to the cloud or data center need to be engineered to match—but the data may be so time sensitive that it needs immediate attention, and waiting for deep analysis in the cloud simply isn't possible.

One industry where data analytics is used extensively is the world of automobile racing. For example, in Formula One racing, each car has between 150 to 200 sensors that, combined, generate more than 1000 data points per second, resulting in hundreds of gigabytes of raw data per race. The sensor data is transmitted from the car and picked up by track-side wireless sensors. During a race, weather conditions may vary, tire conditions change, and accidents or other racing incidents almost always require an adaptable and flexible racing strategy. As the race develops, decisions such as when to pit, what tires to use, when to pass, and when to slow down all need to be made in seconds. Teams have found that enormous insights leading to better race results can be gained by analyzing data on the fly—and the data may come from many different sources, including trackside sensors, car telemetry, and weather reports.

Most teams use sophisticated data analytics systems to enhance racing strategy, but in many cases, this equipment resides back in the team's data center, far away from the track. For a team that has its analytics software in a data center in the UK, the latency to Australia (the most remote race) is several hundred milliseconds away. The time it takes to collect and analyze this data as a batch process in a distant part of the world is not only inefficient but can mean the difference between a successful race strategy that adapts to changing conditions and one that lacks the flexibility and agility to send meaningful instructions to the drivers. In short, it can mean the difference between winning and losing a race.

4.4.1 Comparing Big Data and Edge Analytics

- When you hear the term big data, it is usually in reference to unstructured data that has been collected and stored in the cloud

- Tools like Hadoop and MapReduce are great at tackling problems that require deep analytics on a large and complex quantity of unstructured data;
- However, due to their distance from the IoT endpoints and the bandwidth required to bring all the data back to the cloud, they are generally not well suited to real-time analysis of data as it is generated.
- In applying data analytics to the car racing example, big data analytics is used to examine all the statistics of the racing team and players based on their performance in the data center or cloud
- Streaming analytics involves analyzing a race while it is happening and trying to figure out who is going to win based on the actual performance in real-time—and this analysis is typically performed as close to the edge as possible.
- Streaming analytics allows you to continually monitor and assess data in real-time so that you can adjust or fine-tune your predictions as the race progresses.
- In the context of IoT, with streaming analytics performed at the edge (either at the sensors themselves or very close to them, in a fog node that is, for example, integrated into the gateway), it is possible to process and act on the data in realtime without waiting for the results from a future batch- processing job in the cloud.

The key values of edge streaming analytics include the following:

- **Reducing data at the edge:** The aggregate data generated by IoT devices is generally in proportion to the number of devices. The scale of these devices is likely to be huge, and so is the quantity of data they generate. Passing all this data to the cloud is inefficient and is unnecessarily expensive in terms of bandwidth and network infrastructure.
- **Analysis and response at the edge:** Some data is useful only at the edge (such as a factory control feedback system). In cases such as this, the data is best analyzed and acted upon where it is generated.
- **Time sensitivity:** When timely response to data is required, passing data to the cloud for future processing results in unacceptable latency. Edge analytics allows immediate responses to changing conditions.

4.4.2 Edge Analytics Core Functions

To perform analytics at the edge, data needs to be viewed as real-time flows. Whereas big data analytics is focused on large quantities of data at rest, edge analytics continually processes streaming flows of data in motion Streaming analytics at the edge can be broken down into three simple stages:

- **Raw input data:** This is the raw data coming from the sensors into the analytics processing unit.
- **Analytics processing unit (APU):** The APU filters and combines data streams (or separates the streams, as necessary), organizes them by time windows, and performs various analytical functions. It is at this point that the results may be acted on by micro services running in the APU
- **Output streams:** The data that is output is organized into insightful streams and is used to influence the behavior of smart objects, and passed on for storage and further processing in the cloud. Communication with the cloud often happens through a standard publisher/subscriber messaging protocol, such as MQTT

The Below Figure illustrates the stages of data processing in an edge APU.

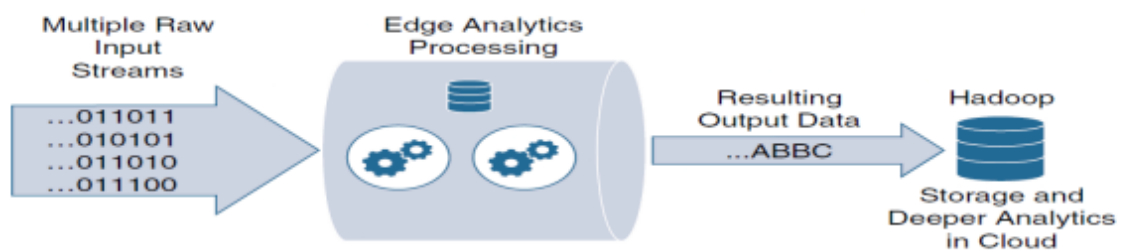


Figure - *Edge Analytics Processing Unit*

In order to perform analysis in real-time, the APU needs to perform the following functions:

■ **Filter:** The streaming data generated by IoT endpoints is likely to be very large, and most of it is irrelevant. For example, a sensor may simply poll on a regular basis to confirm that it is still reachable. This information is not really relevant and can be mostly ignored. The filtering function identifies the information that is considered important.

■ **Transform:** In the data warehousing world, Extract, Transform, and Load (ETL) operations are used to manipulate the data structure into a form that can be used for other purposes. Analogous to data warehouse ETL operations, in streaming analytics, once the data is filtered, it needs to be formatted for processing.

■ **Time:** As the real-time streaming data flows, a timing context needs to be established. This could be to correlated average temperature readings from sensors on a minute-by-minute basis. For example, (Figure Example: Establishing a Time Window for Analytics of Average Temperature from Sensors) shows an APU that takes input data from multiple sensors reporting temperature fluctuations. In this case, the APU is programmed to report the average temperature every minute from the sensors, based on an average of the past two minutes. (An example where this may be

used in real-time monitoring of food in a grocery store, where rolling averages of the temperature in open-air refrigeration units needs to be monitored to ensure the safety of the food.) Note that on the left side is the cleaned stream data. This data is presented as streams to the analytics engine (note the syntax at the bottom right of the figure) that establishes the time window and calculates the average temperature over the past two minutes. The results are reported on a per-minute basis (on the right side of the figure).

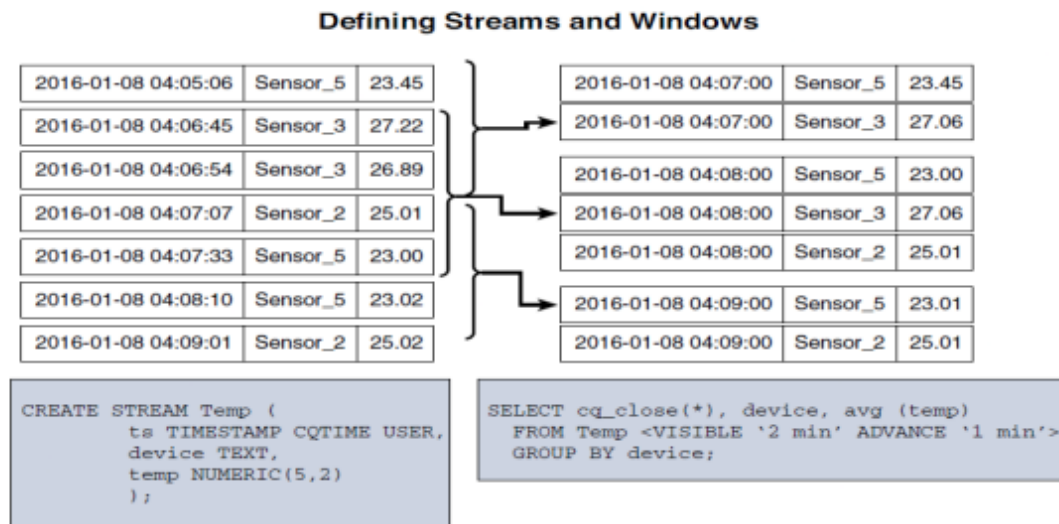


Figure - Example: Establishing a Time Window for Analytics of Average Temperature from Sensors

■ **Correlate:** Streaming data analytics becomes most useful when multiple data streams are combined from different types of sensors. For example, in a hospital, several vital signs are measured for patients, including body temperature, blood pressure, heart rate, and respiratory rate. These different types of data come from different instruments, but when this data is combined and analyzed, it provides an invaluable picture of the health of the patient at any given time.⁴ However, correlation goes beyond just combining real-time data streams. Another key aspect is combining and correlating real-time measurements with preexisting, or historical, data. For example, historical data may include the patient's past medical history, such as blood test results. Combining historical data gives the live streaming data a powerful context and promotes more insights into the current condition of the patient (see Figure Correlating Data Streams with Historical Data).

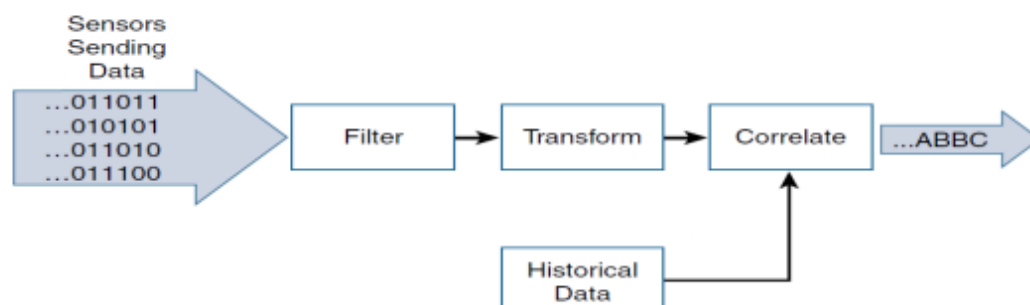


Figure - Correlating Data Streams with Historical Data

- **Match patterns:** Once the data streams are properly cleaned, transformed, and correlated with other live streams as well as historical data sets, pattern matching operations are used to gain deeper insights to the data. For example, say that the APU has been collecting the patient's vitals for some time and has gained an understanding of the expected patterns for each variable being monitored. If an unexpected event arises, such as a sudden change in heart rate or respiration, the pattern matching operator recognizes this as out of the ordinary and can take certain actions, such as generating an alarm to the nursing staff. The patterns can be simple relationships, or they may be complex, based on the criteria defined by the application. Machine learning may be leveraged to identify these patterns.
- **Improve business intelligence:** Ultimately, the value of edge analytics is in the improvements to business intelligence that were not previously available. For example, conducting edge analytics on patients in a hospital allows staff to respond more quickly to the patient's changing needs and also reduces the volume of unstructured (and not always useful) data sent to the cloud. Over time, the resulting changes in business logic can produce improvements in basic operations, bringing in higher levels of care as well as better efficiencies for the hospital.

Distributed Analytics Systems

- Depending on the application and network architecture, analytics can happen at any point throughout the IoT system.
- Streaming analytics may be performed directly at the edge, in the fog, or in the cloud data center.
- There are no hard and- fast rules dictating where analytics should be done, but there are a few guiding principles
- Sometimes better insights can be gained and data responded to more intelligently when we step back from the edge and look at a wider data set.
- This is the value of fog computing.
- Fog analytics allows you to see beyond one device, giving you visibility into an aggregation of edge nodes and allowing you to correlate data from a wider set
- Example of an oil drilling company that is measuring both pressure and temperature on an oil rig

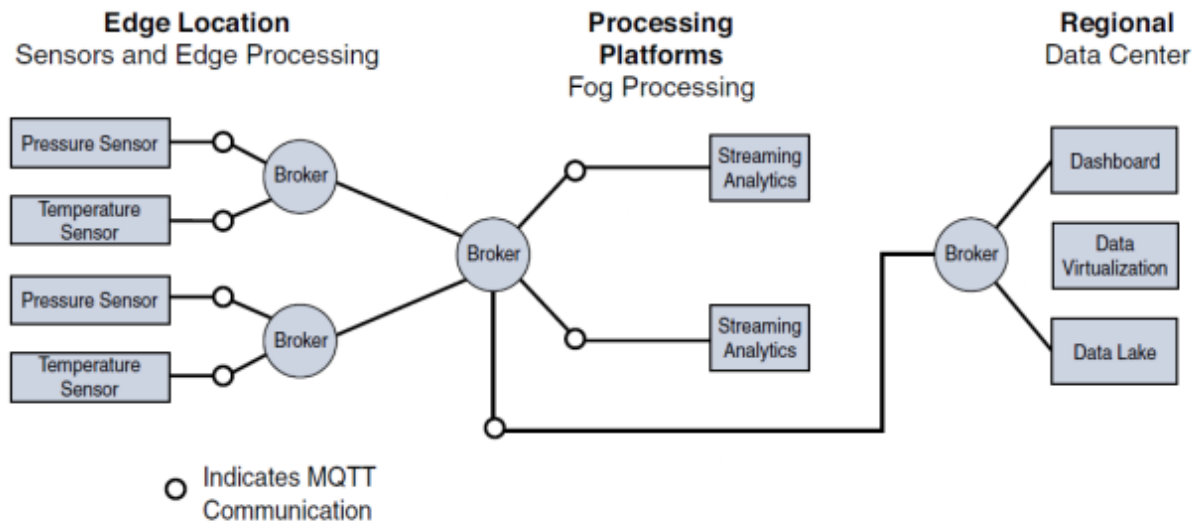


Figure - *Distributed Analytics Throughout the IoT System*

4.5 Network Analytics

- Another form of analytics that is extremely important in managing IoT systems is network-based analytics
- Network analytics is concerned with discovering patterns in the communication flows from a network traffic perspective.
- Network analytics has the power to analyze details of communications patterns made by protocols and correlate this across the network.
- It allows you to understand what should be considered normal behavior in a network and to quickly identify anomalies that suggest network problems due to suboptimal paths, intrusive malware, or excessive congestion.

IoT endpoints, contrary to generic computing platforms, are designed to directly communicate with a very small number of specific application servers, such as an IoT message or data broker, or specific application servers and network management systems. Therefore, it could be said that IoT solutions and use cases tightly couple devices and applications. (Figure: Smart Grid FAN Analytics with NetFlow Example) shows field area network (FAN) traffic analytics performed on the aggregation router in a smart grid.

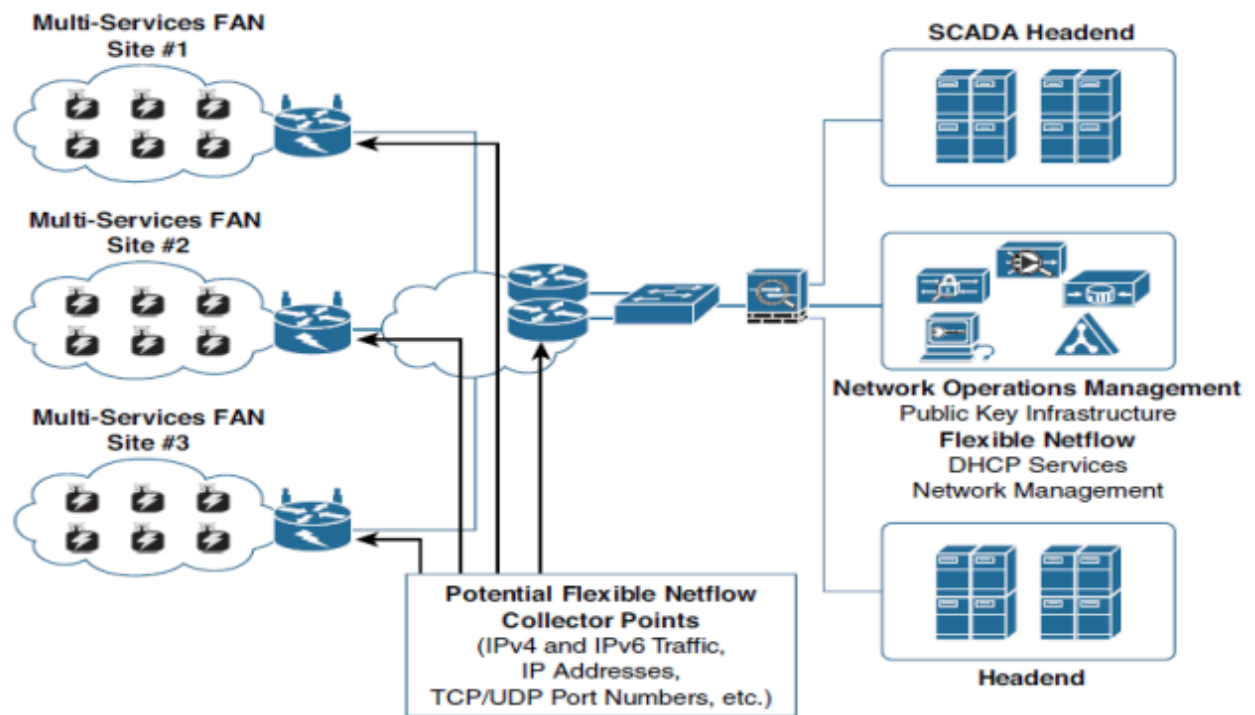


Figure - *Smart Grid FAN Analytics with NetFlow Example*

This behavior represents a key aspect that can be leveraged when performing network analytics: Network analytics offer capabilities to cope with capacity planning for scalable IoT deployment as well as security monitoring in order to detect abnormal traffic volume and patterns (such as an unusual traffic spike for a normally quiet protocol) for both centralized or distributed architectures, such as fog computing.

Consider that an IoT device sends its traffic to specific servers, either directly to an application or an IoT broker with the data payload encapsulated in a given protocol. This represents a pair of source and destination addresses, as well as application layer-dependent TCP or UDP port numbers, which can be used for network analytics.

One of the drivers of the adoption of an IP architectural framework for IoT is to leverage tools and processes largely known and deployed by Internet service providers (ISPs) as well as private corporate enterprise networks. To monitor network infrastructure, de facto industry standards and protocols allow pervasive characterization of IP traffic flows, including identification of source and/or destination addresses, data timing and volume, and application types within a network infrastructure. Flow statistics can be collected at different locations in the network. For example, centralized routers or switches that mile of the infrastructure can be used to collect flow information. After data is collected in a known format, it can be sent to an external network analytics tools that delivers unique services to network managers, like security and performance monitoring and capacity planning.

CHAPTER 5

IOT PHYSICAL SERVERS AND CLOUD OFFERING

5.1 Cloud storage models of IoT Systems

Cloud storage in IoT systems allows the seamless and scalable management of the vast amounts of data generated by IoT devices. There are primarily three cloud storage models used in IoT:

- **Object Storage:** Object storage is a scalable and distributed storage model that stores data as objects, each with a unique identifier. It is ideal for unstructured data, such as images, videos, sensor readings, and log files. Examples of object storage services include Amazon S3 (Simple Storage Service) and Google Cloud Storage.
- **Time-Series Database:** Time-series databases are designed to efficiently store and manage time-stamped data, which is common in IoT applications. They are optimized for handling time-series data streams, making them suitable for storing sensor data and historical trends. Examples include InfluxDB and TimescaleDB.
- **Relational Database:** In some IoT applications, structured data needs to be stored and organized in a relational format. Relational databases, such as MySQL, PostgreSQL, or Microsoft SQL Server, can be used for this purpose. They are suitable for storing metadata, configuration settings, and relational data generated by IoT systems.

5.2 Communication APIs of IoT Systems

Communication APIs enable seamless data exchange and interaction between IoT devices, applications, and cloud platforms. Some common communication APIs used in IoT systems are:

- **MQTT (Message Queuing Telemetry Transport):** MQTT is a lightweight and efficient messaging protocol designed for constrained devices and low-bandwidth networks. It is widely used in IoT for its publish-subscribe communication pattern and minimal overhead. MQTT facilitates real-time data streaming between devices and the cloud.
- **CoAP (Constrained Application Protocol):** CoAP is a specialized protocol designed for resource-constrained devices and networks, such as those commonly found in IoT. It is particularly useful for applications that require efficient communication and minimal data exchange.

- **HTTP/REST APIs:** HTTP-based RESTful APIs are widely used in IoT applications for their simplicity and compatibility with web services. They allow devices and applications to interact with cloud platforms and access various IoT services through standard HTTP methods like GET, POST, PUT, and DELETE.
- **WebSockets:** WebSockets provide full-duplex communication channels over a single TCP connection, allowing real-time bidirectional data transfer between clients (IoT devices or applications) and servers (cloud platforms).
- **AMQP (Advanced Message Queuing Protocol):** AMQP is an open-standard messaging protocol designed for reliable message-oriented communication. It provides features like message queuing, routing, and broker-based communication, making it suitable for IoT applications with high reliability requirements.

CHAPTER 6

SECURING IOT SYSTEMS

6.1 IoT Security Challenges

The security challenges faced in IoT are by no means new and are not limited to specific industrial environments. The following sections discuss some of the common challenges faced in IoT.

6.1.1 Erosion of Network Architecture

Two of the major challenges in securing industrial environments have been initial design and ongoing maintenance. The initial design challenges arose from the concept that networks were safe due to physical separation from the enterprise with minimal or no connectivity to the outside world, and the assumption that attackers lacked sufficient knowledge to carry out security attacks. In many cases, the initial network design is sound and even follows well-defined industrial best practices and standards, such as the Purdue Model for Control Hierarchy, “IoT Network Architecture and Design.” The challenge, and the biggest threat to network security, is standards and best practices either being misunderstood or the network being poorly maintained. In fact, from a security design perspective, it is better to know that communication paths are insecure than to not know the actual communication paths. It is more common that, over time, what may have been a solid design to begin with is eroded through ad hoc updates and individual changes to hardware and machinery without consideration for the broader network impact. This kind of organic growth has led to miscalculations of expanding networks and the introduction of wireless communication in a standalone fashion, without consideration of the impact to the original security design. These uncontrolled or poorly controlled OT network evolutions have, in many cases, over time led to weak or inadequate network and systems security.

There is a wide variety in secured network designs within and across different industries. For example, power utilities have a strong history of leveraging modern technologies for operational activities, and in North America there are regulatory requirements in place from regulatory authorities, such as North American Electric Reliability Corporation’s (NERC’s) Critical Infrastructure Protection (CIP), “Utilities”, to implement secure network connectivity and control with reasonably prescriptive actions. By contrast, in other industries, there are often no legislative requirements or compliance policies, which has resulted in widespread differences in security capabilities.

In many industries, the control systems consist of packages, skids, or components that are self-contained and may be integrated as semi-autonomous portions of the network. These packages may not be as fully or tightly integrated into the overall control system, network management tools, or security applications, resulting in potential risk.

6.1.2 Pervasive Legacy Systems

Due to the static nature and long lifecycles of equipment in industrial environments, many operational systems may be deemed legacy systems. For example, in a power utility environment, it is not uncommon to have racks of old mechanical equipment still operating alongside modern intelligent electronic devices (IEDs). In many cases, legacy components are not restricted to isolated network segments but have now been consolidated into the IT operational environment. From a security perspective, this is potentially dangerous as many devices may have historical vulnerabilities or weaknesses that have not been patched and updated, or it may be that patches are not even available due to the age of the equipment.

Beyond the endpoints, the communication infrastructure and shared centralized compute resources are often not built to comply with modern standards. In fact, their communication methods and protocols may be generations old and must be interoperable with the oldest operating entity in the communications path. This includes switches, routers, firewalls, wireless access points, servers, remote access systems, patch management, and network management tools. All of these may have exploitable vulnerabilities and must be protected.

6.1.3 Insecure Operational Protocols

Many industrial control protocols, particularly those that are serial based, were designed without inherent strong security requirements. Furthermore, their operation was often within an assumed secure network. In addition to any inherent weaknesses or vulnerabilities, their operational environment may not have been designed with secured access control in mind.

Industrial protocols, such as supervisory control and data acquisition (SCADA) particularly the older variants, suffer from common security issues. Three examples of this are a frequent lack of authentication between communication endpoints, no means of securing and protecting data at rest or in motion, and insufficient granularity of control to properly specify recipients or avoid default broadcast approaches. These may not be as critical in self-contained systems, but between zones or on longer network segments, such as a WAN (particularly a public WAN), they may be significant considerations.

The structure and operation of most of these protocols is often publicly available. While they may have been originated by a private firm, for the sake of interoperability, they are typically published for others to implement. Thus, it becomes a relatively simple matter to compromise the protocols themselves and introduce malicious actors that may use them to compromise control systems for either reconnaissance or attack purposes that could lead to undesirable impacts in normal system operation.

The following sections discuss some common industrial protocols and their respective security concerns. Note that many have serial, IP, or Ethernet-based versions, and the security challenges and vulnerabilities are different for the different variants.

6.1.4 Modbus

Modbus is commonly found in many industries, such as utilities and manufacturing environments, and has multiple variants (for example, serial, TCP/IP). It was created by the first programmable logic controller (PLC) vendor, Modicon, and has been in use since the 1970s. It is one of the most widely used protocols in industrial deployments, and its development is governed by the Modbus Organization.

The security challenges that have existed with Modbus are not unusual. Authentication of communicating endpoints was not a default operation because it would allow an inappropriate source to send improper commands to the recipient. For example, for a message to reach its destination, nothing more than the proper Modbus address and function call (code) is necessary. Some older and serial-based versions of Modbus communicate via broadcast. The ability to curb the broadcast function does not exist in some versions. There is potential for a recipient to act on a command that was not specifically targeting it. Furthermore, an attack could potentially impact unintended recipient devices, thus reducing the need to understand the details of the network topology.

Validation of the Modbus message content is also not performed by the initiating application.

Instead, Modbus depends on the network stack to perform this function. This could open up the potential for protocol abuse in the system.

6.1.5 DNP3 (Distributed Network Protocol)

DNP3 is found in multiple deployment scenarios and industries. It is common in utilities and is also found in discrete and continuous process systems. Like many other ICS/SCADA protocols, it was intended for serial communication between controllers and simple IEDs.

There is an explicit “secure” version of DNP3, but there also remain many insecure implementations of DNP3 as well. DNP3 has placed great emphasis on the reliable delivery of messages. That emphasis, while normally highly desirable, has a specific weakness from a security perspective. In the case of DNP3, participants allow for unsolicited responses, which could trigger an undesired response. The missing security element here is the ability to establish trust in the system’s state and thus the ability to trust the veracity of the information being presented. This is akin to the security flaws presented by Gratuitous ARP messages in Ethernet networks, which has been addressed by Dynamic ARP Inspection (DAI) in modern Ethernet switches.

6.1.6 ICCP (Inter-Control Center Communications Protocol)

ICCP is a common control protocol in utilities across North America that is frequently used to communicate between utilities. Given that it must traverse the boundaries between different networks, it holds an extra level of exposure and risk that could expose a utility to cyber-attack.

Unlike other control protocols, ICCP was designed from inception to work across a WAN. Despite this role, initial versions of ICCP had several significant gaps in the area of security. One key vulnerability is that the system did not require authentication for communication. Second, encryption across the protocol was not enabled as a default condition, thus exposing connections to man-in-the-middle (MITM) and replay attacks.

6.1.7 OPC (OLE for Process Control)

OPC is based on the Microsoft interoperability methodology Object Linking and Embedding (OLE). This is an example where an IT standard used within the IT domain and personal computers has been leveraged for use as a control protocol across an industrial network.

In industrial control networks, OPC is limited to operation at the higher levels of the control space, with a dependence on Windows-based platforms. Concerns around OPC begin with the operating system on which it operates. Many of the Windows devices in the operational space are old, not fully patched, and at risk due to a plethora of wellknown vulnerabilities. The dependence on OPC may reinforce that dependence. While newer versions of OPC have enhanced security capabilities, they have also opened up new communications modes, which have both positive and negative security potential. Of particular concern with OPC is the dependence on the Remote Procedure Call (RPC) protocol, which creates two classes of exposure. The first requires you to clearly understand the many vulnerabilities associated with RPC, and the second requires you to identify the level of risk these vulnerabilities bring to a specific network.

6.1.8 International Electro technical Commission (IEC) Protocols

The IEC 61850 standard was created to allow vendor-agnostic engineering of power utility systems, which would, in turn, allow interoperability between vendors and standardized communication protocols. Three message types were initially defined: MMS (Manufacturing Message Specification), GOOSE (Generic Object Oriented Substation Event), and SV (Sampled Values). Web services was a fourth protocol that was added later. Here we provide a short summary of each, but for more information on IEC protocols,

- MMS (61850-8.1): MMS is a client/server protocol that leverages TCP/IP and operates at Layer 3. It provides the same functionality as other SCADA protocols, such as IEC 60870 and Modbus.
- GOOSE (61850-8.1): GOOSE is a Layer 2 protocol that operates via multicast over Ethernet. It allows IEDs to exchange data “horizontally,” between bays and between substations, especially for interlocking, measurement, and tripping signals.
- SV (61850-9-2): SV is a Layer 2 protocol that operates via multicast over Ethernet. It carries voltage and current samples, typically on the process bus, but it can also flow over the station bus.

Both GOOSE and SV operate via a publisher/subscriber model, with no reliability mechanism to ensure that data has been received.

IEC 61850 has several known security deficiencies that could be leveraged by skilled attackers to compromise a control system. Authentication is embedded in MMS, but it is based on clear-text passwords, and authentication is not available in GOOSE or SV. Firmware is typically not signed, which means there is no way to verify its authenticity or integrity. GOOSE and SV have limited message integrity, which makes it relatively easy to impersonate a publisher.

When the standard was first released, there was minimal security capability in these protocols, but this is being addressed by IEC 62351 with the introduction of well-known IT-based security measures, such as certificate exchange. IEC 60870 is widely used for SCADA tele-control in Europe, particularly in the power utility industry, and for widely geographically dispersed control systems. Part 5 of the standard outlines the communication profiles used between endpoints to exchange telecontrol messages. 60870-5-101 is the serial implementation profile, 60870-5-104 is the IP implementation profile, and 60870-5-103 is used for protection equipment. Again, in the early iterations of IEC 60870-5, security was lacking. This is now being addressed by IEC 62351, with the 60870-5-7 security extensions work, applicable to 60870-101 and 60870-104.

6.1.9 Other Protocols

At times, discussions about the security of industrial systems are decidedly focused on industrial control protocols as if they were the sum total of what would be observed or considered. This assumption is narrow-minded and problematic on many levels. In fact, it is highly recommended that a security practitioner passively identify all aspects of the traffic traversing the network prior to implementing any kind of controls or security measures therein. Of particular importance are proper accounting, handling, and understanding of the most basic protocols, transport mechanisms, and foundational elements of any network, including ARP, UDP, TCP, IP, and SNMP.

Some specialized environments may also have other background control protocols. For example, many IoT networks reach all the way to the individual sensors, so protocols such as Constrained Application Protocol (CoAP) and Datagram Transport Layer Security (DTLS) are used, and have to be considered separately from a security perspective.

6.1.10 Device Insecurity

Beyond the communications protocols that are used and the installation base of legacy systems, control and communication elements themselves have a history of vulnerabilities. The security community paid little attention to industrial compute, and as a result, OT systems have not gone through the same “trial by fire” as IT systems. (Figure: Correlation of Industrial Black Hat Presentations with Discovered Industrial Vulnerabilities) shows this graphically by simply overlaying the count of industrial security topics presented at the Black Hat security conference with the number of vulnerabilities reported for industrial control systems. The correlation between presentations on the subject of OT security at Black Hat and the number of vulnerabilities discovered is obvious, including the associated slowing of discoveries.

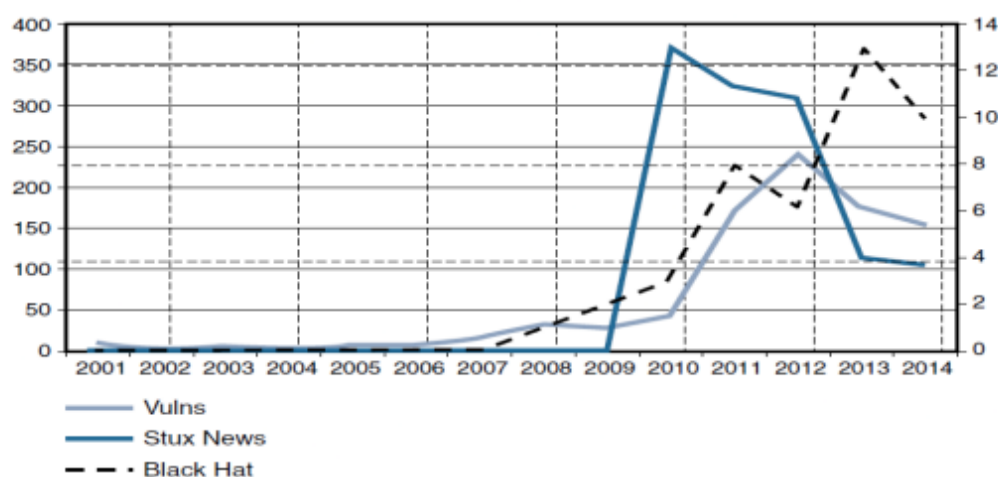


Figure - Correlation of Industrial Black Hat Presentations with Discovered Industrial Vulnerabilities (US Industrial Control Systems Cyber Emergency Response Team (ICS-CERT) <https://ics-cert.us-cert.gov>).

To understand the nature of the device insecurity, it is important to review the history of what vulnerabilities were discovered and what types of devices were affected. A review of the time period 2000 to 2010 reveals that the bulk of discoveries were at the higher levels of the operational network, including control systems trusted to operate plants, transmission systems, oil pipelines, or whatever critical function is in use.

It is not difficult to understand why such systems are frequently found vulnerable. First, many of the systems utilize software packages that can be easily downloaded and worked against. Second, they operate on common hardware and standard operating systems, such as Microsoft Windows. Third, Windows and the components used within those applications are well known to traditionally IT-focused security researchers. There is little need to develop new tools or techniques when those that have long been in place are sufficiently adequate to breach the target's defenses. For example, Stuxnet, the most famous of the industrial compute-based attacks, was initially successful because it was able to exploit a previously unknown vulnerability in Windows.

The ICS vendor community is also lagging behind IT counterparts with regard to security capabilities and practices, as well as cooperation with third-party security researchers. That said, this situation is beginning to get significant industry focus and is improving through a number of recent initiatives designed to formally address security vulnerability and system testing in the industrial environment. While there are some formal standards, such as ISO/IEC 15408 (Common Criteria), ISO/IEC 19790, and a few others, there remain few formal security testing entities. Beyond formal testing, there is little regulatory enforcement of common criteria that address device security testing.

It was not too long ago that the security research community was viewed as a threat, rather than as a valued and often free service to expose potential dangers. While the situation has improved, operational efforts still significantly lag behind IT-based initiatives, such as bug bounty reward programs and advanced vulnerability preparation programs, along the lines of something like the Microsoft Active Protections Program (MAPP). To go a step further, in the industrial realm, there aren't even parallels to the laws that protect individuals' private data. While many states and countries require notification if an individual's personal and financial data is possibly exposed, outside the electrical utility industry, very few laws require the reporting of incidents that may have put lives at risk.

6.1.11 Dependence on External Vendors

While modern IT environments may be outsourcing business operations or relegating certain processing or storage functions to the cloud, it is less common for the original equipment manufacturers of the IT hardware assets to be required to operate the equipment. However, that level of vendor dependence is not uncommon in some industrial spaces.

Direct and on-demand access to critical systems on the plant floor or in the field are sometimes written directly into contracts or are required for valid product warranties. This has clear benefits in many industries as it allows vendors to remotely manage and monitor equipment and to proactively alert the customer if problems are beginning to creep in. While contracts may be written to describe equipment monitoring and management requirements with explicit statements of what type of access is required and under what conditions, they generally fail to address questions of shared liability for security breaches or processes to ensure communication security.

Such vendor dependence and control are not limited to remote access. Onsite management of non-employees that are to be granted compute and network access are also required, but again, control conditions and shared responsibility statements are yet to be observed.

6.1.12 Security Knowledge

In the industrial operations space, the technical investment is primarily in connectivity and compute. It has seen far less investment in security relative to its IT counterpart. According to the research firm Infonetics, the industrial firewall market in 2015 was only approximately 4% the size of the overall firewall market.

Another relevant challenge in terms of OT security expertise is the comparatively higher age of the industrial workforce. According to a study by the US Bureau of Labor, in North America the average age gap between manufacturing workers and other non-farm workers doubled between 2000 and 2012, and the trend shows no sign of reversing. Simultaneously, new connectivity technologies are being introduced in OT industrial environments that require up-to-date skills, such as TCP/IP, Ethernet, and wireless that are quickly replacing serial-based legacy technologies. The rapid expansion of extended communications networks and the need for an industrial controls-aware workforce creates an equally serious gap in security awareness.

This gap in OT security knowledge is actively being addressed. Education for industrial security environments has grown steadily, particularly in the electrical utility space, where regulations such as NERC CIP (CIP 004) and IEC 62351 (01) require ongoing training.

Due to the importance of security in the industrial space, all likely attack surfaces are treated as unsafe. Unfortunately, considering the potential massive public impact of breaching these systems, there remains a healthy paranoia concerning the connection of IT-centric technologies and external connections, despite the massive amount of investment in security in these areas. Bringing industrial networks up to the latest and most secure levels is a slow process due to deep historical cultural and philosophical differences between OT and IT environments.

IoT Security Challenges:

Securing IoT systems is of paramount importance due to the potential risks associated with interconnected devices and data. Some key challenges in IoT security include:

- **Device Vulnerabilities:** Many IoT devices have limited computing power and memory, making them susceptible to security vulnerabilities and exploits.
- **Data Privacy and Protection:** IoT devices collect and transmit sensitive data, such as personal information and user behavior. Ensuring data privacy and protection is crucial to prevent unauthorized access and data breaches.
- **Lack of Standardization:** The IoT ecosystem comprises a wide range of devices from different manufacturers, leading to a lack of uniform security standards across devices.
- **Firmware and Software Updates:** IoT devices often lack mechanisms for automatic firmware and software updates, leaving them vulnerable to known security issues.
- **Authentication and Authorization:** Weak authentication and authorization mechanisms can result in unauthorized access and control of IoT devices.
- **Physical Security:** Many IoT devices are deployed in physically exposed locations, making them susceptible to tampering and physical attacks.
- **Distributed Nature:** The distributed nature of IoT systems can complicate security management and monitoring.

6.2 IoT System's Security Practices

To address the security challenges in IoT systems, the following security practices should be implemented:

- **Secure Boot and Firmware Updates:** Use secure boot processes to ensure that only authenticated firmware is executed. Enable Over-The-Air (OTA) updates for easy and secure firmware updates to patch vulnerabilities.
- **Encryption and Data Protection:** Implement end-to-end encryption for data transmitted between devices and the cloud. Encrypt stored data to safeguard against unauthorized access.
- **Authentication and Access Control:** Use strong authentication mechanisms, such as two-factor authentication, to prevent unauthorized access. Employ access control mechanisms to limit device privileges and actions.
- **Network Security:** Secure communication channels using protocols like TLS/SSL and VPNs. Use firewalls and intrusion detection systems to monitor network traffic and detect anomalies.
- **Secure APIs:** Implement secure APIs to ensure that only authorized applications can interact with IoT devices and access data.
- **Secure Bootstrapping and Provisioning:** Ensure secure device onboarding to the network to prevent unauthorized devices from joining the system.
- **Physical Security Measures:** Implement physical security measures to protect devices from tampering, theft, and unauthorized access.
- **Monitoring and Incident Response:** Employ continuous monitoring of IoT devices and networks to detect potential security threats. Establish an incident response plan to address security breaches promptly.
- **Security Testing and Auditing:** Regularly conduct security testing, such as penetration testing and vulnerability assessments, to identify and address potential weaknesses in the system.
- **Vendor and Supply Chain Security:** Collaborate with trusted vendors and suppliers who follow robust security practices to ensure the security of components used in IoT devices.

REFERENCE:

1. Arshdeep Bahga, Vijay Madisetti, "Internet of Things (A Hands-on-Approach)". University Press India Pvt. Ltd, 2015.
2. David Hanes, Gonzalo Salgueiro, Patrick Grossetete, Robert Barton, Jerome Henry, "IoT Fundamentals: Networking Technologies, Protocols, and Use Cases for the Internet of Things", Pearson Education (Cisco Press Indian Reprint).
3. Raj Kamal, "Internet of Things: Architecture and Design Principles", McGraw Hill
4. Gary Smart, "Practical Python Programming for IoT ISBN-10:1838982469
5. Gaston C. Hillar Internet of Things with Python, ISBN-10
- 6 <https://www.pdfdrive.com/big-data-analytics-a-hands-on-approach-d158549112.html>
7. <https://www.airtel.in/blog/business/physical-and-logical-design-of-iot/>
8. https://mite.ac.in/wp-content/uploads/2021/04/iot_module4.pdf
- 9 <https://www.pdfdrive.com/iot-fundamentals-networking-technologies-protocols-and-use-cases-for-the-internet-of-things-d57499885.html>
10. <https://www.pdfdrive.com/internet-of-things-iot-technologies-applications-challenges-and-solutions-e158467863.html>